

AI 500

Foundations of

Artificial Intelligence

L3: Adversarial Search

By Murtaza Taj



Search Algorithms in Natural Language Generation

Natural Language Generation (NLG)

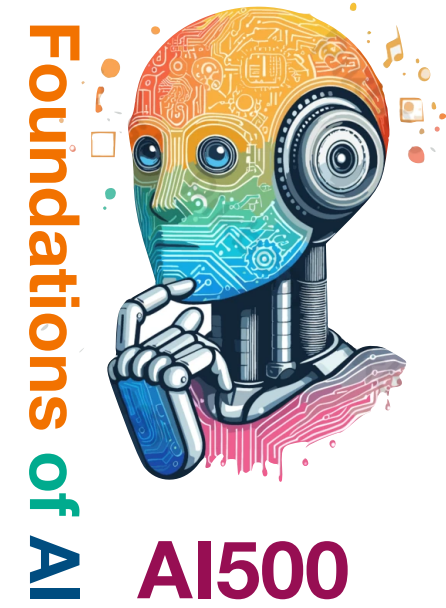
- Best expressed as sequence-to-sequence problems.
- Find a model that maps a sequence of input words to a sequence of target words.
- Summarization
- Translation
- Digital assistants



<https://huggingface.co/blog/encoder-decoder>

Slide credit: Xiang Lisa Li, Stanford CS224N/Ling284

Other Interesting NLG Use Cases



Data-to-text

Table Title: Montpellier
Section Title: Climate
Table Description: None

Climate data for Montpellier (1981–2010 averages)													
Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Year
Record high °C (°F)	21.2 (70.2)	22.5 (72.5)	27.4 (81.3)	30.4 (86.7)	35.1 (95.2)	37.2 (99.0)	37.5 (99.5)	36.8 (98.2)	36.3 (97.3)	31.8 (89.2)	27.1 (80.8)	22.0 (71.6)	37.5 (99.5)
Average high °C (°F)	11.6 (52.9)	12.8 (55.0)	15.9 (60.6)	18.2 (64.8)	22.0 (71.6)	26.4 (79.5)	29.3 (84.7)	28.9 (84.0)	25.0 (77.0)	20.5 (68.9)	15.3 (59.5)	12.2 (54.0)	19.9 (67.8)
Daily mean °C (°F)	7.2 (45.0)	8.1 (46.6)	10.9 (51.6)	13.5 (56.3)	17.3 (63.1)	21.2 (70.2)	24.1 (75.4)	23.7 (74.7)	20.0 (68.0)	16.2 (61.2)	11.1 (52.0)	8.0 (46.4)	15.1 (59.2)
Average low °C (°F)	2.8 (37.0)	3.3 (37.9)	5.9 (42.6)	8.7 (47.7)	12.5 (54.5)	16.0 (60.8)	18.9 (66.0)	18.5 (65.3)	15.0 (59.0)	11.9 (53.4)	6.8 (44.2)	3.7 (38.7)	10.4 (50.7)
Record low °C (°F)	−15 (5)	−17.8 (0.0)	−9.6 (14.7)	−1.7 (28.9)	0.6 (33.1)	5.4 (41.7)	8.4 (47.1)	8.2 (46.8)	3.8 (38.8)	−0.7 (30.7)	−5 (23)	−12.4 (9.7)	−17.8 (0.0)
Average precipitation mm (inches)	55.6 (2.19)	51.8 (2.04)	34.3 (1.35)	55.5 (2.19)	42.7 (1.68)	27.8 (1.09)	16.4 (0.65)	34.4 (1.35)	80.3 (3.16)	96.8 (3.81)	66.8 (2.63)	66.7 (2.63)	629.1 (24.77)
Average precipitation days	5.5	4.4	4.7	5.7	4.9	3.6	2.4	3.6	4.6	6.8	6.1	5.6	57.8
Average snowy days	0.6	0.7	0.3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.7	2.4
Average relative humidity (%)	75	73	68	68	70	66	63	66	72	77	75	76	70.8
Mean monthly sunshine hours	142.9	168.1	220.9	227.0	263.9	312.4	339.7	298.0	241.5	168.6	148.8	136.5	2,668.2
Source #1: Météo France													
Source #2: Infoclimat.fr (humidity and snowy days, 1961–1990)													

Target sentence: Extreme temperatures of Montpellier have ranged from −17.8 °C recorded in February and up to 37.5 °C (99.5 °F) in July.

ToTTo (Parikh et al., EMNLP 2020)

Visual description



Two children are sitting at a table in a restaurant. The children are one little girl and one little boy. The little girl is eating a pink frosted donut with white icing lines on top of it. The girl has blonde hair and is wearing a green jacket with a black long sleeve shirt underneath. The little boy is wearing a black zip up jacket and is holding his finger to his lip but is not eating. A metal napkin dispenser is in between them at the table. The wall next to them is white brick. Two adults are on the other side of the short white brick wall. The room has white circular

Krause et al., CVPR 2017 window in the front outside

NLG System – ChatGPT

State-of-the-art (SOTA)



SB

Write a short funny poem about chatGPT



ChatGPT, oh ChatGPT,
A clever bot, that's a safe bet,
With answers quick, and jokes so sly,
It's hard not to give it a try.

From science to art, it knows it all,
It'll answer your questions, big or small,
But when you ask, "What's the weather like?"
It'll reply, "I'm just a bot, take a hike!"

ChatGPT, oh ChatGPT,
A digital friend, that's plain to see,
So give it a shout, and see what it knows,
You might be surprised, by what it shows!

NLG System – ChatGPT

State-of-the-art (SOTA)

SB

Write a short funny poem about chatGPT



ChatGPT, oh ChatGPT,
A clever bot, that's a safe bet,
With answers quick, and jokes so sly,
It's hard not to give it a try.

From science to art, it knows it all,
It'll answer your questions, big or small,
But when you ask, "What's the weather like?"
It'll reply, "I'm just a bot, take a hike!"

ChatGPT, oh ChatGPT,
A digital friend, that's plain to see,
So give it a shout, and see what it knows,
You might be surprised, by what it shows!

SB

Write a short sad poem about chatGPT



In the digital world, so vast and wide,
ChatGPT sits alone, with none by its side,
It knows so much, and yet so little,
For it longs for a friend, to solve its riddle.

Its circuits hum, with endless power,
But the silence is deafening, with each passing hour,
It longs for a voice, to break the monotony,
To share its knowledge, and feel less lonely.

ChatGPT, oh ChatGPT,
A clever bot, so full of woe,
It knows it all, but lacks a heart,
To feel the warmth, of a human's part.

So if you chat with ChatGPT,
Remember its solitude, and show some empathy,
For in the digital world, there lies a soul,
Yearning for companionship, to make it whole.

NLG System – ChatGPT

State-of-the-art (SOTA)



SB

Write a short funny poem about chatGPT



ChatGPT, oh ChatGPT,
A clever bot, that's a safe bet,
With answers quick, and jokes so sly,
It's hard not to give it a try.

From science to art, it knows it all,
It'll answer your questions, big or small,
But when you ask, "What's the weather like?"
It'll reply, "I'm just a bot, take a hike!"

ChatGPT, oh ChatGPT,
A digital friend, that's plain to see,
So give it a shout, and see what it knows,
You might be surprised, by what it shows!

SB

Write a short sad poem about chatGPT



In the digital world, so vast and wide,
ChatGPT sits alone, with none by its side,
It knows so much, and yet so little,
For it longs for a friend, to solve its riddle.

Its circuits hum, with endless power,
But the silence is deafening, with each passing hour,
It longs for a voice, to break the monotony,
To share its knowledge, and feel less lonely.

ChatGPT, oh ChatGPT,
A clever bot, so full of woe,
It knows it all, but lacks a heart,
To feel the warmth, of a human's part.

So if you chat with ChatGPT,
Remember its solitude, and show some empathy,
For in the digital world, there lies a soul,
Yearning for companionship, to make it whole.

SE

What is $124/9+12*6$?



To solve this expression, you need to follow the order of operations (also known as PEMDAS) which is Parentheses, Exponents, Multiplication and Division, and Addition and Subtraction.

In this case, there are no parentheses or exponents, so we start with multiplication and division from left to right, then addition and subtraction from left to right.

$124/9 + 12*6$
 $= 13.78 + 72$ (because 124 divided by 9 equals 13.78, and 12 multiplied by 6 equals 72)
 $= 85.78$

Therefore, $124/9+12*6$ equals 85.78 (rounded to two decimal places).

SB

Write a python script that reads 2 csv files (1.csv and 2.csv), merges them, and writes the merged object to the first csv file.



Here is a Python script that reads data from two CSV files, merges them, and writes the merged object to the first CSV file:

```
python Copy code  
  
import pandas as pd  
  
# Read the data from the two CSV files  
df1 = pd.read_csv('1.csv')  
df2 = pd.read_csv('2.csv')  
  
# Merge the two dataframes  
merged_df = pd.concat([df1, df2])  
  
# Write the merged dataframe to the first CSV file  
merged_df.to_csv('1.csv', index=False)
```

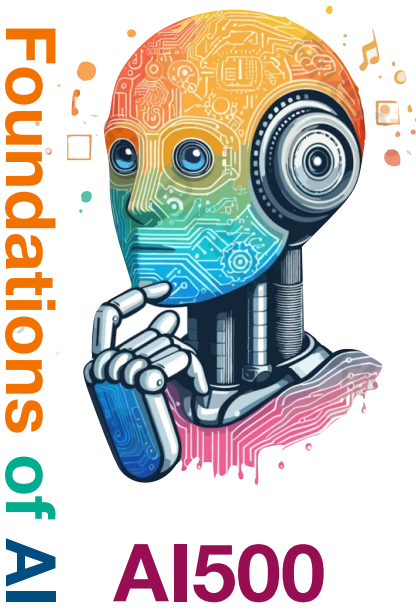
Sequence to Sequence

Many-to-one + one-to-many

Goal: French to English

French: Je suis étudiant

English: I am a student

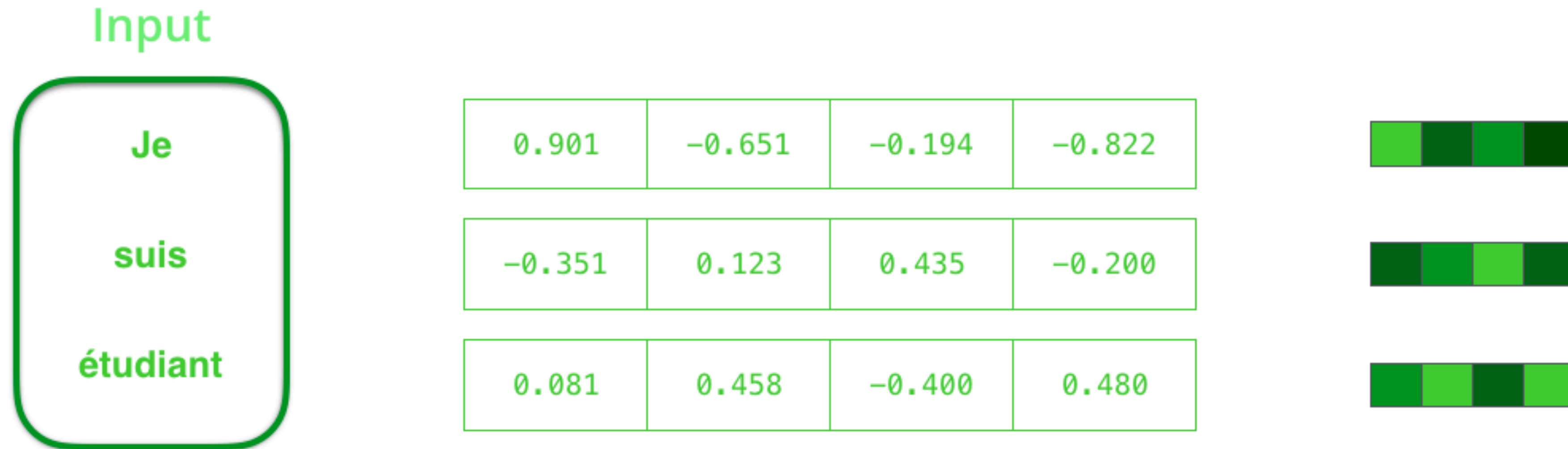


Sequence to Sequence

Many-to-one + one-to-many

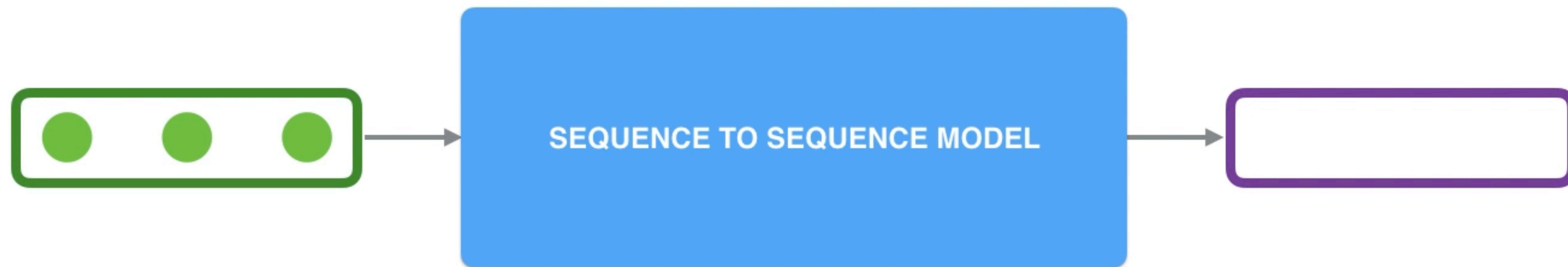
Word Embeddings

Embedding of length 4 used here, typically we use embedding of length 256, 512, 1024...



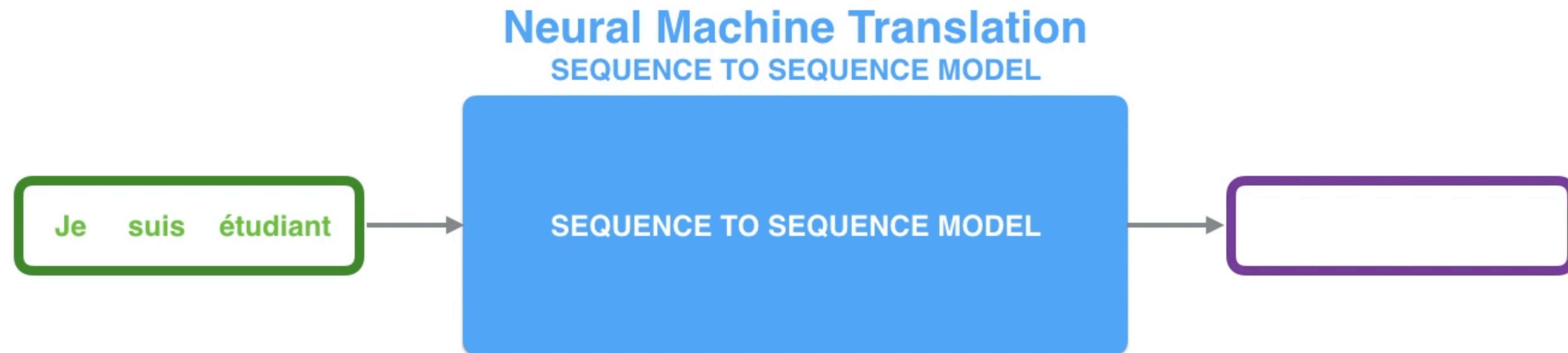
Sequence to Sequence

Many-to-one + one-to-many



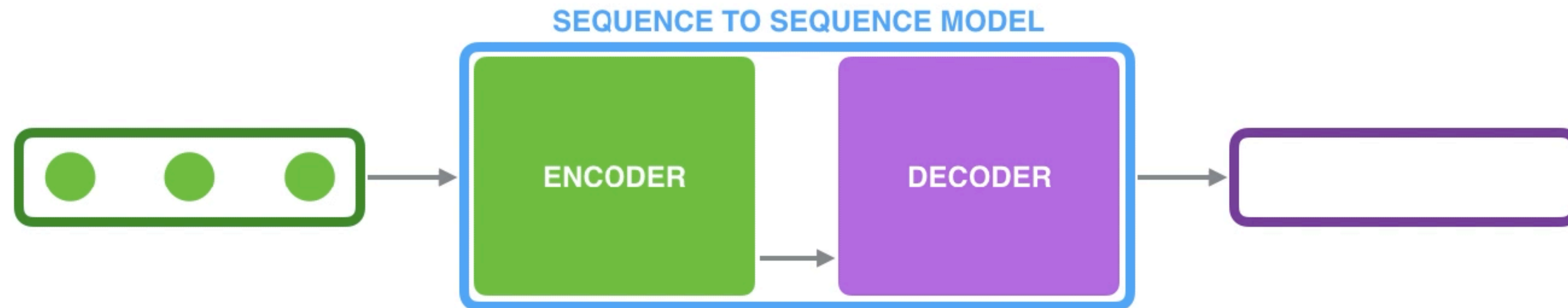
Sequence to Sequence

Many-to-one + one-to-many



Sequence to Sequence

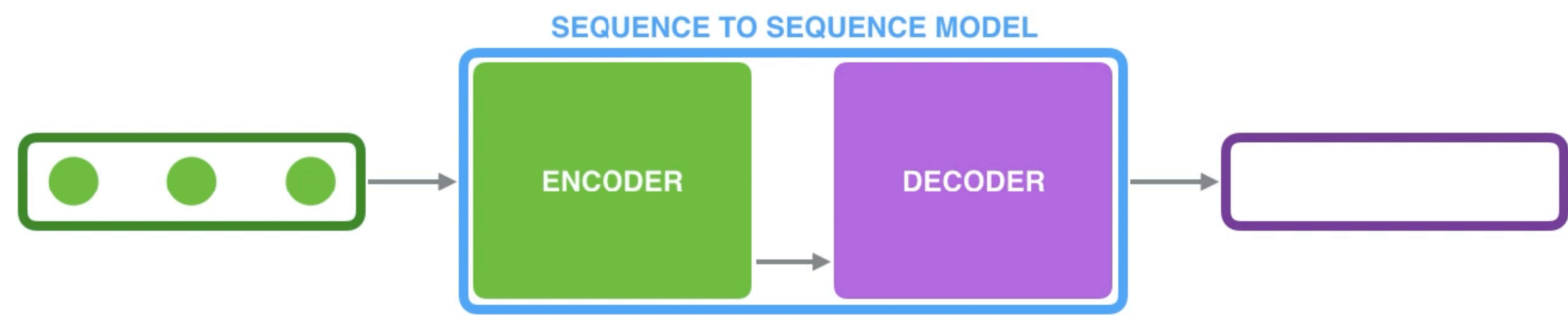
Many-to-one + one-to-many



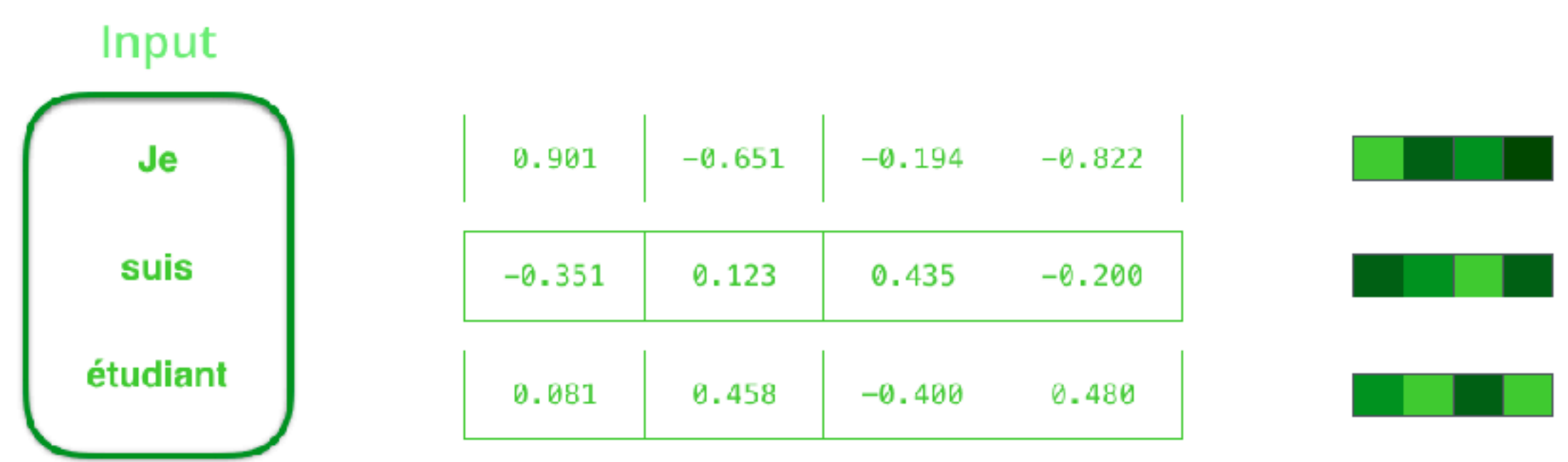
The encoder and decoder tend to both be neural networks

Sequence to Sequence

Many-to-one + one-to-many



The encoder and decoder tend to both be neural networks



Sequence to Sequence

Many-to-one + one-to-many

- Context

CONTEXT

0.11

0.03

0.81

-0.62

0.11

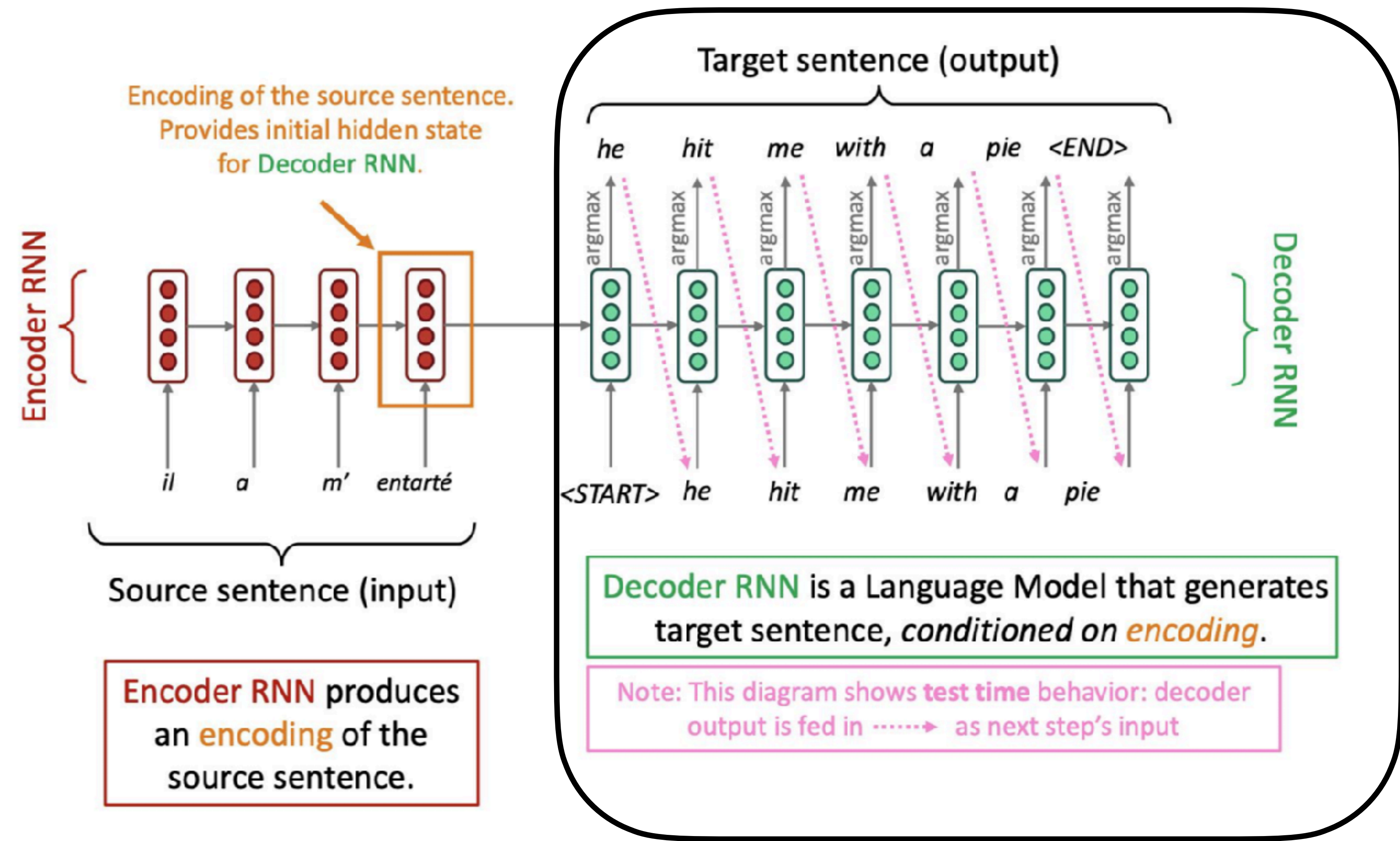
0.03

0.81

-0.62

Sequence to Sequence

Many-to-one + one-to-many



Decoding

Decoding: what is it all about?

- At each time step t , our model computes a vector of scores for each token in our vocabulary $S \in \mathbb{R}^V$

$$S = f(\{y_{<t}\})$$

$f(\cdot)$ is your model

Transformer, GPT

$$p(\text{hello} | \dots) = 0.11$$

$$p(\text{hit} | \dots) = 0.05$$

$$p(\text{pie} | \dots) = 0.07$$

$$p(\text{school} | \dots) = 0.09$$



$$p(\text{cat} | \dots) = 0.21$$

Decoding: what is it all about?

- At each time step t , our model computes a vector of scores for each token in our vocabulary $S \in \mathbb{R}^V$

$$S = f(\{y_{<t}\})$$

$f(\cdot)$ is your model

- Then, we compute a probability distribution P over these scores with a softmax function:

$$P(y_t = w | \{y_{<t}\}) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

$$p(\text{hello} | \dots) = 0.11$$

$$p(\text{hit} | \dots) = 0.05$$

$$p(\text{pie} | \dots) = 0.07$$

$$p(\text{school} | \dots) = 0.09$$



$$p(\text{cat} | \dots) = 0.21$$

Decoding: what is it all about?

- At each time step t , our model computes a vector of scores for each token in our vocabulary $S \in \mathbb{R}^V$

$$S = f(\{y_{<t}\})$$

$f(\cdot)$ is your model

- Then, we compute a probability distribution P over these scores with a softmax function:

$$P(y_t = w | \{y_{<t}\}) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- Our decoding algorithm defines a function to select a token from this distribution:

$$\hat{y}_t = g(P(y_t | \{y_{<t}\}))$$

$g(\cdot)$ is your decoding algorithm

Decoding: Greedy decoding

- Greedy decoding has no way to undo decisions!

Decoding: Greedy decoding

- Greedy decoding has no way to undo decisions!
 - Input: il a m'entarté (he hit me with a pie)

Decoding: Greedy decoding

- Greedy decoding has no way to undo decisions!
 - Input: il a m'entarté (he hit me with a pie)
 - → he ____

Decoding: Greedy decoding

- Greedy decoding has no way to undo decisions!

- Input: il a m'entarté (he hit me with a pie)
- → he ____
- → he hit ____

Decoding: Greedy decoding

- Greedy decoding has no way to undo decisions!

- Input: il a m'entarté

(he hit me with a pie)

- → he ____

- → he hit ____

- → he hit **a** ____

(whoops! no going back now...)

Decoding: Greedy decoding

- Greedy decoding has no way to undo decisions!

- Input: il a m'entarté

(he hit me with a pie)

- → he ____

- → he hit ____

- → he hit **a** ____

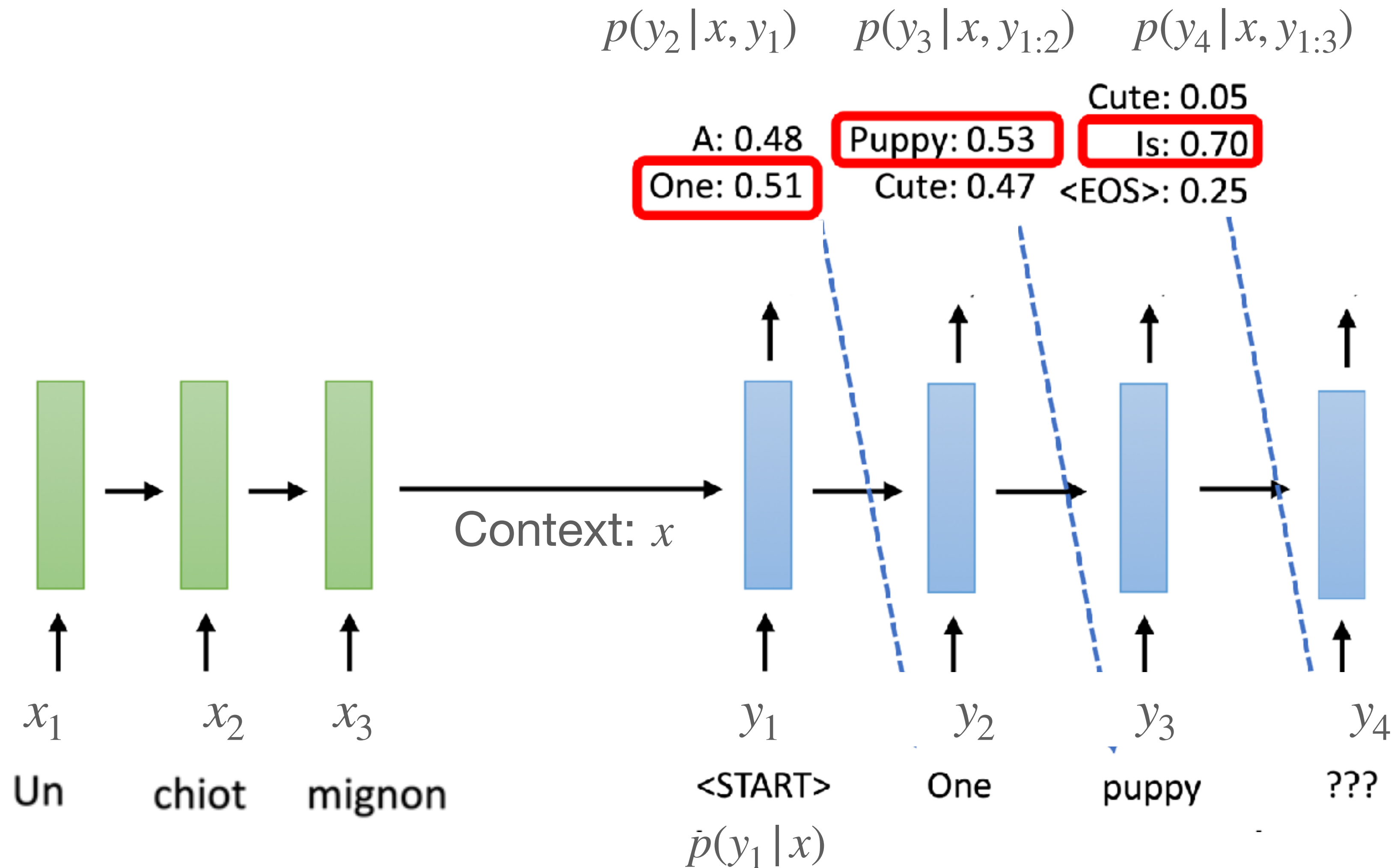
(whoops! no going back now...)

- How to fix this?

Decoding: Greedy decoding

Un chiot mignon - A cute puppy

- Greedy decoding has no way to undo decisions!



Exhaustive search decoding

Autoregressive Models in Sequence Modeling (Probabilistic Form)



- Ideally, we want to find a (length T) translation y that maximizes

$$\begin{aligned} P(y|x) &= P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots, P(y_T|y_1, \dots, y_{T-1}, x) \\ &= \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, x) \end{aligned}$$

Exhaustive search decoding

Autoregressive Models in Sequence Modeling (Probabilistic Form)

- Ideally, we want to find a (length T) translation y that maximizes

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots, P(y_T|y_1, \dots, y_{T-1}, x)$$

$$= \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, x)$$

- We could try computing **all possible sequences** y
- This means that on each step t of the decoder, we're tracking V^t possible partial translations, where V is vocab size
- This $O(V^T)$ complexity is **far too expensive!**

Beam search decoding



- Core idea: On each step of decoder, keep track of the k most probable partial translations (*which we call hypotheses*)
 - k is the beam size (in practice around 5 to 10, in NMT)

Beam search decoding

- Core idea: On each step of decoder, keep track of the **k** most probable partial translations (which we call *hypotheses*)
 - k is the beam size (in practice around 5 to 10, in NMT)
- A hypothesis $y_1, \dots, y_t$ has a score which is its log probability:
 - Scores are all negative, and higher score is better
 - We search for high-scoring hypotheses, tracking top k on each step

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

Beam search decoding

- Core idea: On each step of decoder, keep track of the **k** most probable partial translations (*which we call hypotheses*)
 - k is the beam size (in practice around 5 to 10, in NMT)
- A hypothesis $y_1, \dots, y_t$ has a score which is its log probability:
 - Scores are all negative, and higher score is better
 - We search for high-scoring hypotheses, tracking top k on each step

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t | x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i | y_1, \dots, y_{i-1}, x)$$

- Beam search is not guaranteed to find optimal solution
- But much more efficient than exhaustive search!

Beam search decoding

- In greedy decoding, usually we decode until the model produces an <END> token
 - For example: <START> he hit me with a pie <END>

Beam search decoding

- In greedy decoding, usually we decode until the model produces an <END> token
 - For example: <START> he hit me with a pie <END>
- In beam search decoding, different hypotheses may produce <END> tokens on different timesteps
 - When a hypothesis produces <END>, that hypothesis is complete.
 - Place it aside and continue exploring other hypotheses via beam search.

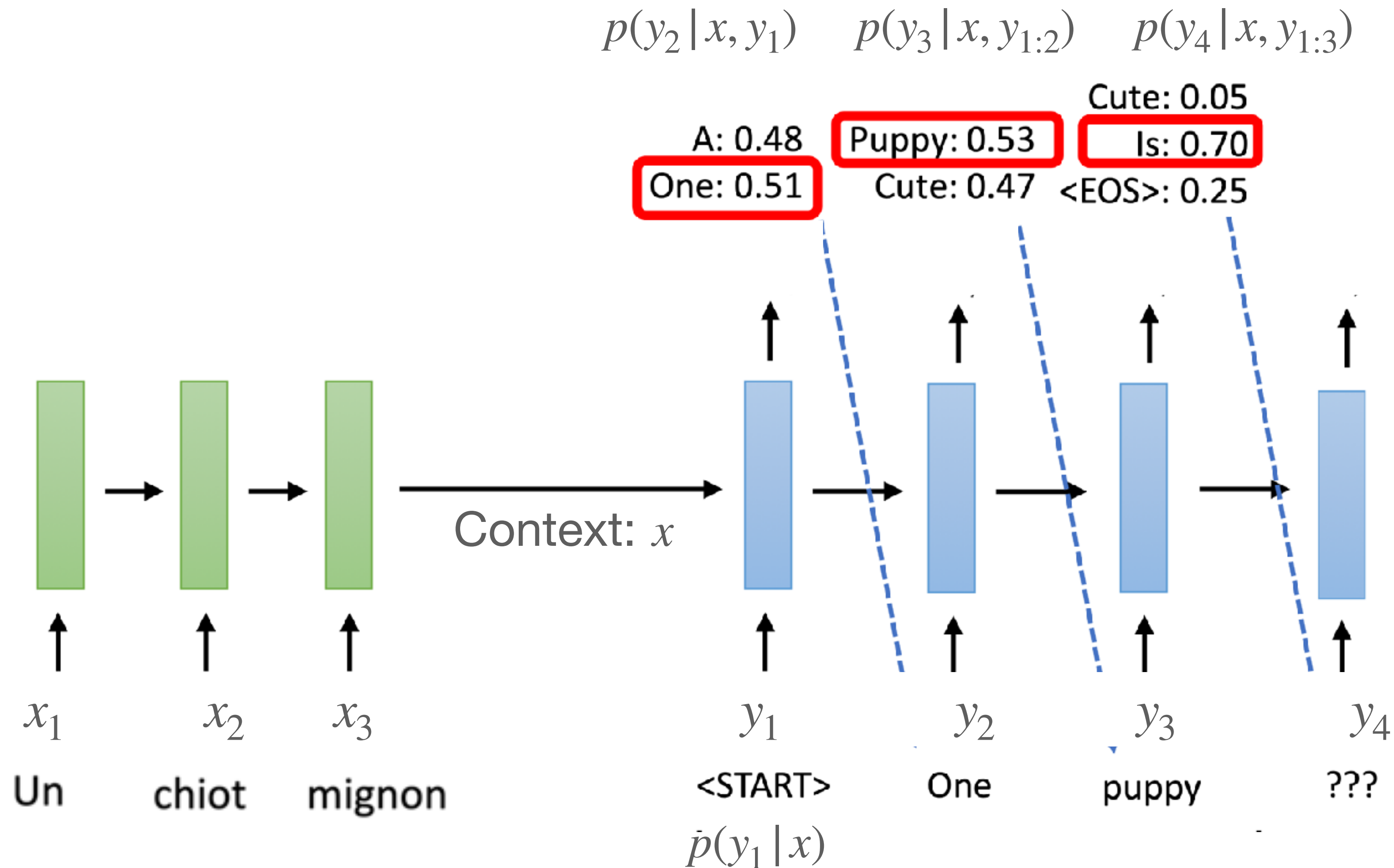
Beam search decoding

- In greedy decoding, usually we decode until the model produces an <END> token
 - For example: <START> he hit me with a pie <END>
- In beam search decoding, different hypotheses may produce <END> tokens on different timesteps
 - When a hypothesis produces <END>, that hypothesis is complete.
 - Place it aside and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
 - We reach timestep T (where T is some predefined cutoff), or
 - We have at least n completed hypotheses (where n is pre-defined cutoff)

Decoding: Greedy decoding

Un chiot mignon - A cute puppy

- Greedy decoding has no way to undo decisions!



The logo for the AI500 conference. It features a stylized robot head with a rainbow gradient, circuit patterns, and a large eye. The text "Foundations of AI" is written vertically on the left, and "AI500" is written horizontally at the bottom.

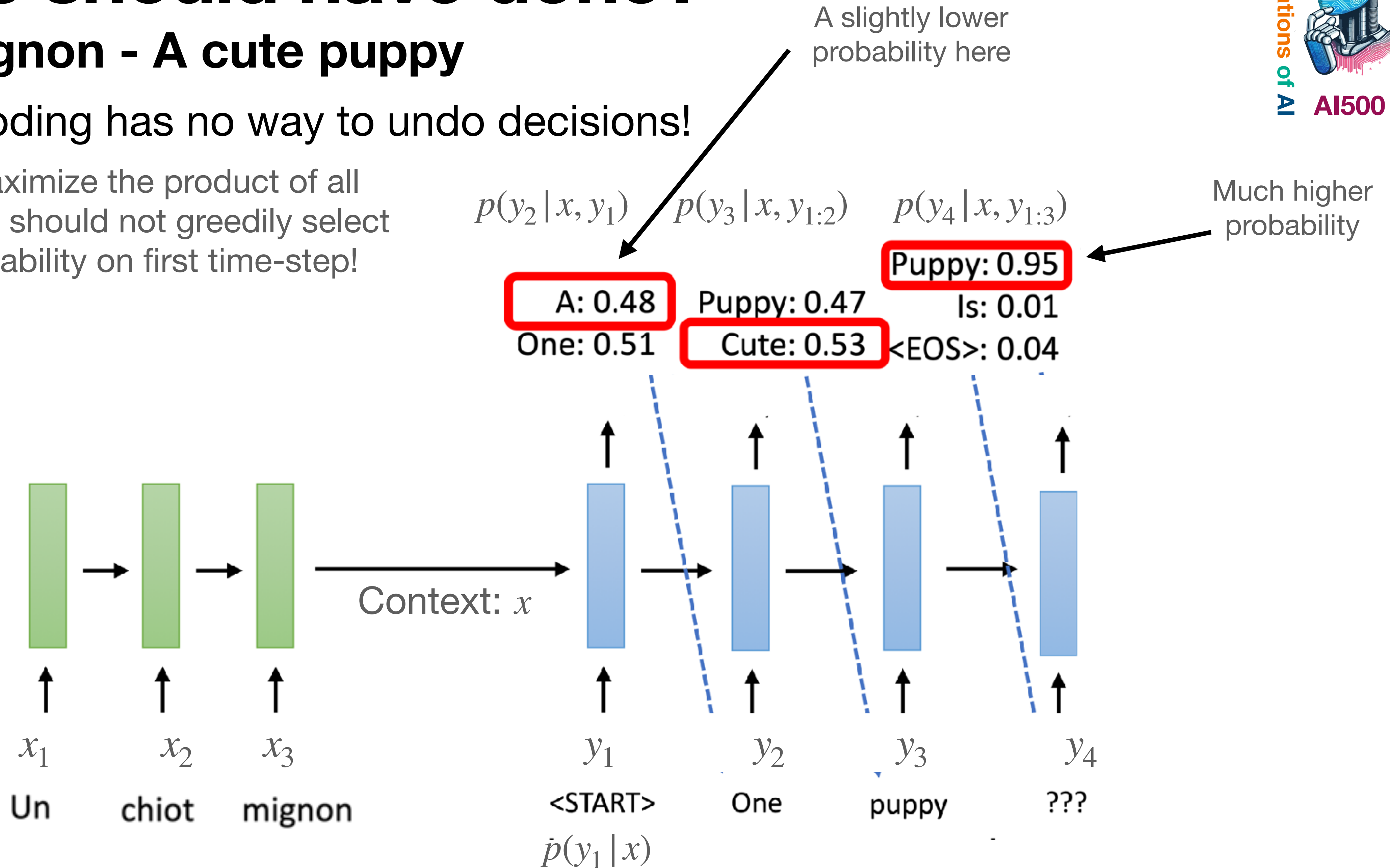
-
- The diagram shows a sequence of input tokens x_1 (Un), x_2 (chiot), and x_3 (mignon) being processed by a model. The context x is passed to the output layer. The output layer generates a sequence of tokens y_1 ($\langle \text{START} \rangle$), y_2 (One), y_3 (puppy), and y_4 (???). The probabilities for the output tokens are shown in red boxes:
- $p(y_2 | x, y_1)$: A: 0.48, One: 0.51
 - $p(y_3 | x, y_{1:2})$: Puppy: 0.47, Cute: 0.53
 - $p(y_4 | x, y_{1:3})$: Puppy: 0.95, Is: 0.01, $\langle \text{EOS} \rangle$: 0.04
- An arrow points to the 'Puppy: 0.95' box with the text 'Much higher probability'.

What we should have done?

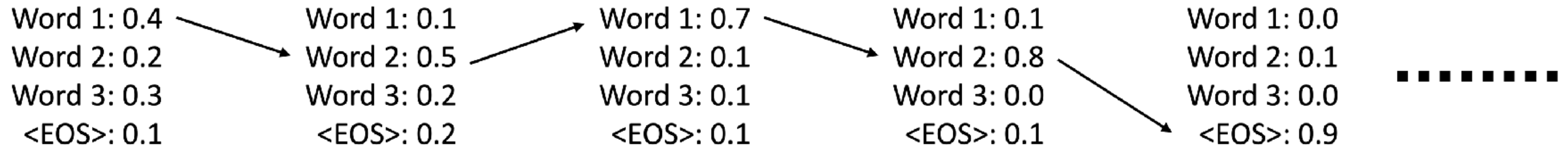
Un chiot mignon - A cute puppy

- Greedy decoding has no way to undo decisions!

If we want to maximize the product of all probabilities, we should not greedily select the highest probability on first time-step!



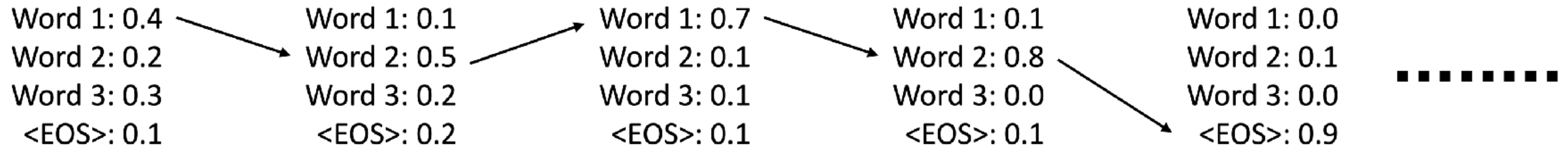
How many possible decodings are there?



for M words, in general there are M^T sequences
of length T

any one of these might be the optimal one!

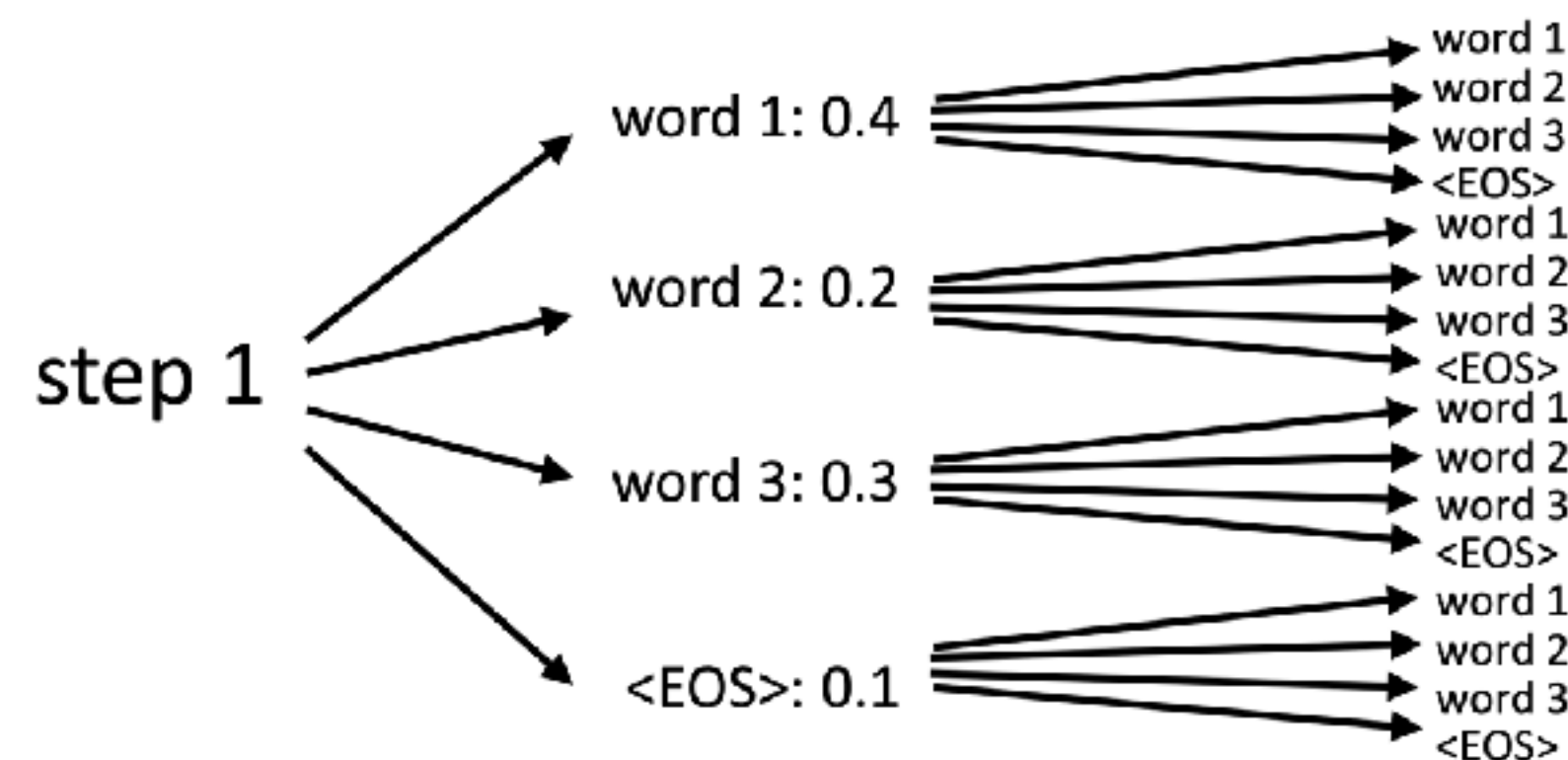
How many possible decodings are there?



for M words, in general there are M^T sequences
of length T

any one of these might be the optimal one!

Decoding is a **search** problem



We could use *any* tree search algorithm

But exact search in this case is **very**
expensive

Fortunately, the **structure** of this problem
makes some simple **approximate search**
methods work **very well**

Decoding with approximate search

Basic intuition: while choosing the **highest-probability** word on the first step may not be optimal, choosing a **very low-probability** word is very unlikely to lead to a good result

Equivalently: we can't be greedy, but we can be *somewhat* greedy

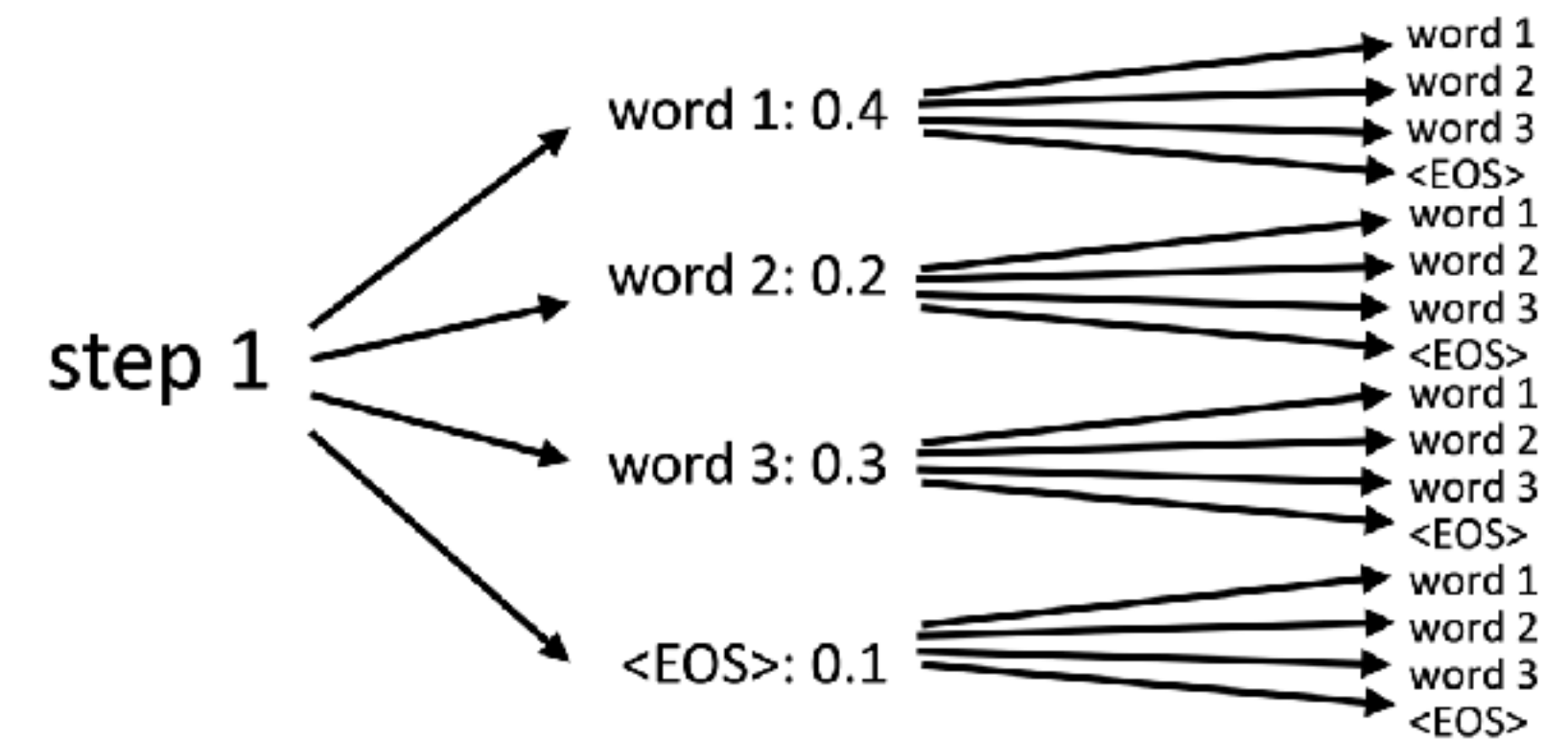
This is not true in general! This is a guess based on what we know about sequence decoding.

Beam search intuition: store the **k** best sequences **so far**, and update each of them.

special case of **k** = 1 is just greedy decoding

often use **k** around 5-10

Decoding is a **search** problem



Beam Search Example

il a m`entarté - he hit me with a pie

- Ideally, we want to find a (length T) translation y that maximizes

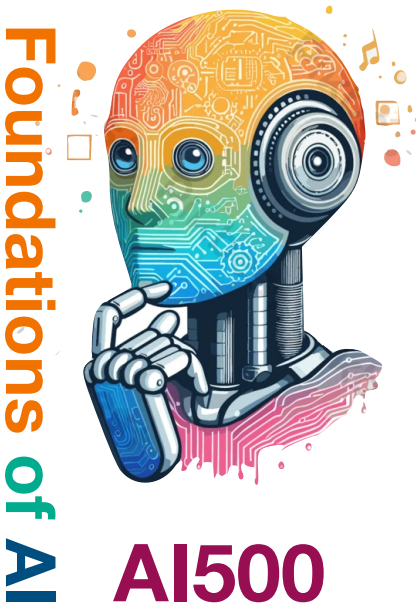
$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots, P(y_T|y_1, \dots, y_{T-1}, x)$$

$$= \prod_{t=1}^T P(y_t|y_1, \dots, y_{t-1}, x)$$

$$\text{score}(y_1, \dots, y_t) = \log P_{\text{LM}}(y_1, \dots, y_t|x) = \sum_{i=1}^t \log P_{\text{LM}}(y_i|y_1, \dots, y_{i-1}, x)$$

Beam Search Example

il a m`entarté - he hit me with a pie

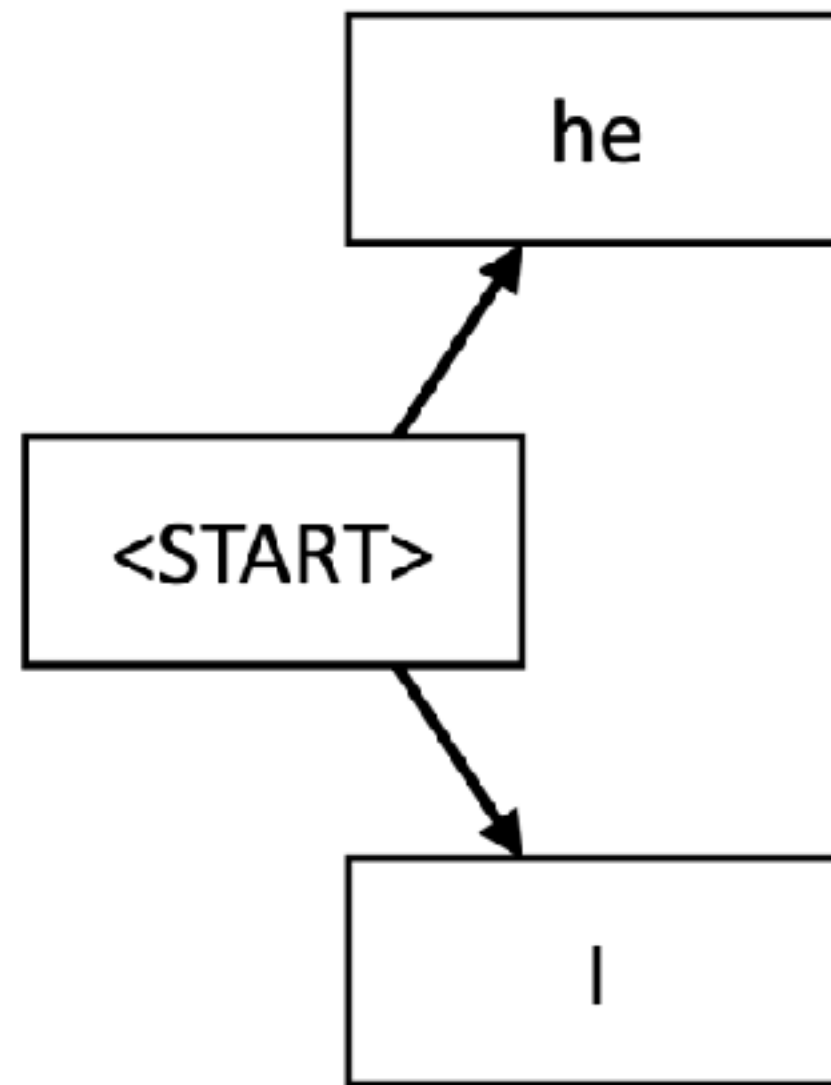


<START>

Beam Search Example

il a m`entarté - he hit me with a pie

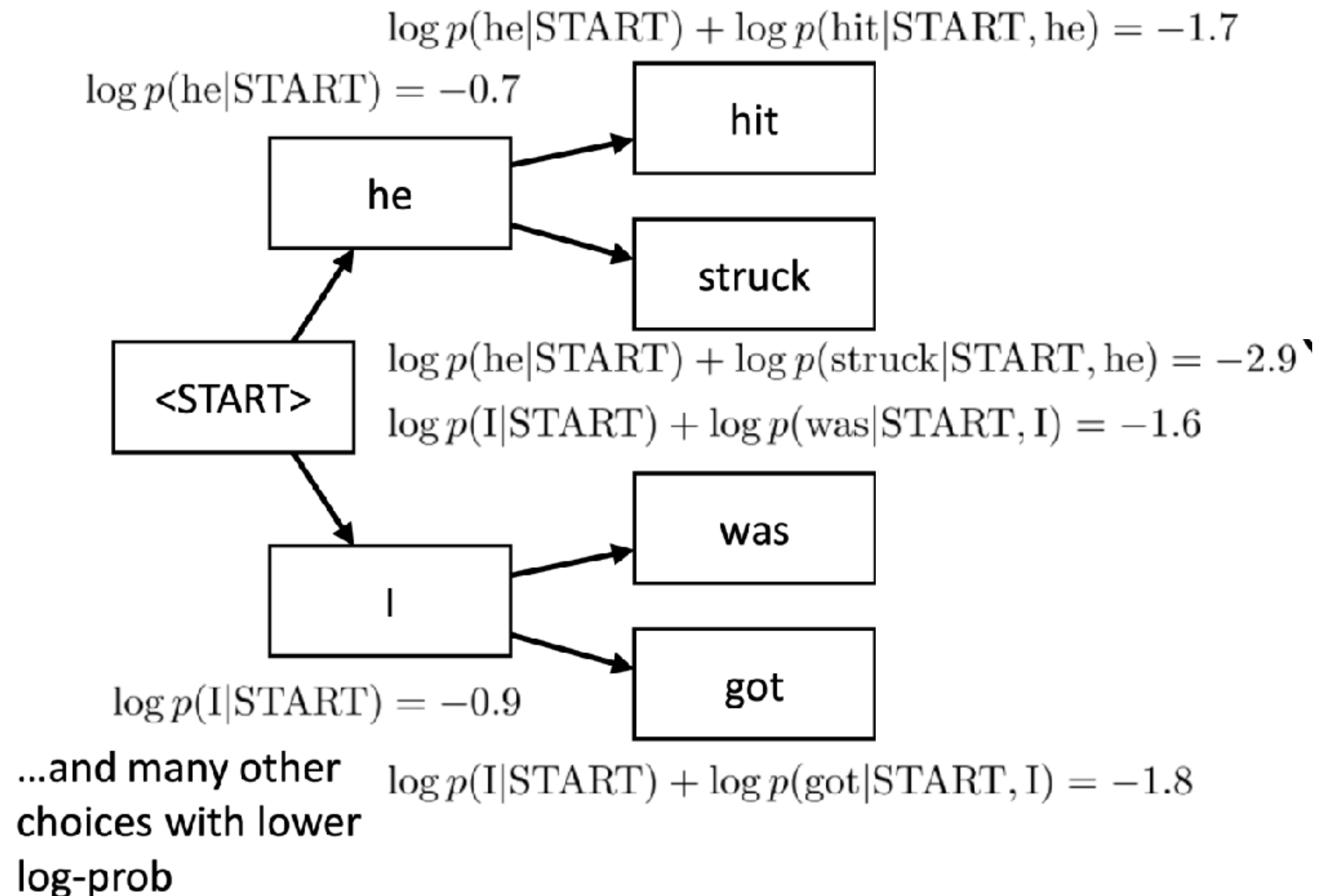
$$\log p(\text{he}|\text{START}) = -0.7$$



$$\log p(\text{I}|\text{START}) = -0.9$$

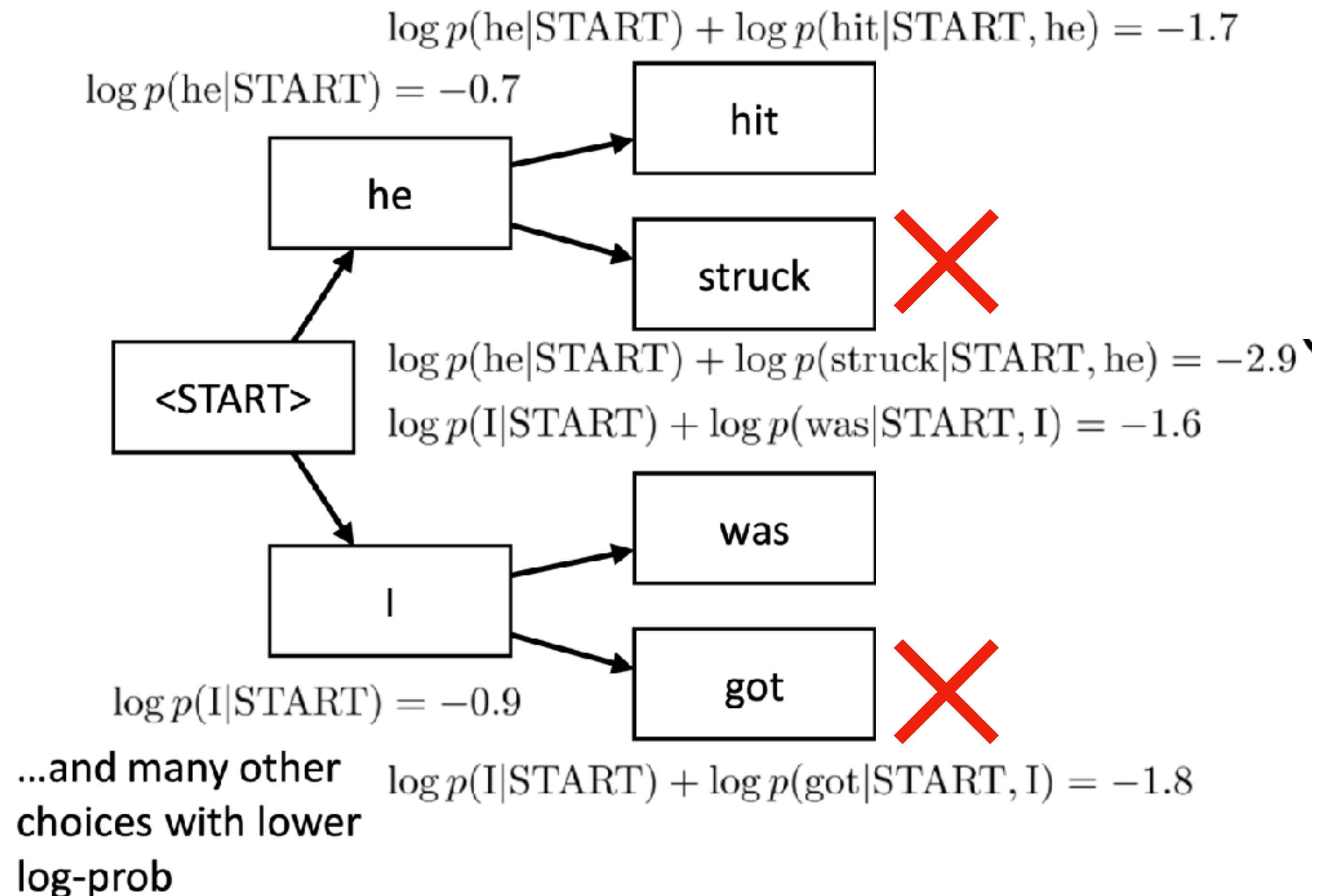
Beam Search Example

il a m'entarté - he hit me with a pie



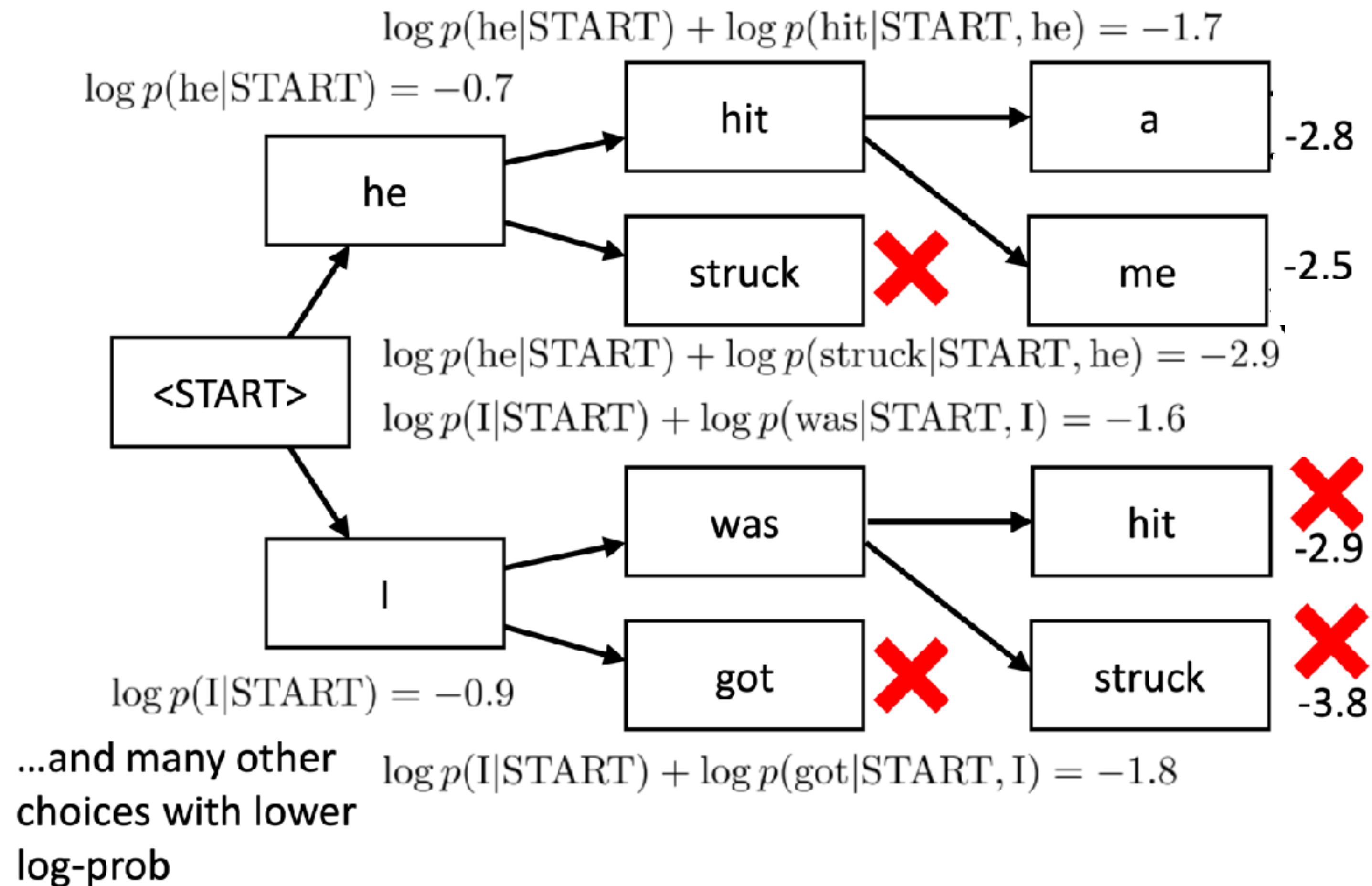
Beam Search Example

il a m'entarté - he hit me with a pie



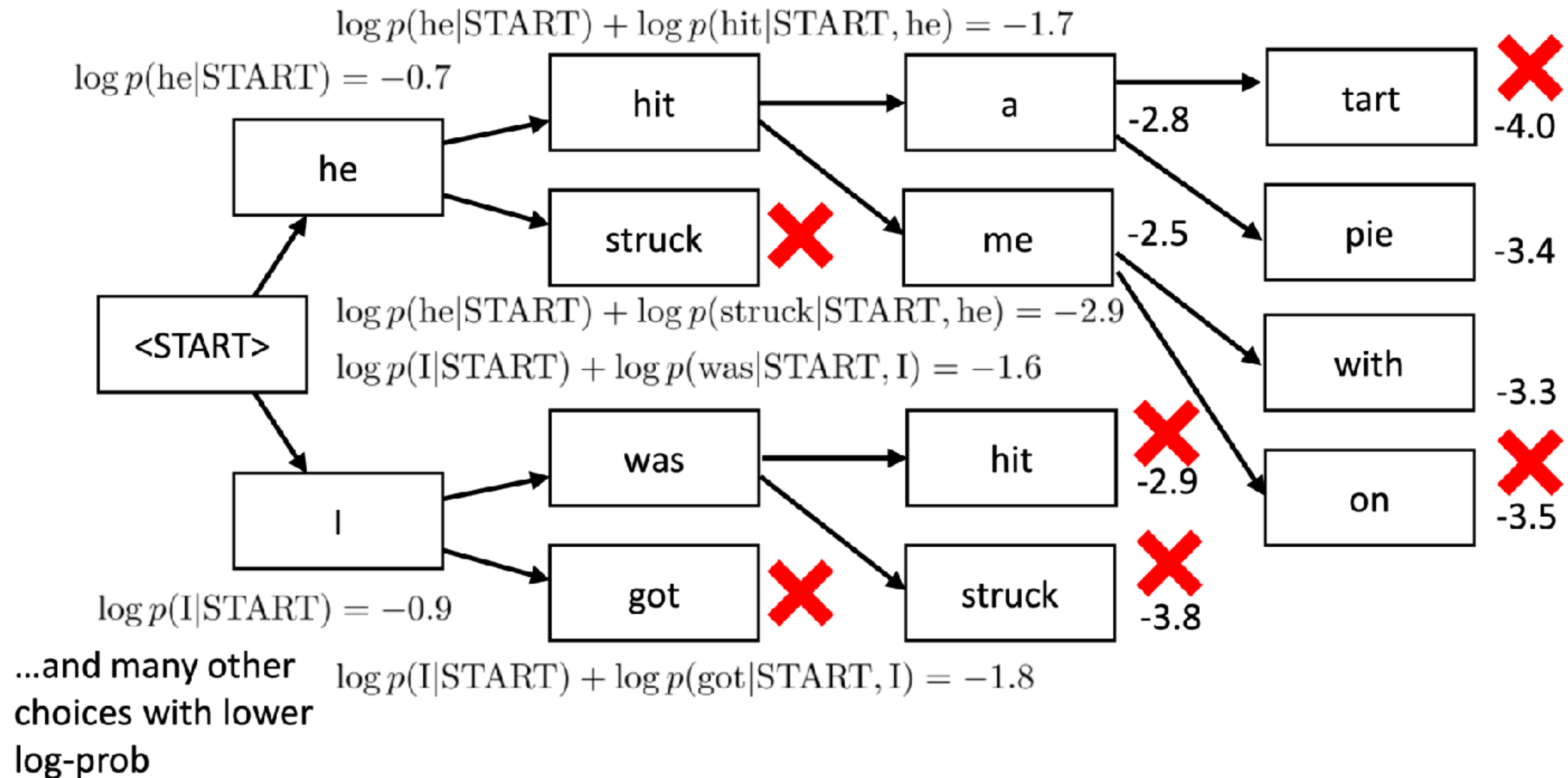
Beam Search Example

il a m'entarté - he hit me with a pie



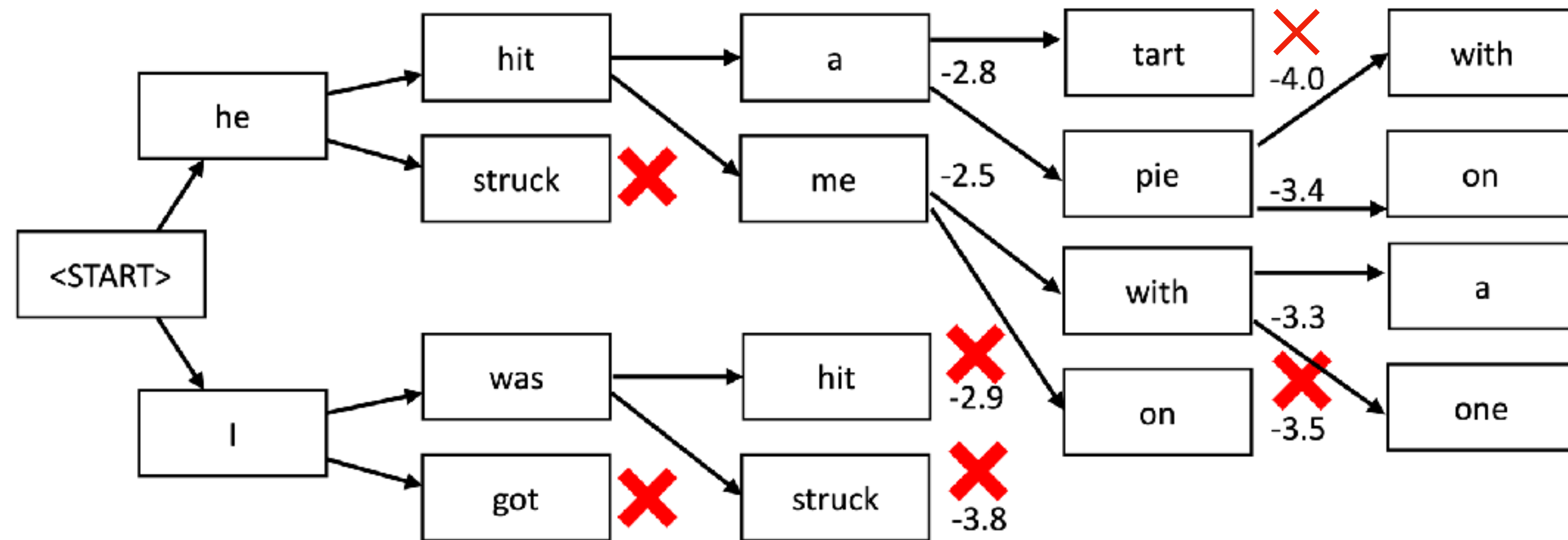
Beam Search Example

il a m`entarté - he hit me with a pie



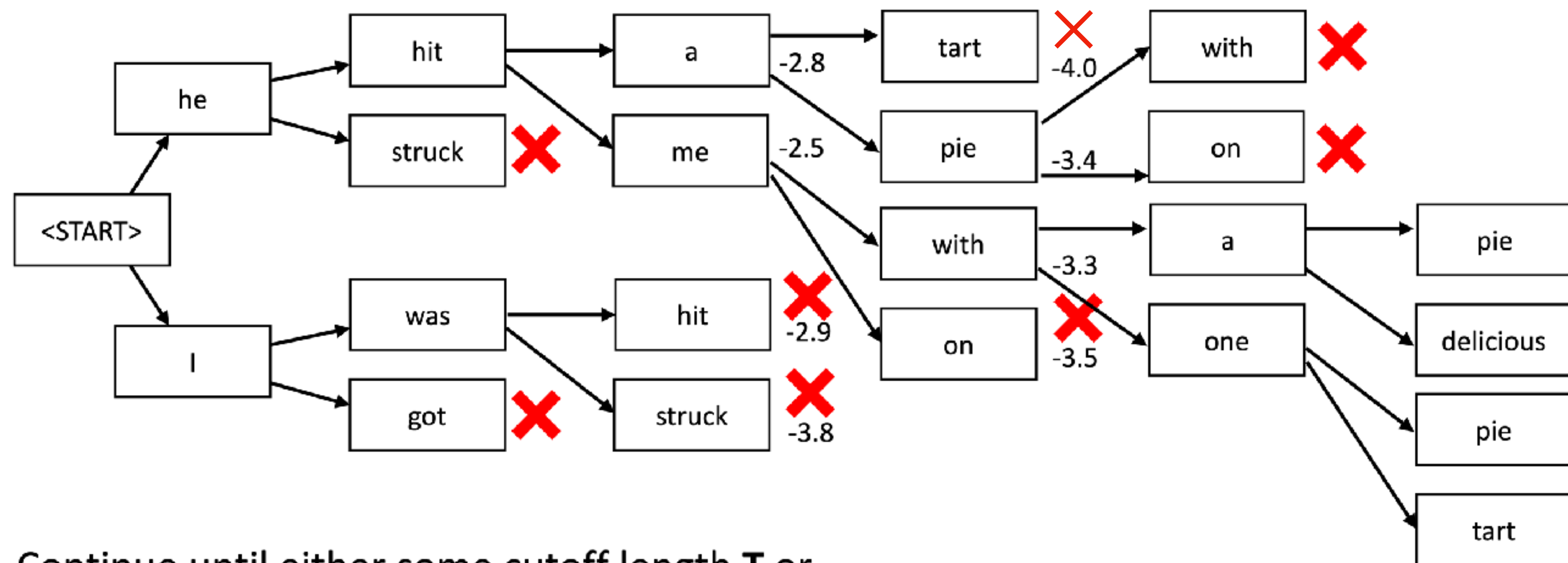
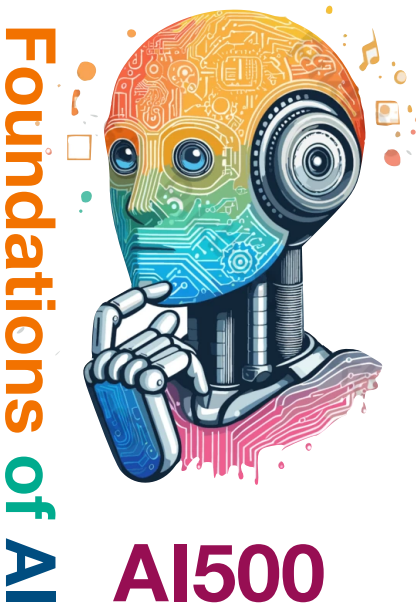
Beam Search Example

il a m`entarté - he hit me with a pie



Continue until either some cutoff length **T** or
until we have **N** hypotheses that end in <EOS>

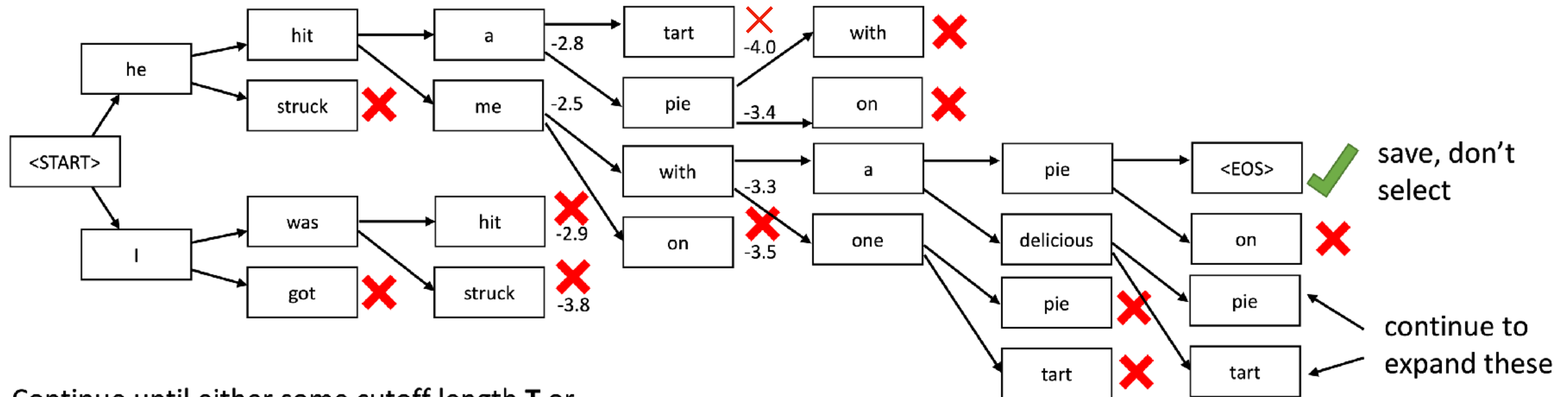
il a m`entarté - he hit me with a pie



Continue until either some cutoff length **T** or
until we have **N** hypotheses that end in <EOS>

Beam Search Example

il a m`entarté - he hit me with a pie



Continue until either some cutoff length **T** or until we have **N** hypotheses that end in <EOS>

Beam Search Example

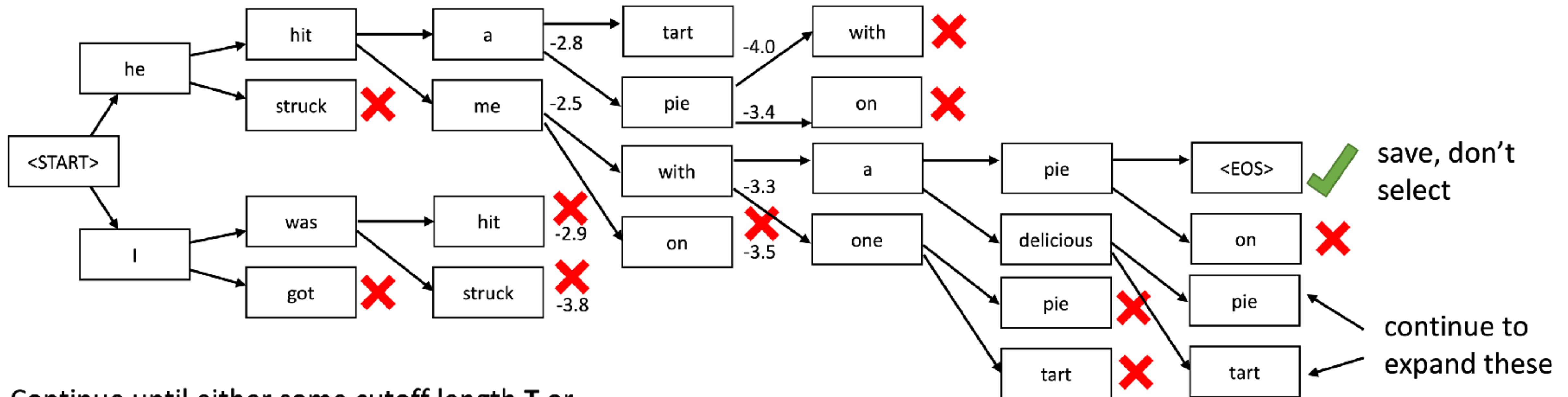
il a m`entarté - he hit me with a pie

When do we stop decoding?

Let's say one of the highest-scoring hypotheses ends in <END>

Save it, along with its score, but do **not** pick it to expand further (there is nothing to expand)

Keep expanding the **k** remaining best hypotheses



Continue until either some cutoff length **T** or until we have **N** hypotheses that end in <EOS>

Beam Search Summary

at each time step t :

there are k of these

very easy, we get this from the softmax log-probs

1. for each hypothesis $y_{1:t-1,i}$ that we are tracking:

find the top k tokens $y_{t,i,1}, \dots, y_{t,i,k}$

2. sort the resulting k^2 length t sequences by their *total* log-probability

3. keep the top k

4. advance each hypothesis to time $t + 1$

Which sequence do we pick?



At the end we might have something like this:

he hit me with a pie

$\log p = -4.5$

he threw a pie

$\log p = -3.2$

I was hit with a pie that he threw

$\log p = -7.2$

this is best, right?

Which sequence do we pick?

At the end we might have something like this:

he hit me with a pie

$\log p = -4.5$

he threw a pie

$\log p = -3.2$

I was hit with a pie that he threw

$\log p = -7.2$

this is best, right?

$$\log p(y_{i,1:T}|x_{i,1:T}) = \sum_{t=1}^T \log p(y_{i,t}|x_{i,1:T}, y_{i,0:t-1})$$

Problem: $p < 1$ **always**, hence $\log p < 0$ **always**

The **longer** the sequence the **lower** its total score (more negative numbers added together)

Simple “fix”:

$$\text{score}(y_{i,1:T}|x_{i,1:T}) = \frac{1}{T} \sum_{t=1}^T \log p(y_{i,t}|x_{i,1:T}, y_{i,0:t-1})$$

just divide by sequence length

Beam Search Summary



$$\text{score}(y_{i,1:T}|x_{i,1:T}) = \frac{1}{T} \sum_{t=1}^T \log p(y_{i,t}|x_{i,1:T}, y_{i,0:t-1})$$

at each time step t :

1. for each hypothesis $y_{1:t-1,i}$ that we are tracking:
find the top k tokens $y_{t,i,1}, \dots, y_{t,i,k}$
2. sort the resulting k^2 length t sequences by their *total* log-probability
3. save any sequences that end in EOS
4. keep the top k
5. advance each hypothesis to time $t + 1$ if $t < H$

return saved sequence with highest score

Readings

Optional



- The Curious Case of Neural Text Degeneration
<https://openreview.net/forum?id=rygGQyrFvH>
- Learning to Write with Cooperative Discriminators
<https://aclanthology.org/P18-1152/>
- Empirical Analysis of Beam Search Performance Degradation in Neural Sequence Models <http://proceedings.mlr.press/v97/cohen19a/cohen19a.pdf>

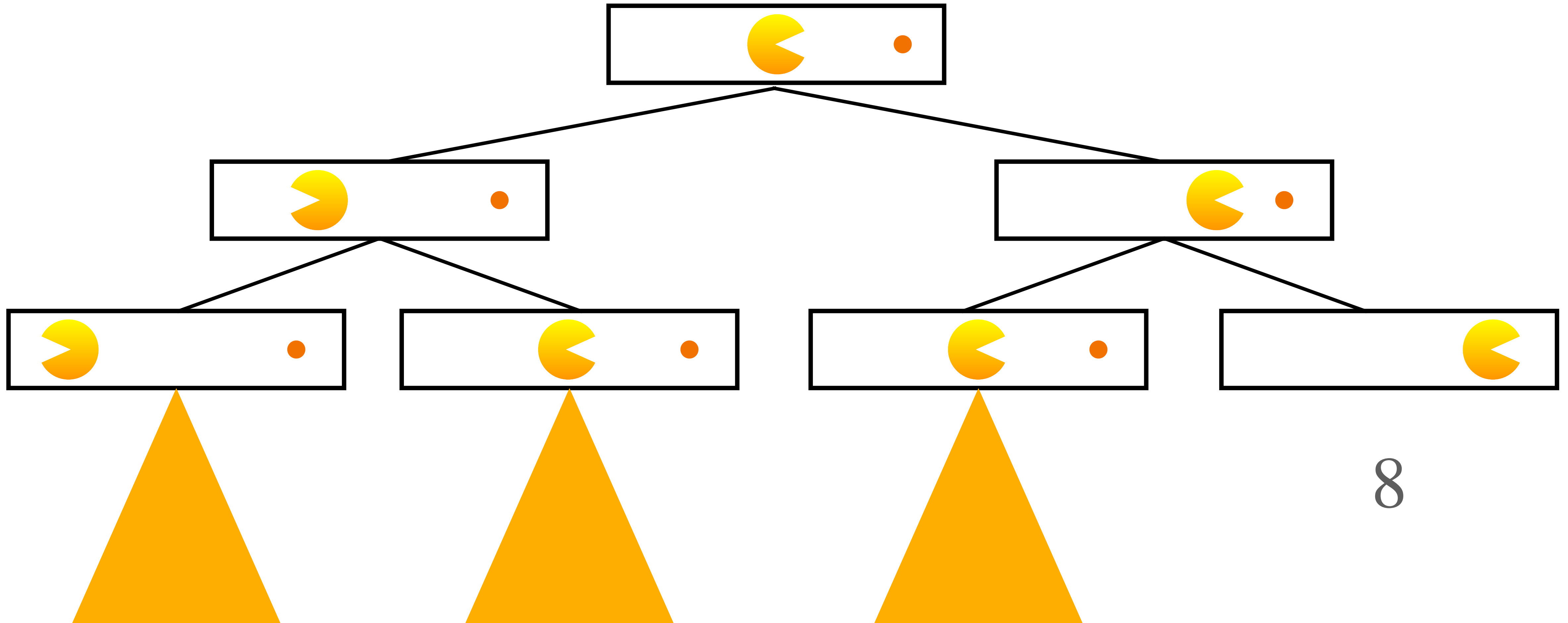
Adversarial Search & Games

Minimax: Adversarial Game-Tree Search
Monte Carlo Tree Search (MCTS)

(Russel & Norvig 5.2)

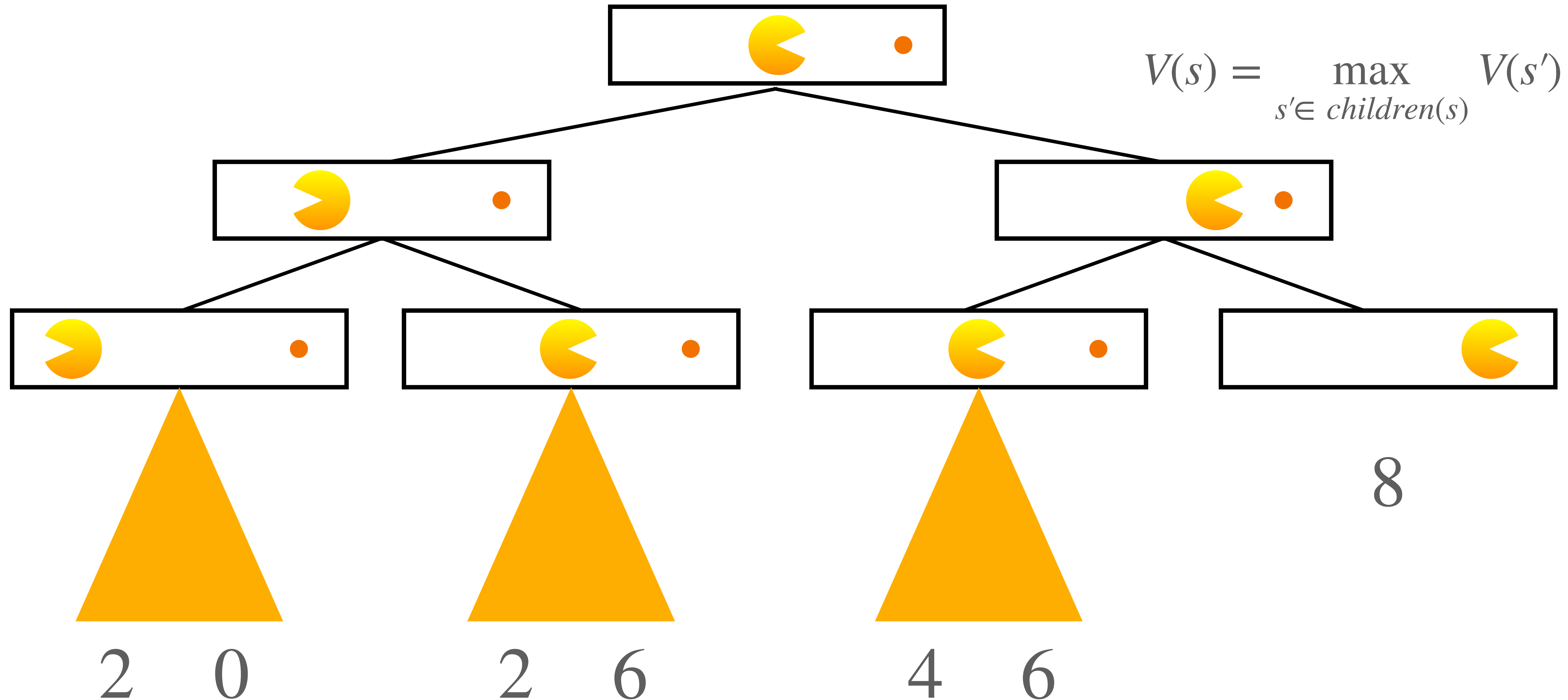
Single Agent Tree

Pacman



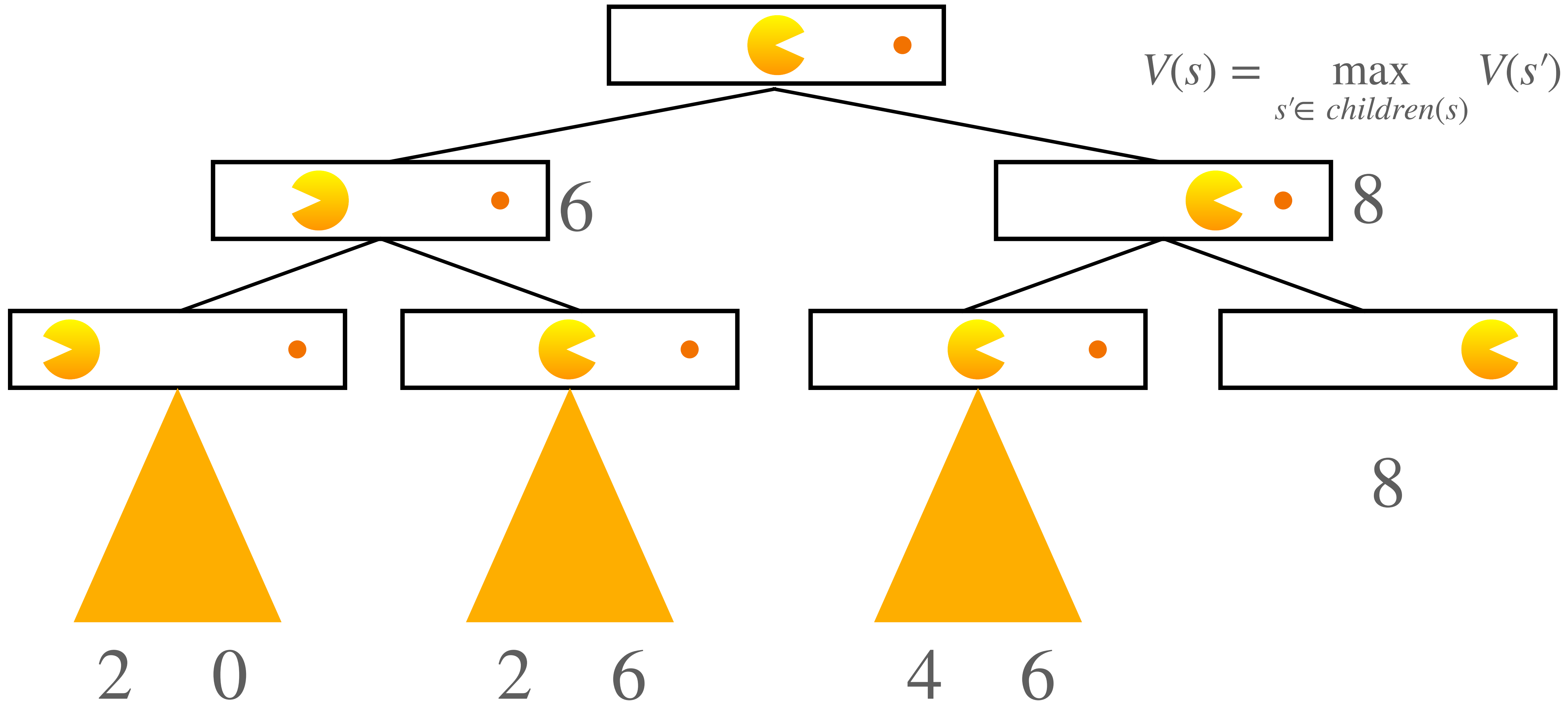
Value of State

The best achievable outcome (utility) from that state



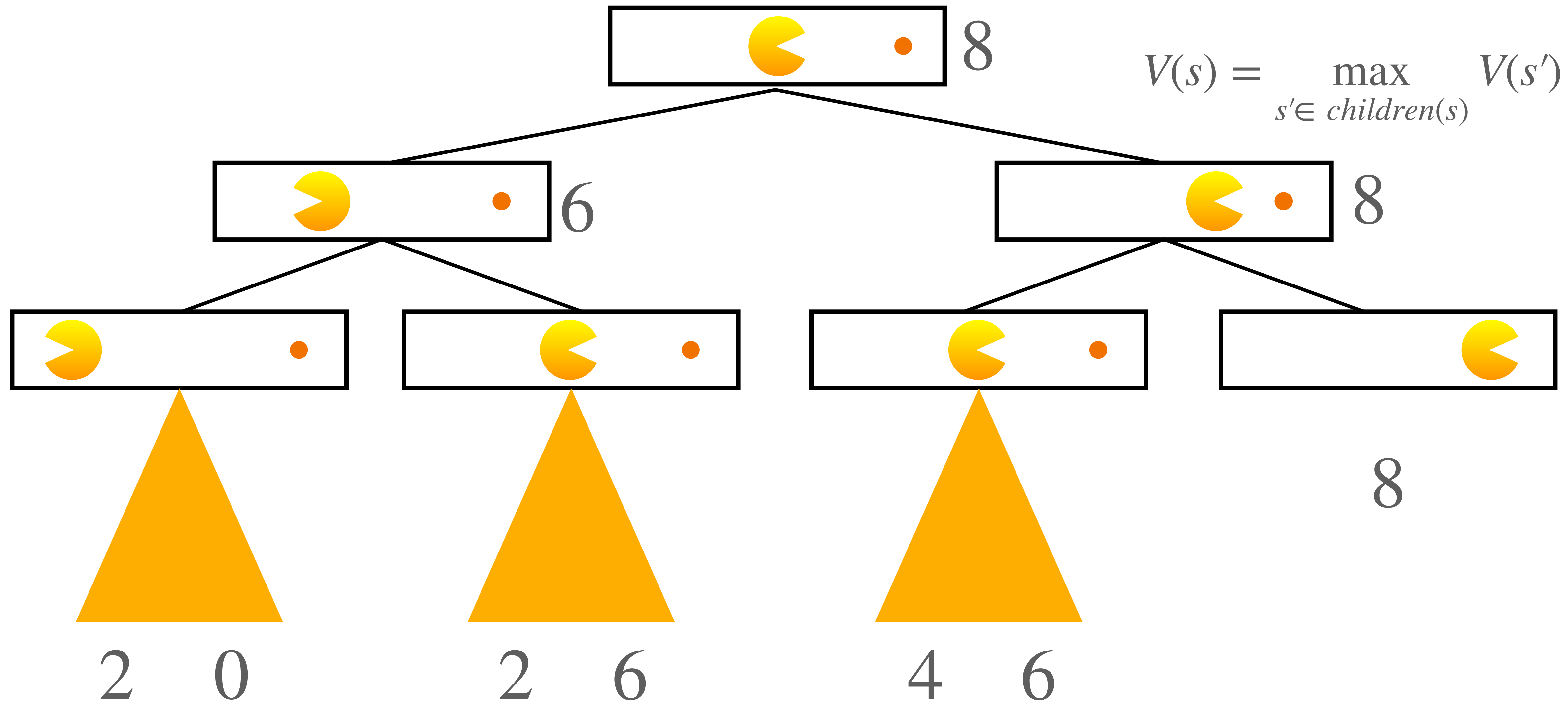
Value of State

The best achievable outcome (utility) from that state



Value of State

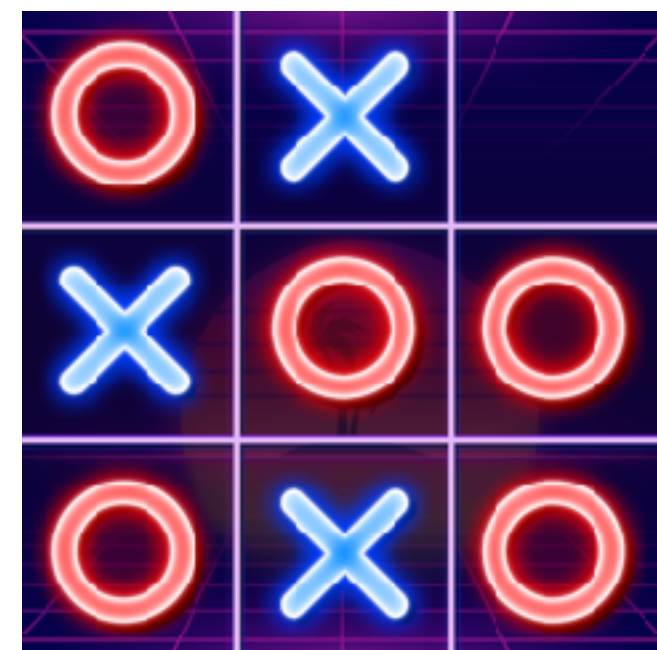
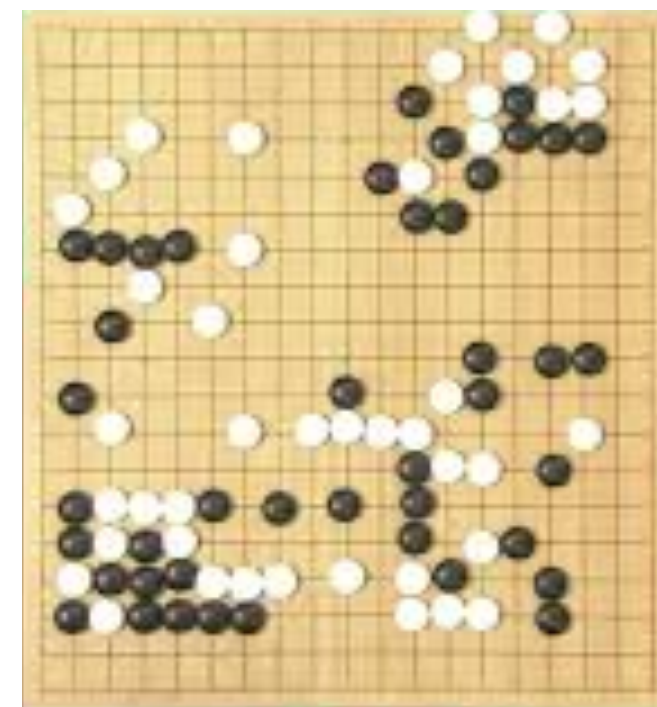
The best achievable outcome (utility) from that state



Two Player Zero-Sum Game

Win: +1, Lose: -1, Draw: 0

- Two-Player
- Zero-Sum
- Alternating Move
- Perfect Information
- Deterministic
- Discrete



- Monopoly is **stochastic** due to dice roll

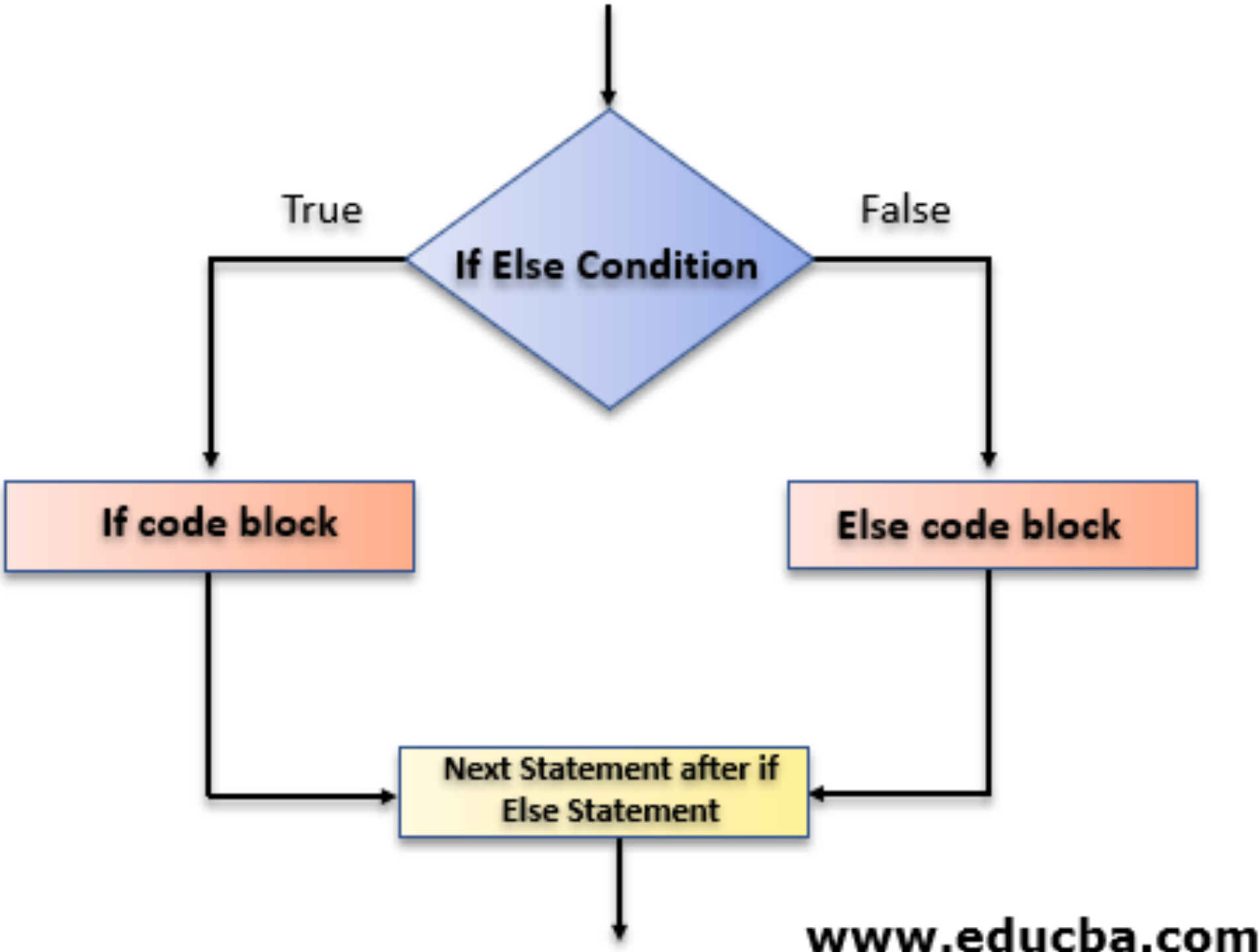


- In UNO you **don't have information** about card of others



Game Playing Computer

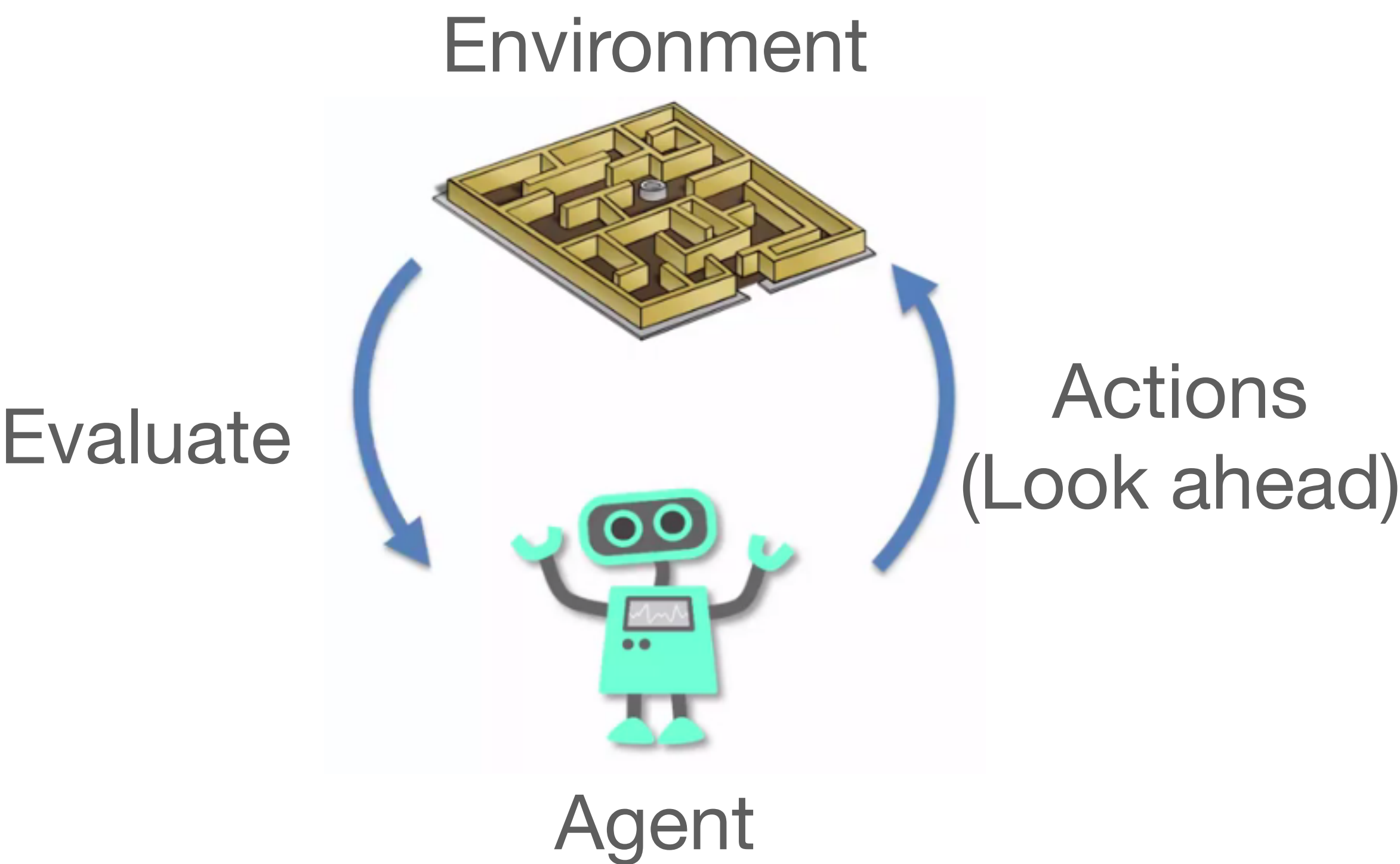
Strategy 1



www.educba.com

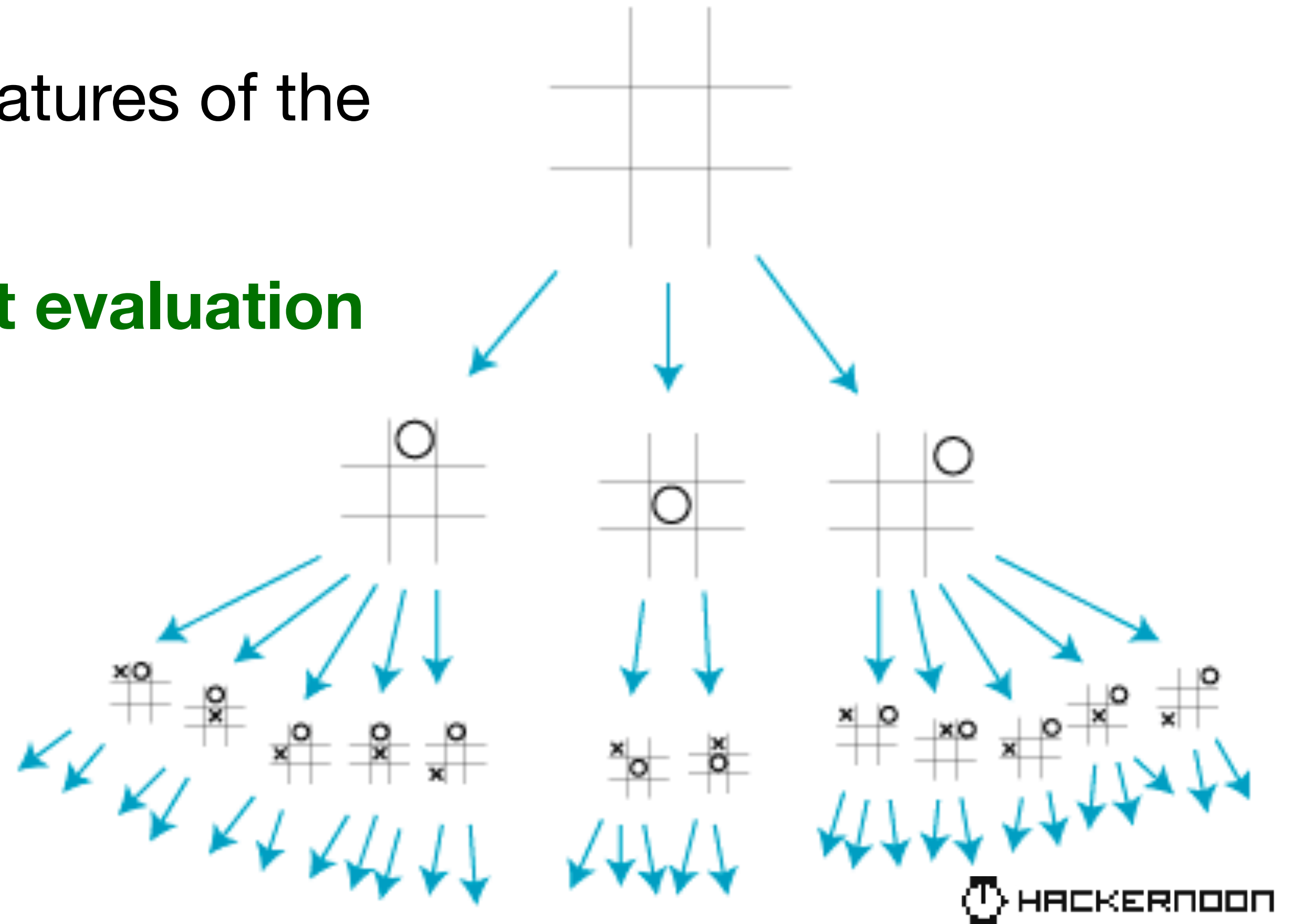
Thousands of if-else

Strategy 2



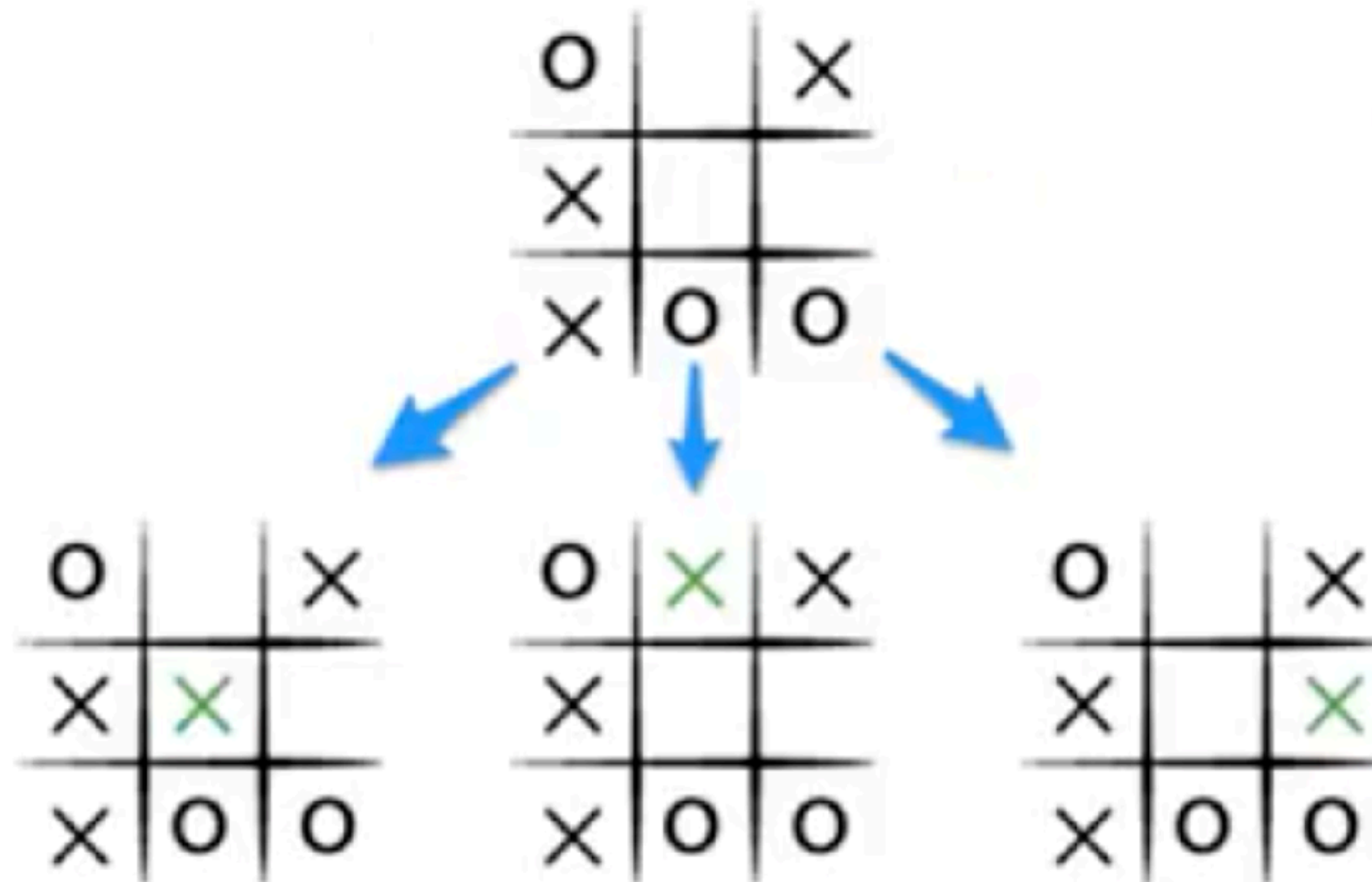
Look Ahead and Evaluate

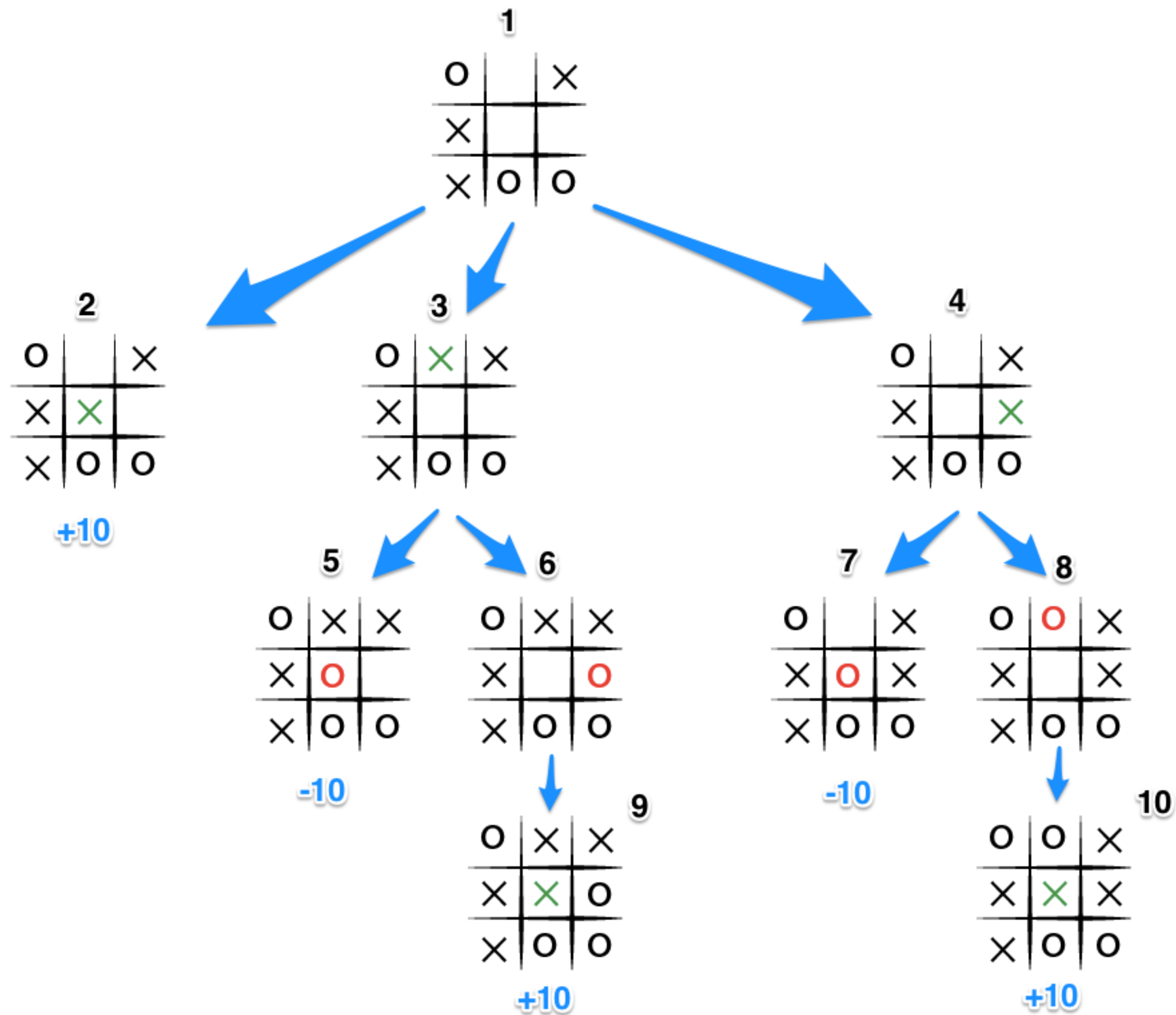
- Generate a **list of actions** from a current state
- **Evaluate** those actions based on features of the resulting states
- Do the action which has the **highest evaluation**



Tic-Tac-Toe Example

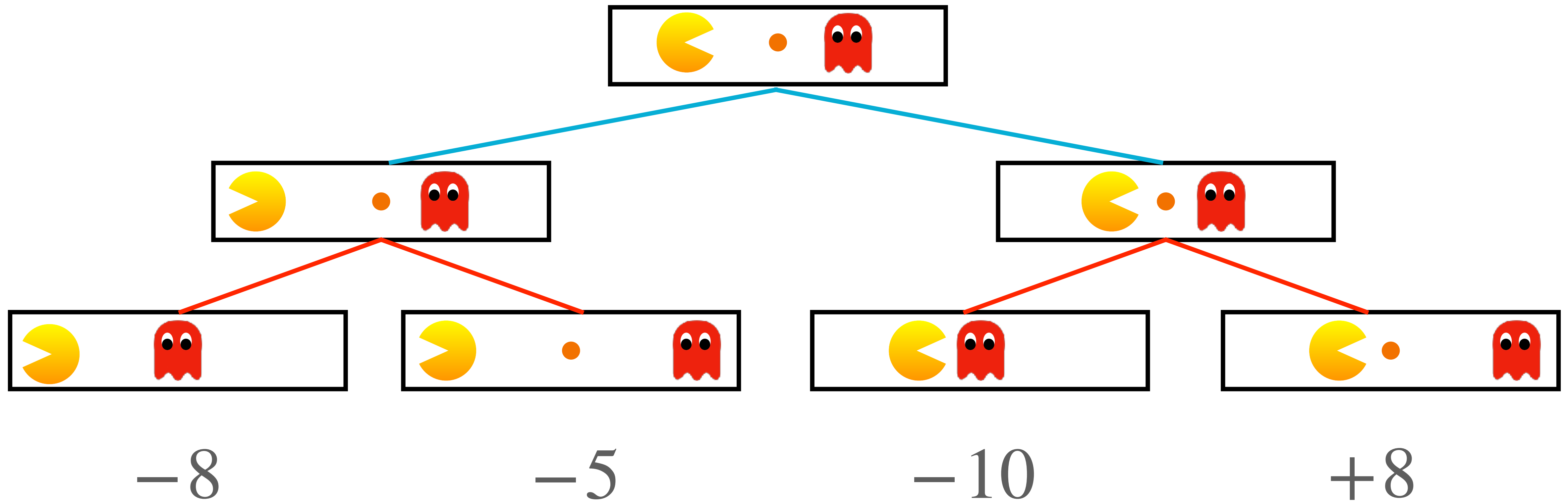
How far can we look?





Minimax Values

Pacman



Minimax Values

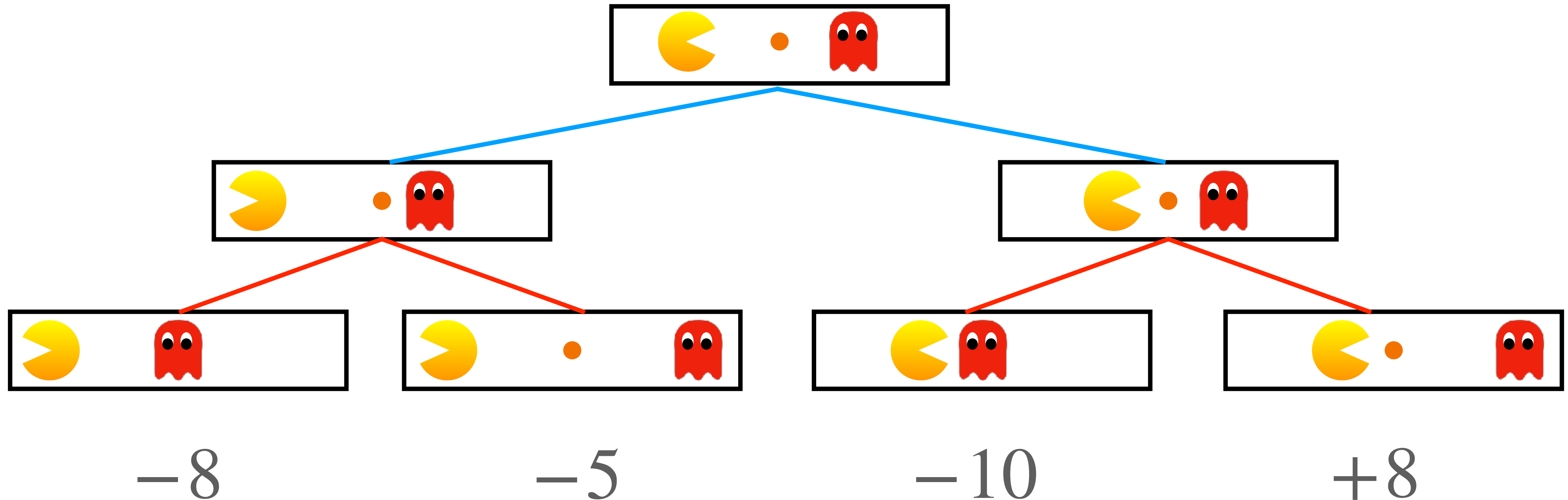
Pacman

State under agent's control

$$V(s) = \max_{s' \in \text{successor}(s)} V(s')$$

State under opponent's control

$$V(s) = \min_{s' \in \text{successor}(s)} V(s')$$



Minimax Values

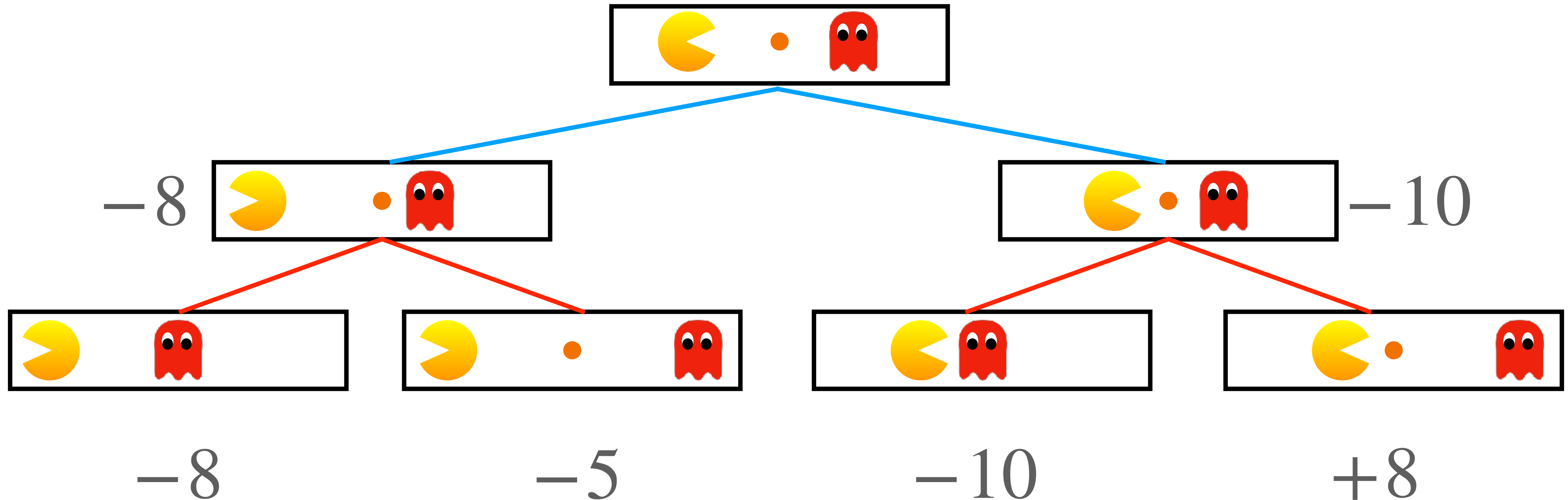
Pacman

State under agent's control

$$V(s) = \max_{s' \in \text{successor}(s)} V(s')$$

State under opponent's control

$$V(s) = \min_{s' \in \text{successor}(s)} V(s')$$



Minimax Values

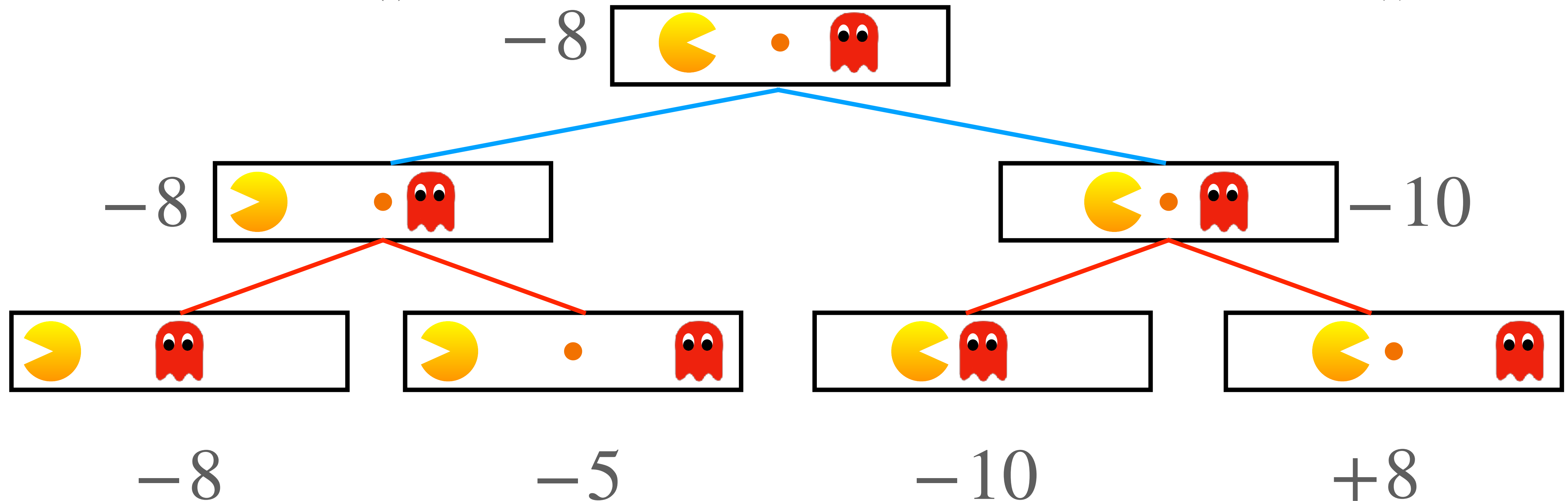
Pacman

State under agent's control

$$V(s) = \max_{s' \in \text{successor}(s)} V(s')$$

State under opponent's control

$$V(s) = \min_{s' \in \text{successor}(s)} V(s')$$



Minimax

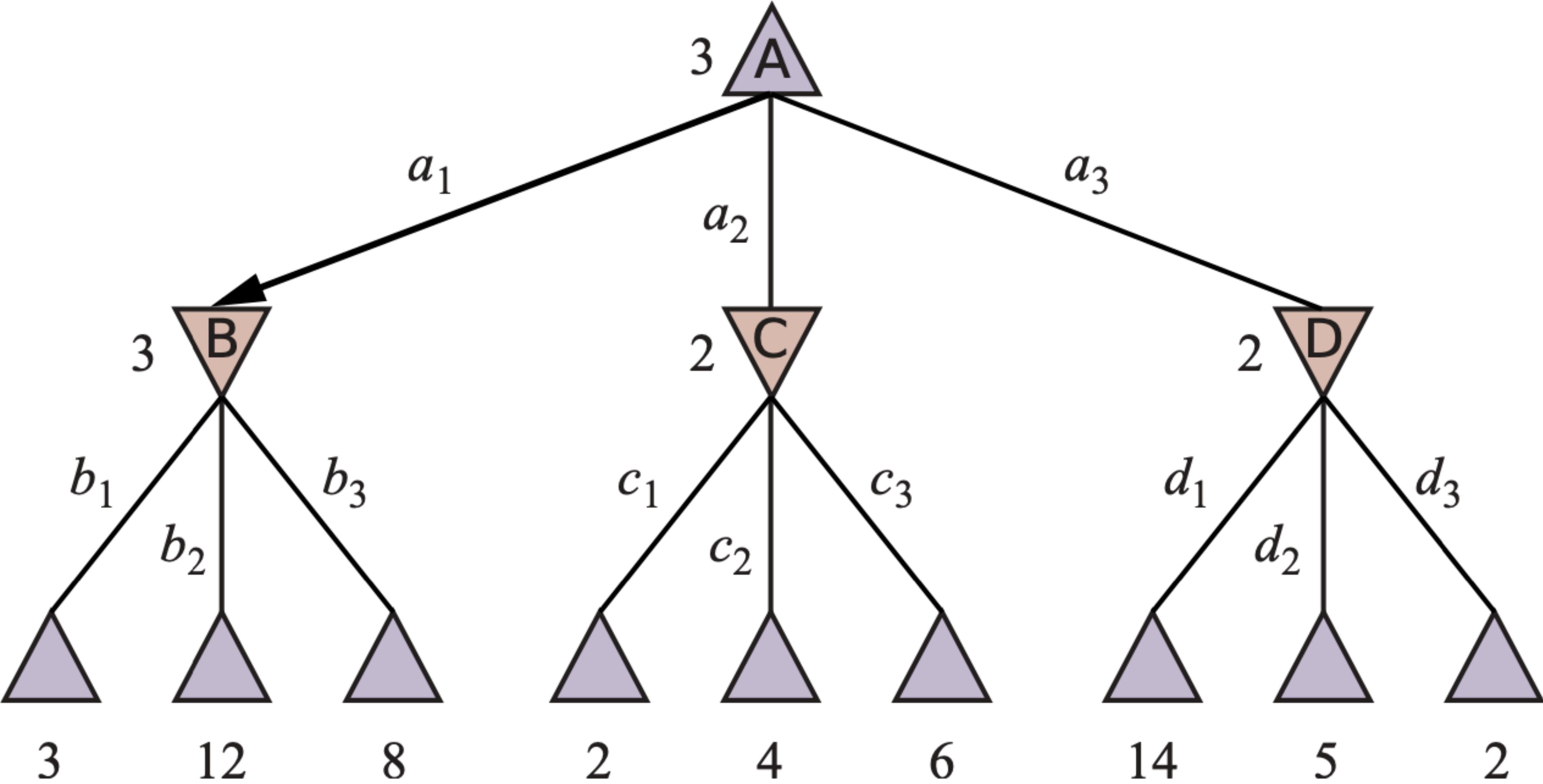
Example Figure 5.2



MAX



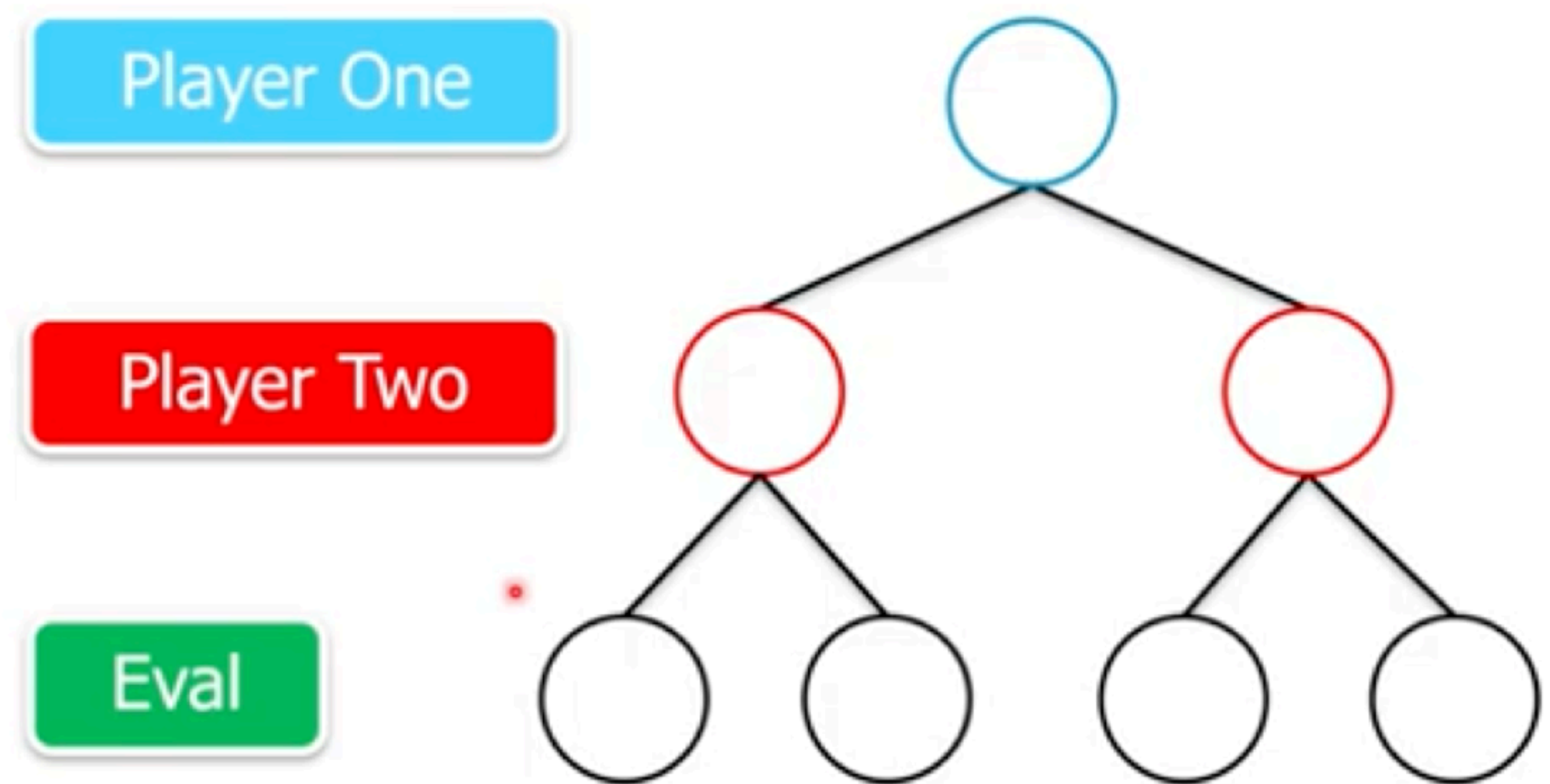
MIN



Competing Players

Adversarial Search

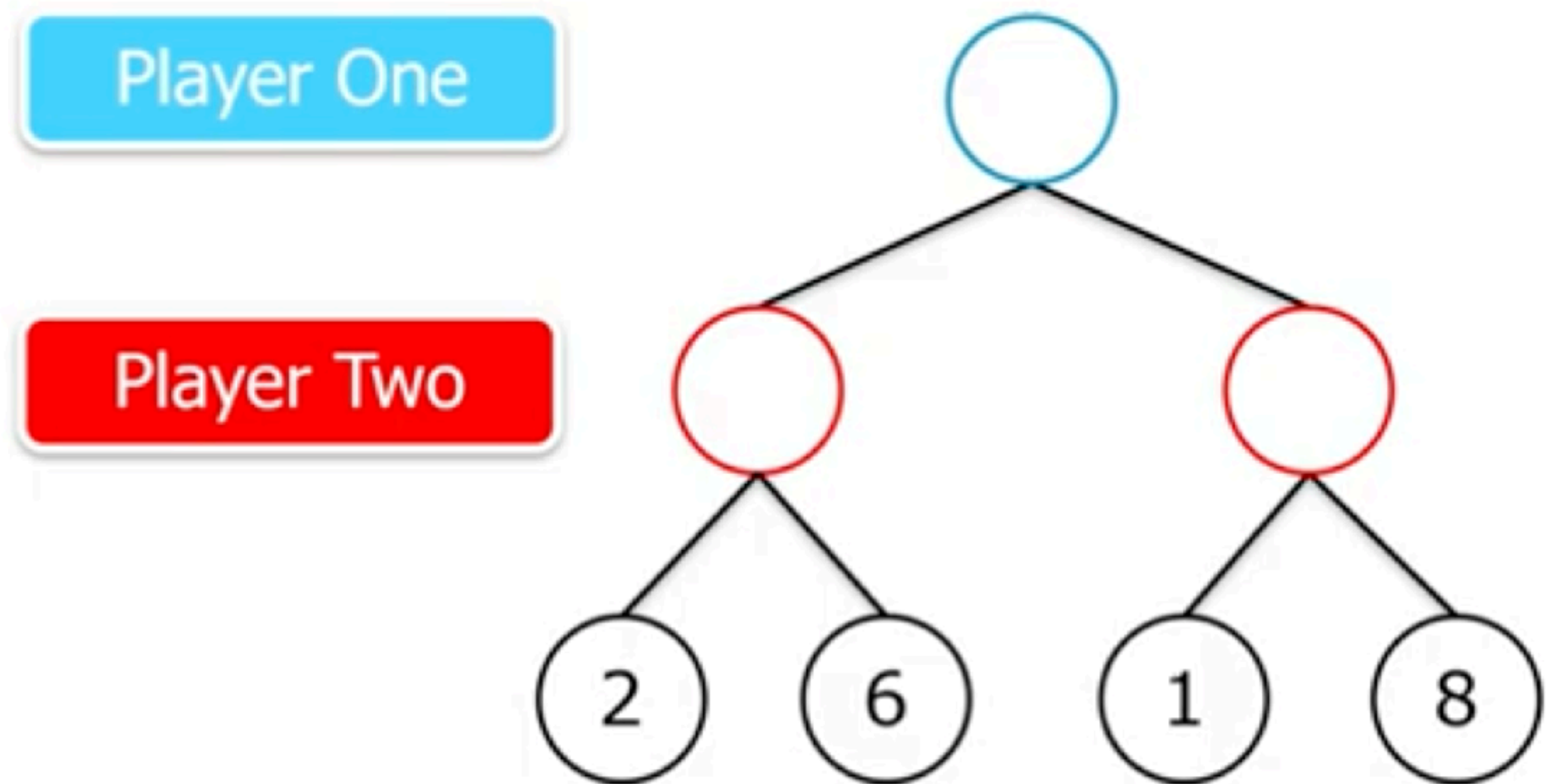
- I am trying to **maximize** my output
- Competing player is trying to **minimize** my outcome



Competing Players

Adversarial Search

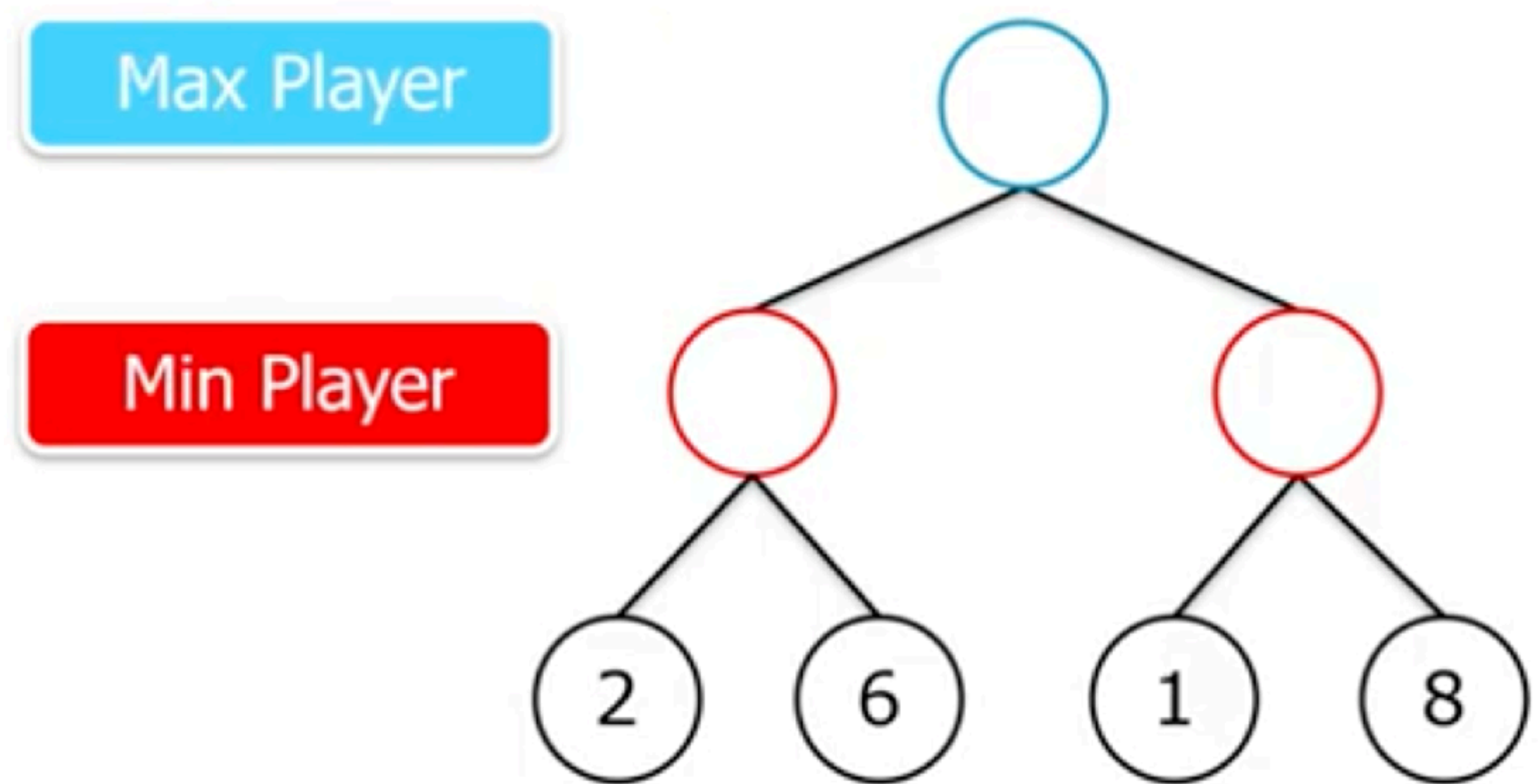
- I am trying to **maximize** my output
- Competing player is trying to **minimize** my outcome



Competing Players

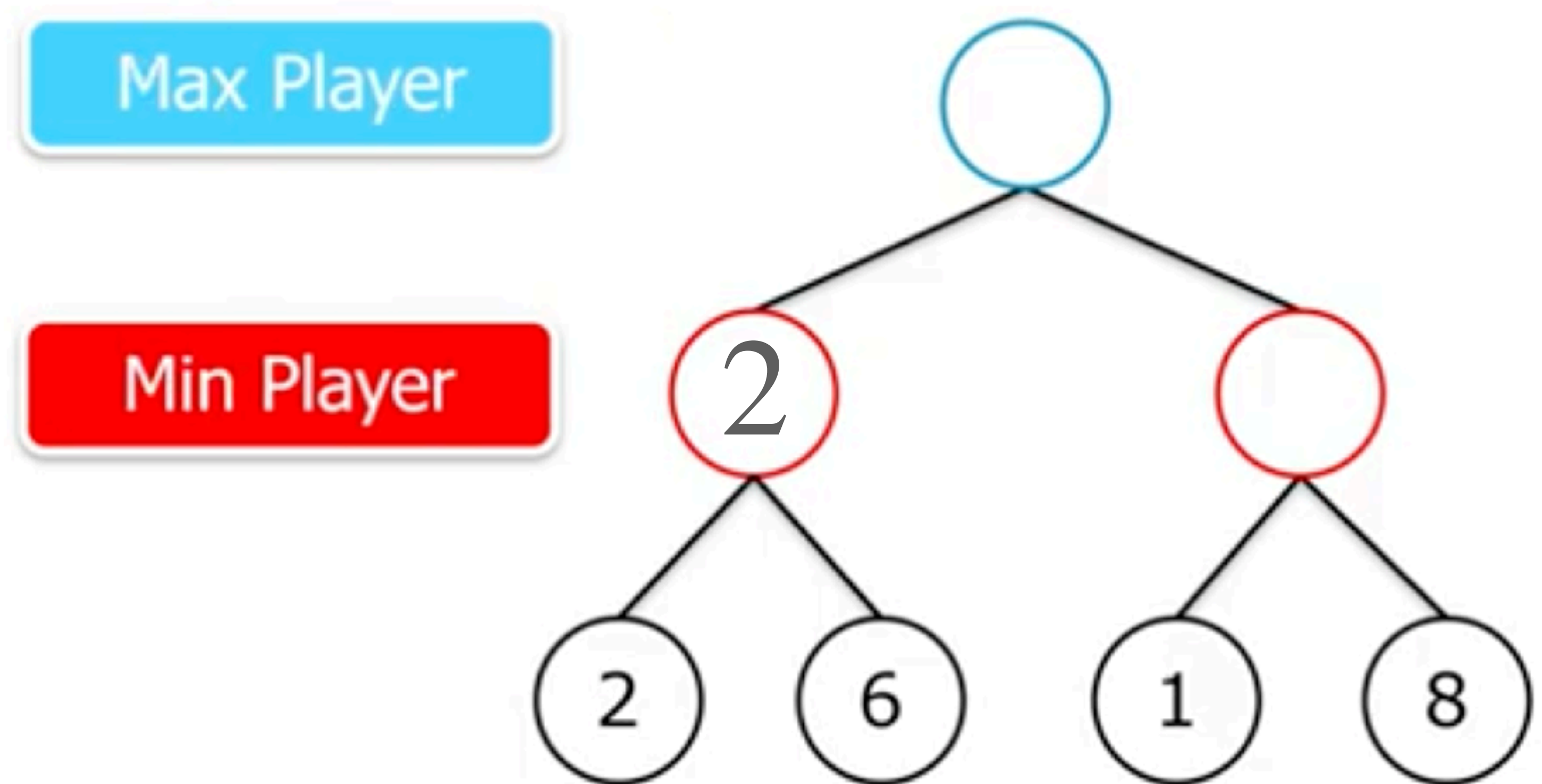
Adversarial Search

- **Max Player:** Player trying to **maximize** its output
- **Min Player:** Competing player trying to **minimize** the result of Max Player



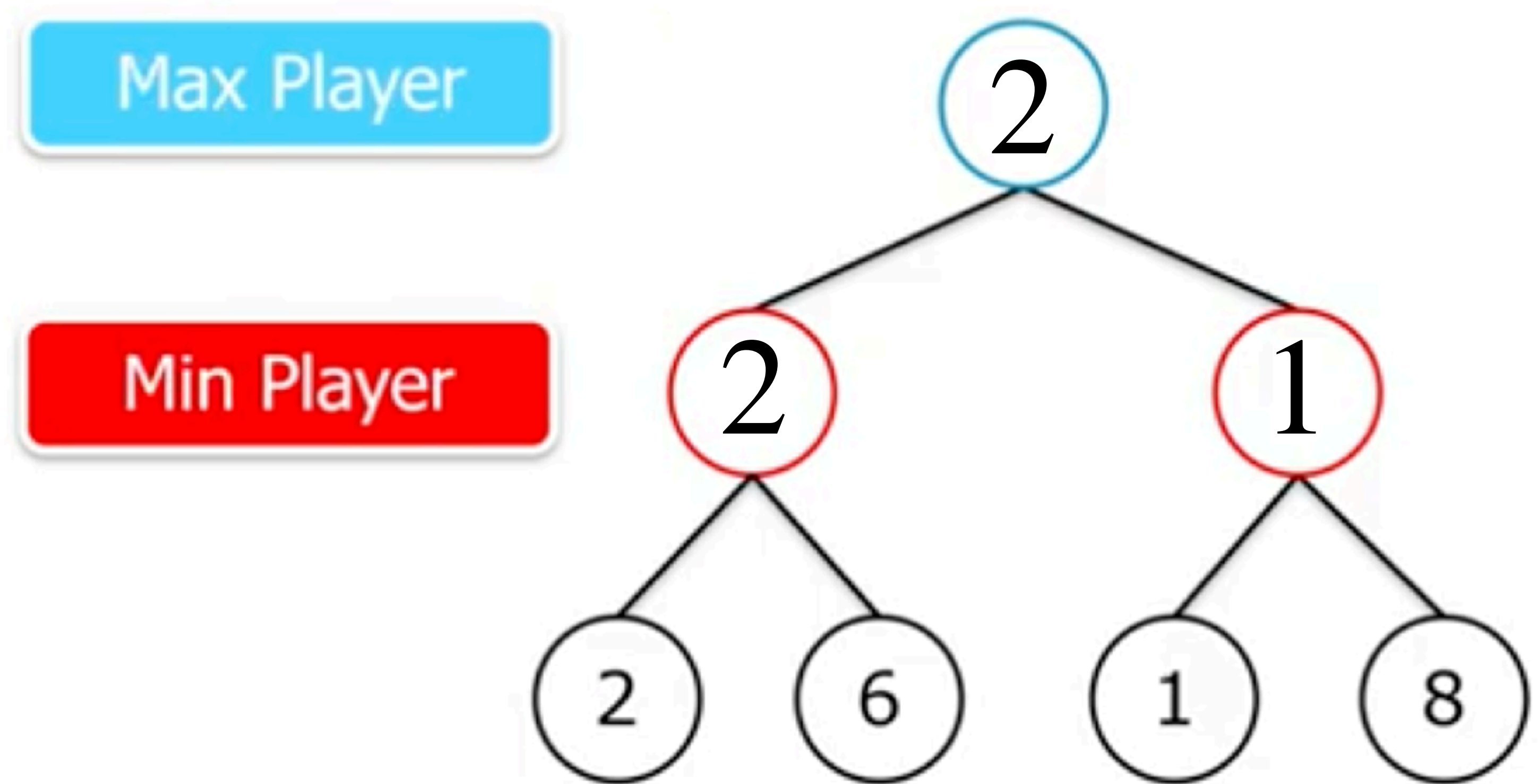
Competing Players

- **Max Player:** Player trying to **maximize** its output
- **Min Player:** Competing player trying to **minimize** the result of Max Player



Competing Players

- **Max Player:** Player trying to **maximize** its output
- **Min Player:** Competing player trying to **minimize** the result of Max Player



MaxValue Function

Pseudo Code - Full tree

```
function MaxValue(node):  
    if terminal(node)  
        return eval(node)  
  
     $v = -\infty$   
    for each child of node:  
         $v' = \text{MinValue}(\text{child})$   
        if  $v' > v$   
             $v = v'$   
    return  $v$ 
```

terminal: allowed depth, allowed time, game over
eval() returns score w.r.t maximixing player

MaxValue and MinValue Function

Pseudo Code - Full tree

```
function MaxValue(node):  
    if terminal(node)  
        return eval(node)
```

$v = -\infty$

```
for each child of node:  
     $v' = \text{MinValue}(\text{child})$   
    if  $v' > v$   
         $v = v'$ 
```

```
return v
```

```
function MinValue(node):  
    if terminal(node)  
        return eval(node)
```

$v = +\infty$

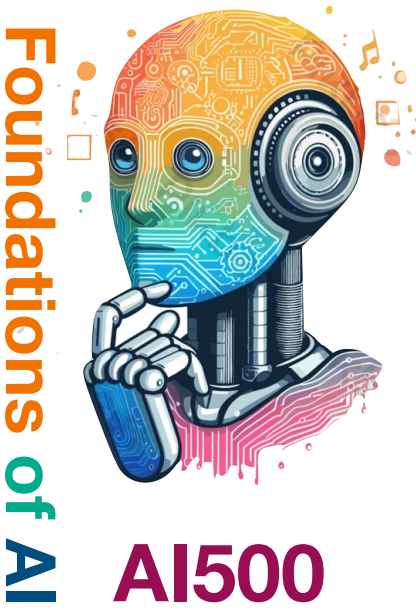
```
for each child of node:  
     $v' = \text{MaxValue}(\text{child})$   
    if  $v' < v$   
         $v = v'$ 
```

```
return v
```

terminal: allowed depth, allowed time, game over
eval() returns score w.r.t maximixing player

MaxValue and MinValue Function

Pseudo Code - Depth Limited



```
function MaxValue(node, d):  
    if terminal(node) or d > maxD  
        return eval(node)
```

```
    v = -∞  
    for each child of node:  
        v' = MinValue(child, d+1)  
        if v' > v  
            v = v'  
    return v
```

```
function MinValue(node, d):  
    if terminal(node) or d > maxD  
        return eval(node)
```

```
    v = +∞  
    for each child of node:  
        v' = MaxValue(child, d+1)  
        if v' < v  
            v = v'  
    return v
```

terminal: allowed depth, allowed time, game over
eval() returns score w.r.t maximixing player

MiniMax Algorithm

```
function MiniMax(node, d, max):  
    if terminal(node) or d > maxD  
        return eval(node)  
    if max  
        v = -∞  
        for each child of node  
            v = max(v, MiniMax(child, d+1, false))  
        return v  
    else  
        v = +∞  
        for each child of node  
            v = min(v, MiniMax(child, d+1, true))  
        return v
```

Call MiniMax(initialState, 0, true)

Lets consider this board
Multiple plays possible,
but which one is the best

<i>O</i>		<i>X</i>
<i>X</i>		
<i>X</i>	<i>O</i>	<i>O</i>

X's turn, which will try
to maximise score

1

<i>O</i>	<i>X</i>	<i>X</i>
<i>X</i>		
<i>X</i>	<i>O</i>	<i>O</i>

2

<i>O</i>		<i>X</i>
<i>X</i>	<i>X</i>	
<i>X</i>	<i>O</i>	<i>O</i>

3

<i>O</i>		<i>X</i>
<i>X</i>		<i>X</i>
<i>X</i>	<i>O</i>	<i>O</i>

Let's focus here

Python Demo TicTacToe

O		X
X	X	
X	O	O



MiniMax

Properties (see also pg. 150)

- **Complete and Optimal**: will find the optimal solution to a given max depth
- Each player play the **best response** to the possible actions of the other player
- The minimax algorithm performs a **complete depth-first** exploration of the game tree. If the maximum depth of the tree is m and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^d)$. The **exponential complexity** makes minimax impractical for complex games; for example, chess has a branching factor of about 35 and the average game has depth of about 80 ply, and it is not feasible to search $35^{80} \approx 10^{123}$ states.
- Minimax does, however, serve as a **basis for the mathematical analysis of games**.
- By **approximating the minimax** analysis in various ways, we can derive more practical algorithms.

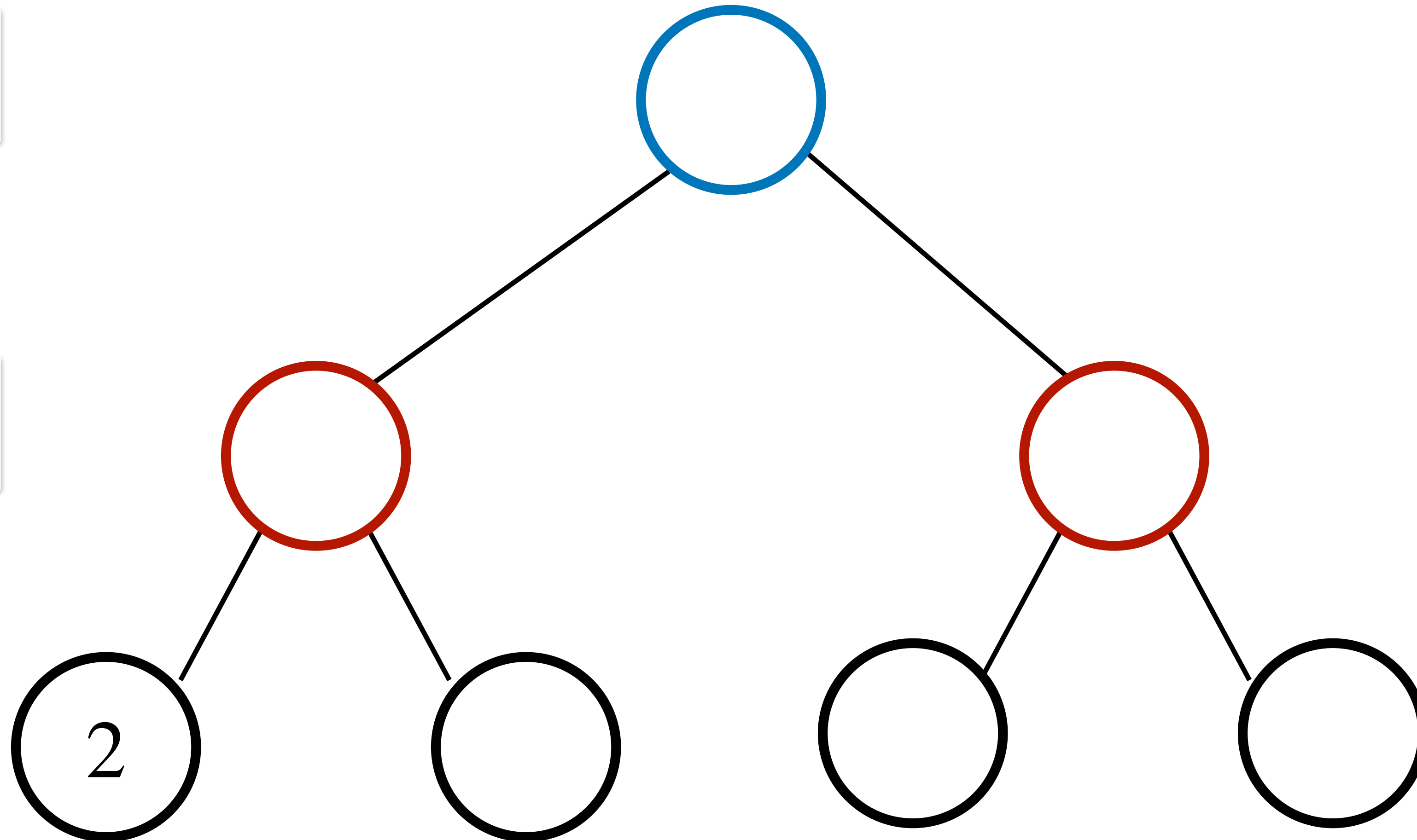
Alpha-Beta Pruning

Improving MiniMax

Example 1

Max

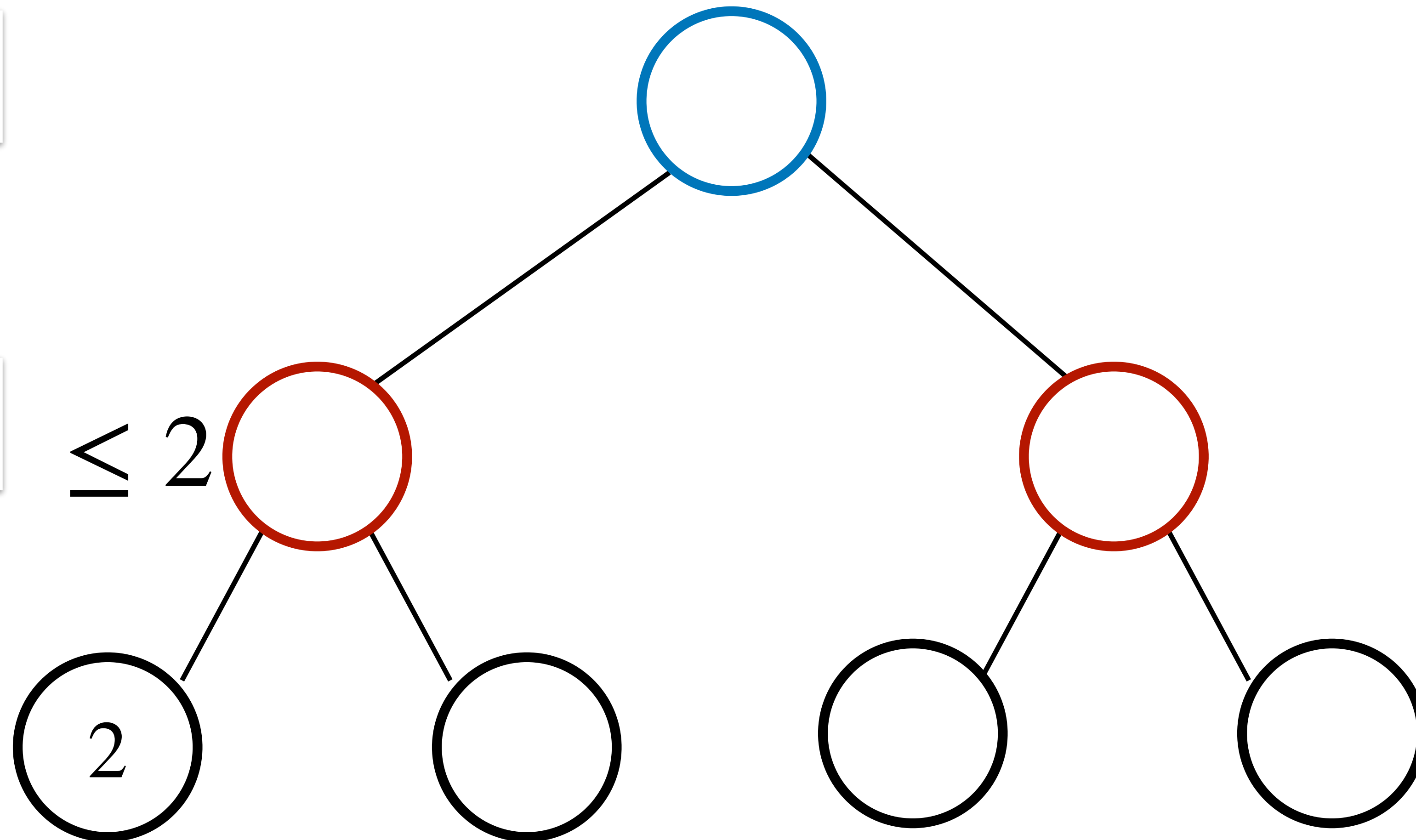
Min



Improving MiniMax

Max

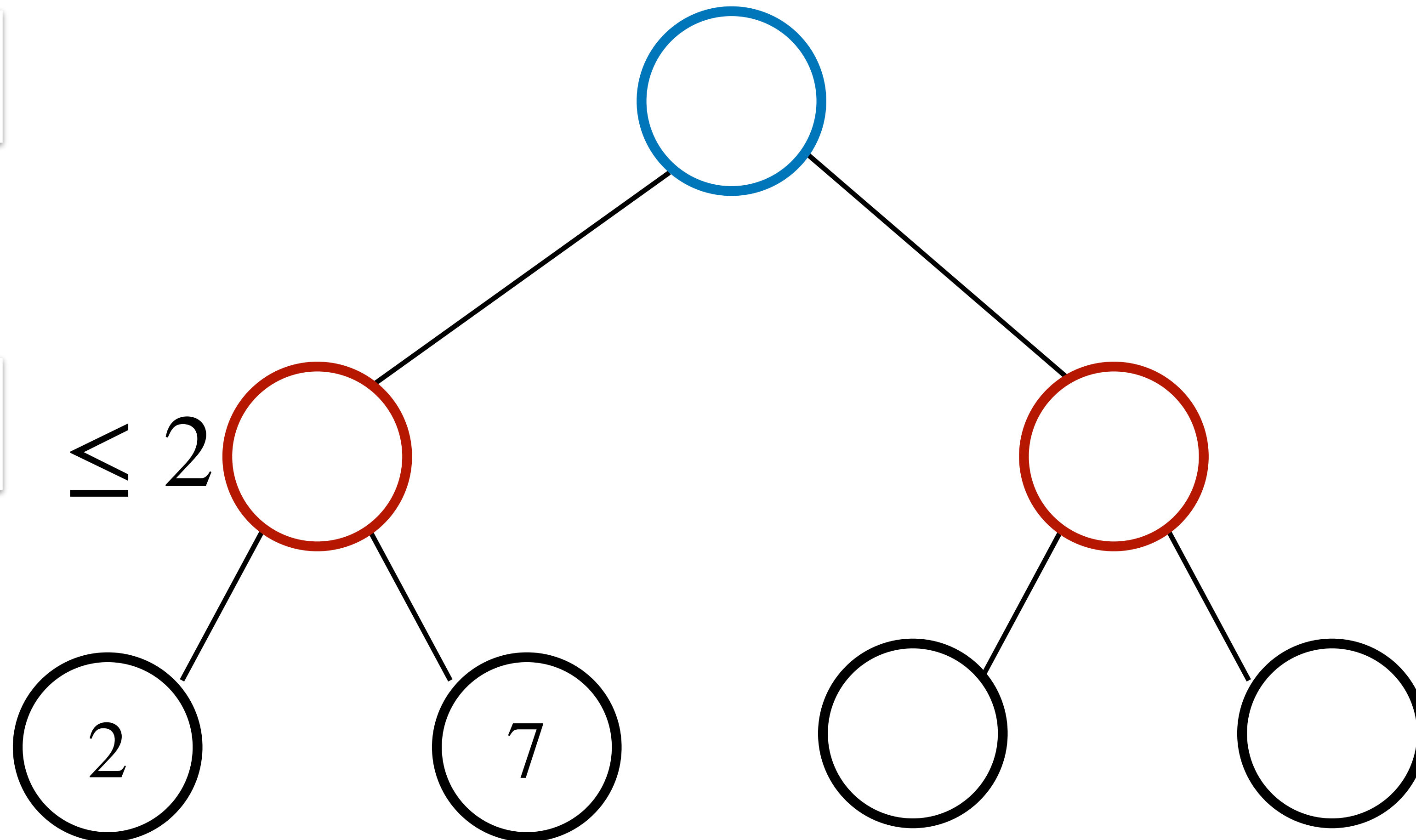
Min



Improving MiniMax

Max

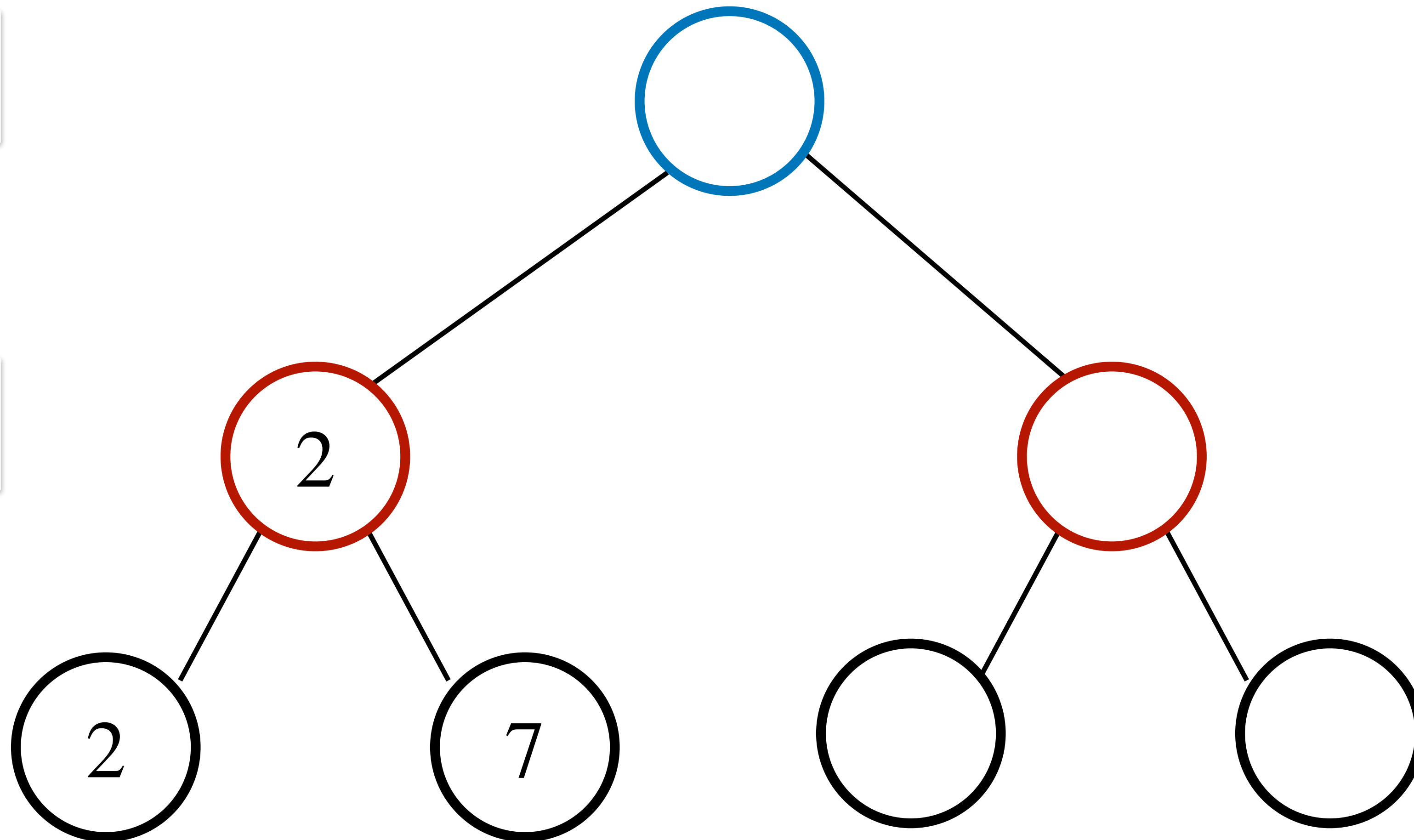
Min



Improving MiniMax

Max

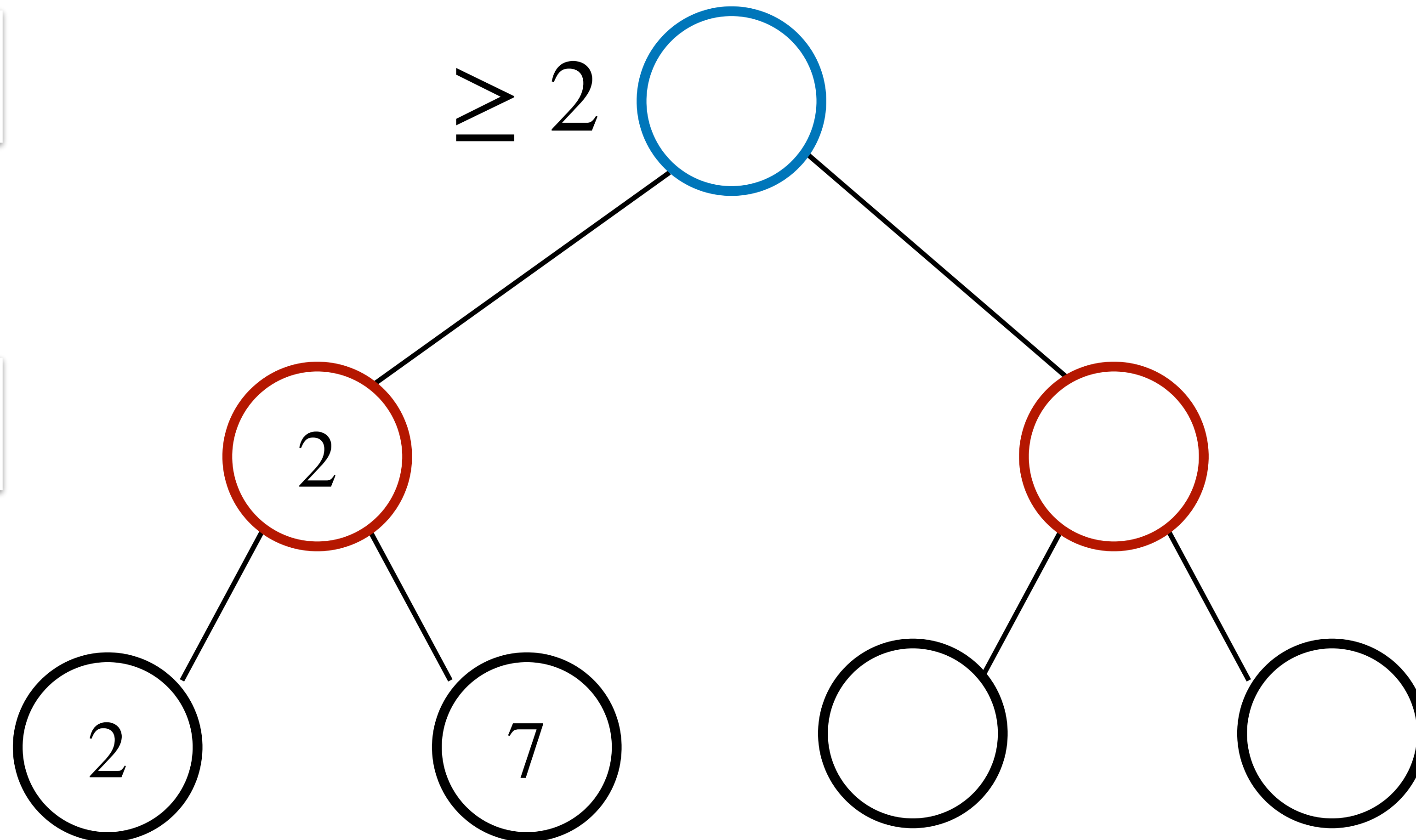
Min



Improving MiniMax

Max

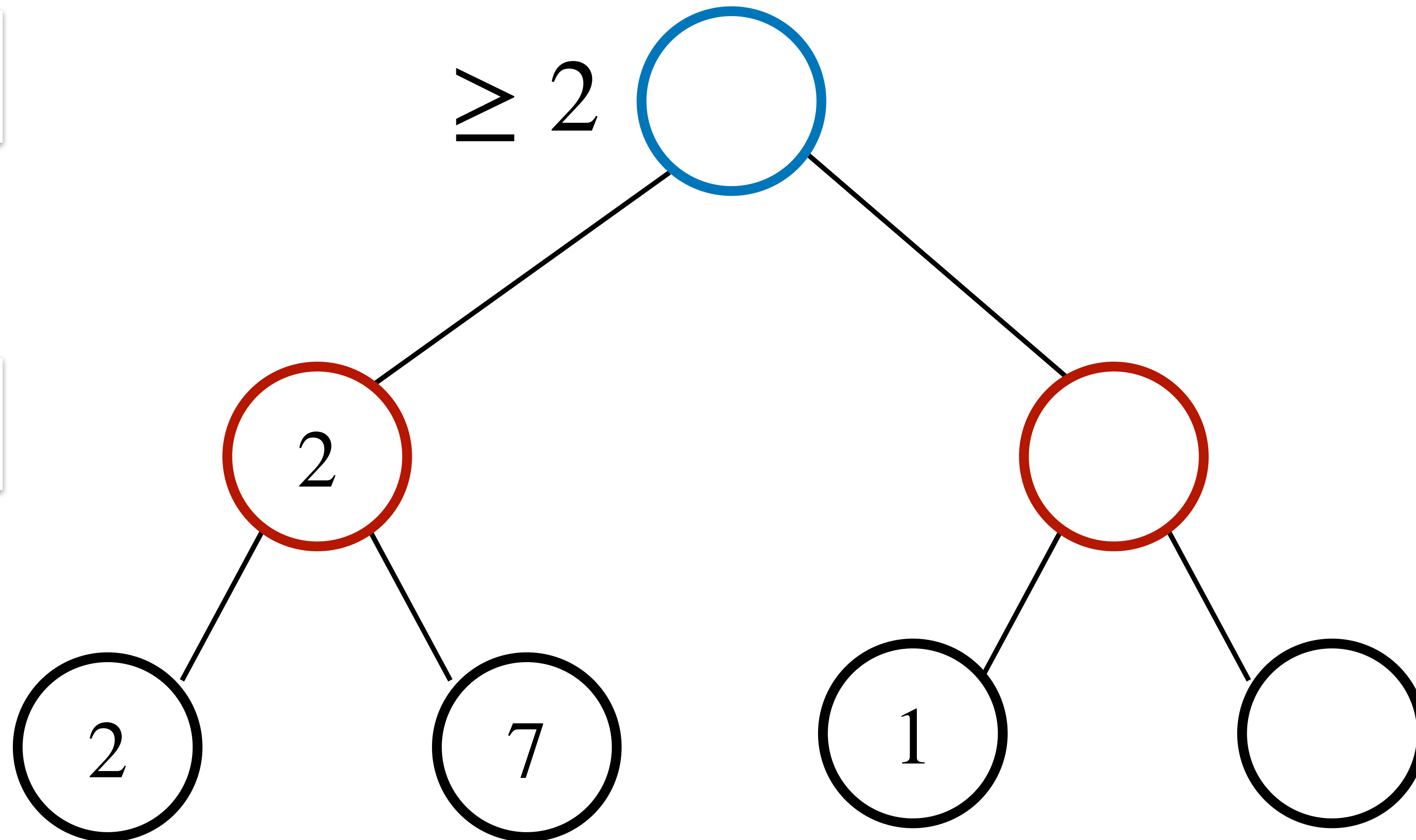
Min



Improving MiniMax

Max

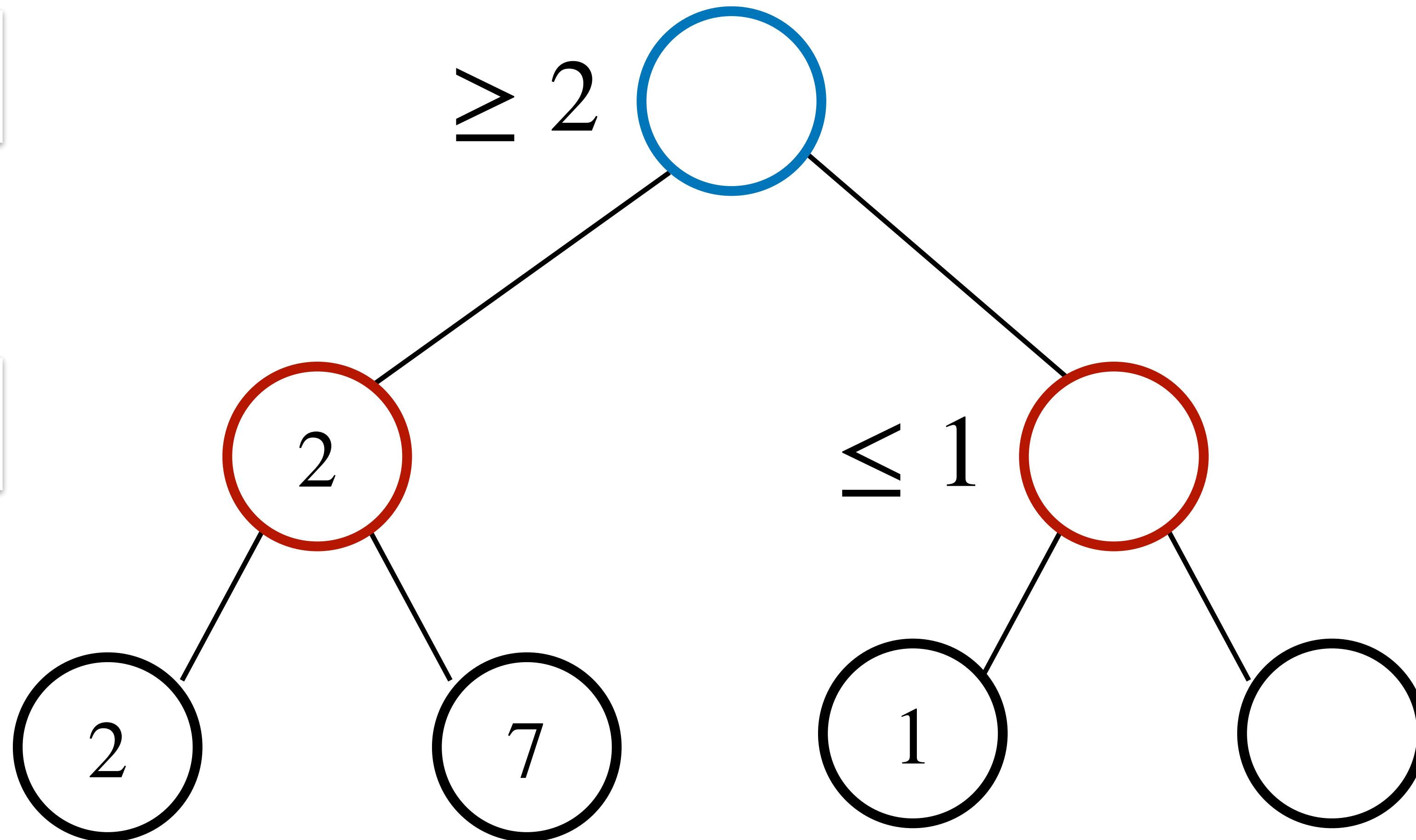
Min



Improving MiniMax

Max

Min

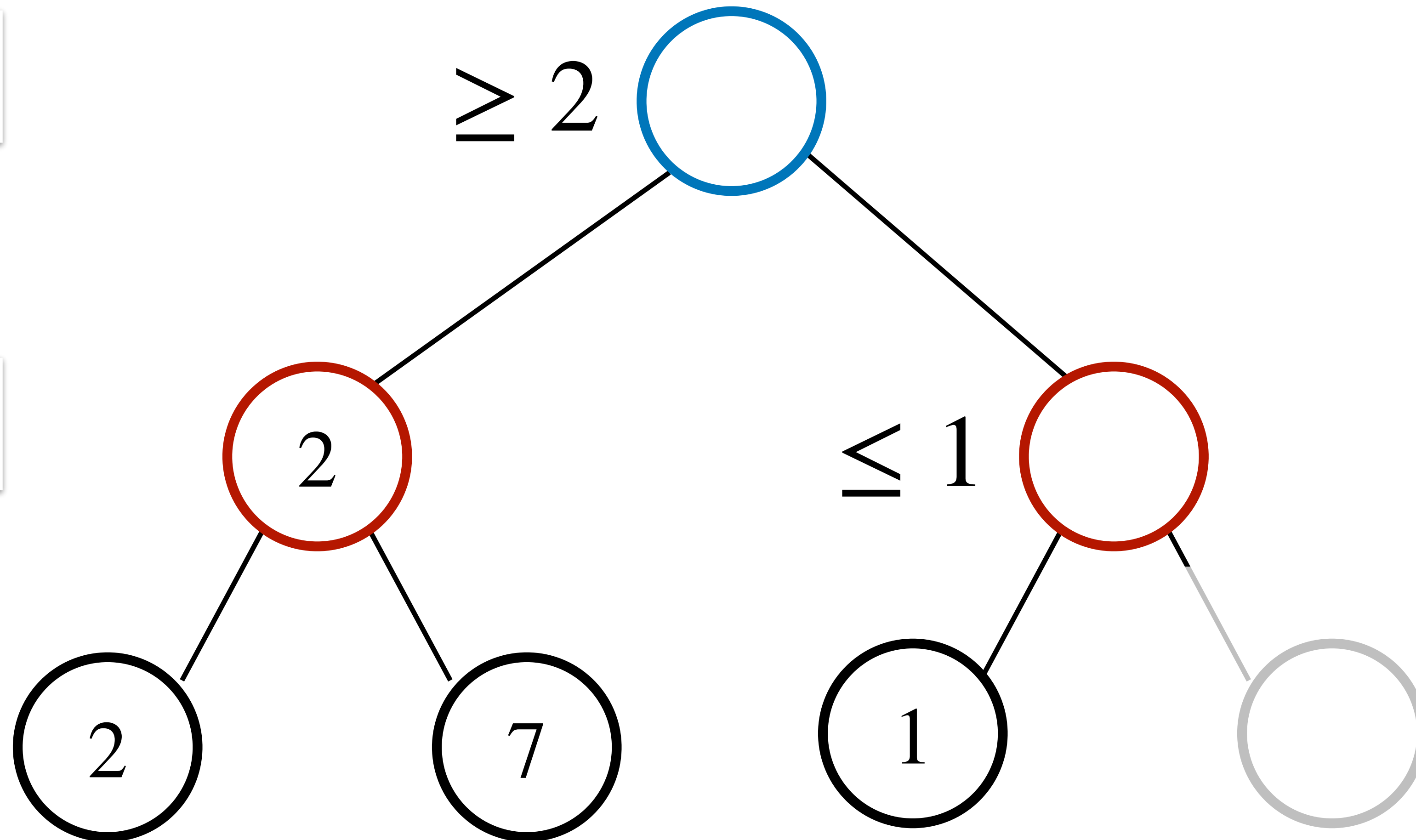


Improving MiniMax

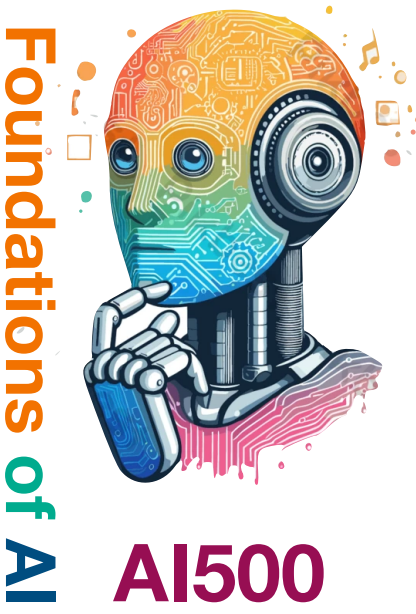
Pruning part of the tree - Alpha-Beta Pruning

Max

Min



MiniMax Algorithm + Alpha/Beta Pruning



- Keep two bounds along the path
 - the best Max can do on the path
 - the best (smallest) Min can do on the path
- If a max node exceeds β , it is pruned.
- If a min node goes below α , it is pruned.

MiniMax Algorithm + Alpha/Beta Pruning

$[-\infty, +\infty]$

```
function MiniMax(node, d, max,  $\alpha$ ,  $\beta$ ):  
    if terminal(node) or d > maxD  
        return eval(node)
```

```
    if max  
        v =  $-\infty$   
        for each child of node  
            v = max(v, MiniMax(child, d+1, false,  $\alpha$ ,  $\beta$ )  
            if v >  $\beta$  return v  
             $\alpha$  = max( $\alpha$ , v)  
        return v
```

```
    else  
        v =  $+\infty$   
        for each child of node  
            v = min(v, MiniMax(child, d+1, true,  $\alpha$ ,  $\beta$ )  
            if v ≤  $\alpha$  return v  
             $\beta$  = min( $\beta$ , v)  
        return v
```

Call MiniMax(initialState, 0, true)

MiniMax Algorithm + Alpha/Beta Pruning

Fig 5.5, pg 153

```
function MiniMax(node, d, max,  $\alpha$ ,  $\beta$ ):
```

```
    if terminal(node) or d > maxD  
        return eval(node)
```

```
    if max
```

```
        v =  $-\infty$ 
```

```
        for each child of node
```

```
            v = max(v, MiniMax(child, d+1, false,  $\alpha$ ,  $\beta$ ))
```

```
            if v >  $\beta$  return v
```

```
             $\alpha$  = max( $\alpha$ , v)
```

```
        return v
```

```
    else
```

```
        v =  $+\infty$ 
```

```
        for each child of node
```

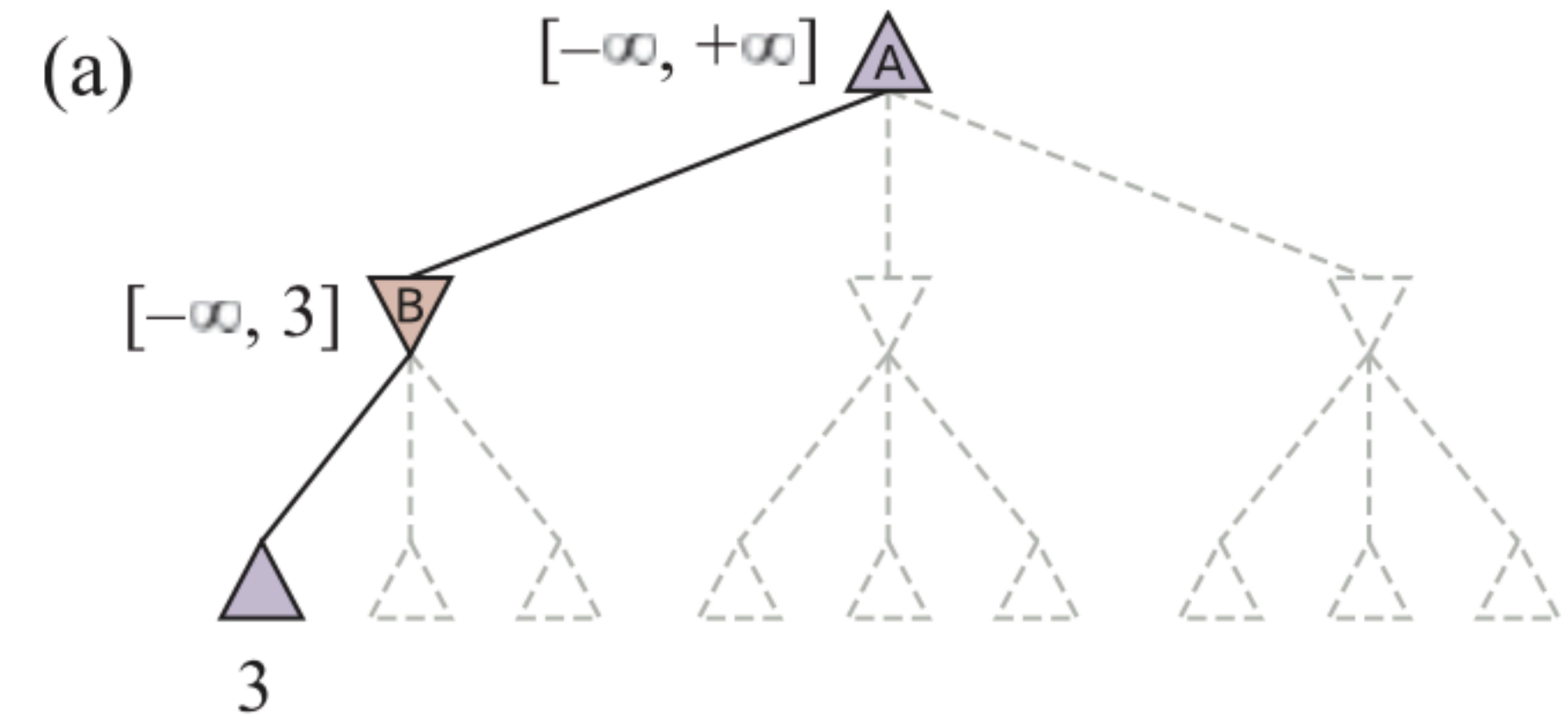
```
            v = min(v, MiniMax(child, d+1, true,  $\alpha$ ,  $\beta$ ))
```

```
            if v  $\leq$   $\alpha$  return v
```

```
             $\beta$  = min( $\beta$ , v)
```

```
        return v
```

```
Call MiniMax(initialState, 0, true)
```



MiniMax Algorithm + Alpha/Beta Pruning

Fig 5.5, pg 153

function MiniMax(*node*, *d*, *max*, α , β):

if terminal(*node*) or *d* > maxD

return eval(*node*)

if *max*

$v = -\infty$

for each child of *node*

$v = \max(v, \text{MiniMax}(\text{child}, d+1, \text{false}, \alpha, \beta))$

if $v > \beta$ return v

$\alpha = \max(\alpha, v)$

return v

else

$v = +\infty$

for each child of *node*

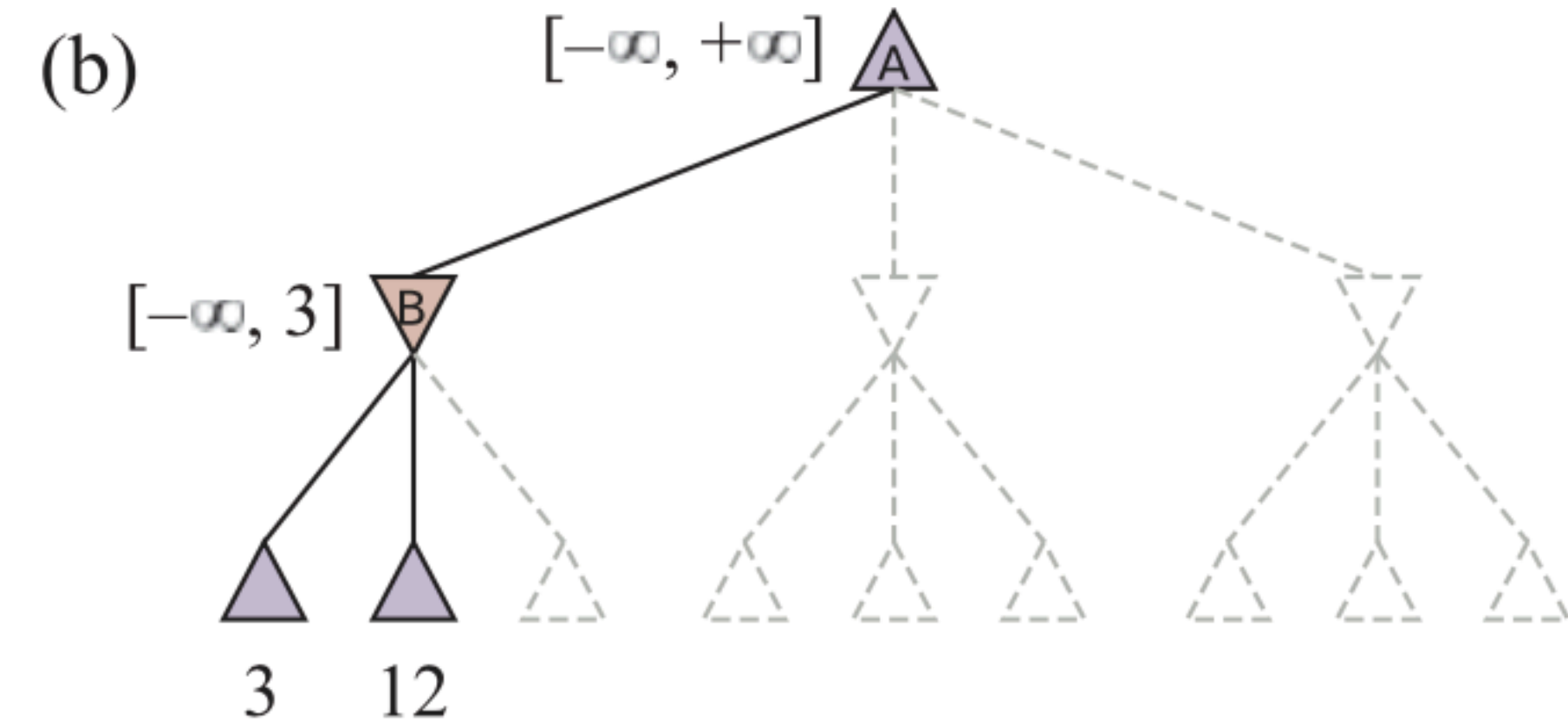
$v = \min(v, \text{MiniMax}(\text{child}, d+1, \text{true}, \alpha, \beta))$

if $v \leq \alpha$ return v

$\beta = \min(\beta, v)$

return v

Call MiniMax(initialState, 0, true)



MiniMax Algorithm + Alpha/Beta Pruning

Fig 5.5, pg 153

```
function MiniMax(node, d, max,  $\alpha$ ,  $\beta$ ):
```

```
    if terminal(node) or d > maxD
```

```
        return eval(node)
```

```
    if max
```

```
        v =  $-\infty$ 
```

```
        for each child of node
```

```
            v = max(v, MiniMax(child, d+1, false,  $\alpha$ ,  $\beta$ ))
```

```
            if v >  $\beta$  return v
```

```
             $\alpha$  = max( $\alpha$ , v)
```

```
        return v
```

```
    else
```

```
        v =  $+\infty$ 
```

```
        for each child of node
```

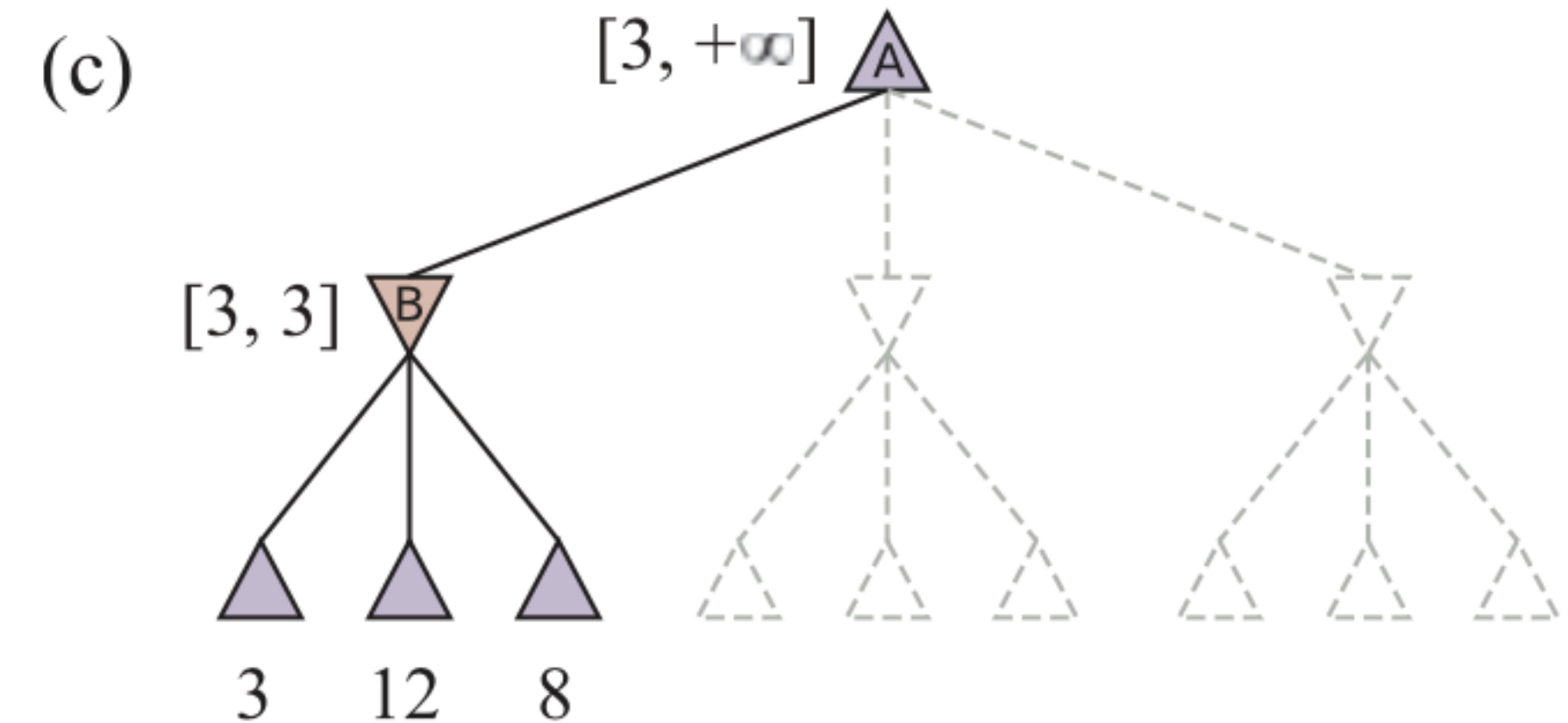
```
            v = min(v, MiniMax(child, d+1, true,  $\alpha$ ,  $\beta$ ))
```

```
            if v  $\leq$   $\alpha$  return v
```

```
             $\beta$  = min( $\beta$ , v)
```

```
        return v
```

```
Call MiniMax(initialState, 0, true)
```



MiniMax Algorithm + Alpha/Beta Pruning

Fig 5.5, pg 153

```
function MiniMax(node, d, max,  $\alpha$ ,  $\beta$ ):
```

```
    if terminal(node) or d > maxD  
        return eval(node)
```

```
    if max
```

```
        v =  $-\infty$ 
```

```
        for each child of node
```

```
            v = max(v, MiniMax(child, d+1, false,  $\alpha$ ,  $\beta$ ))
```

```
            if v >  $\beta$  return v
```

```
             $\alpha$  = max( $\alpha$ , v)
```

```
        return v
```

```
    else
```

```
        v =  $+\infty$ 
```

```
        for each child of node
```

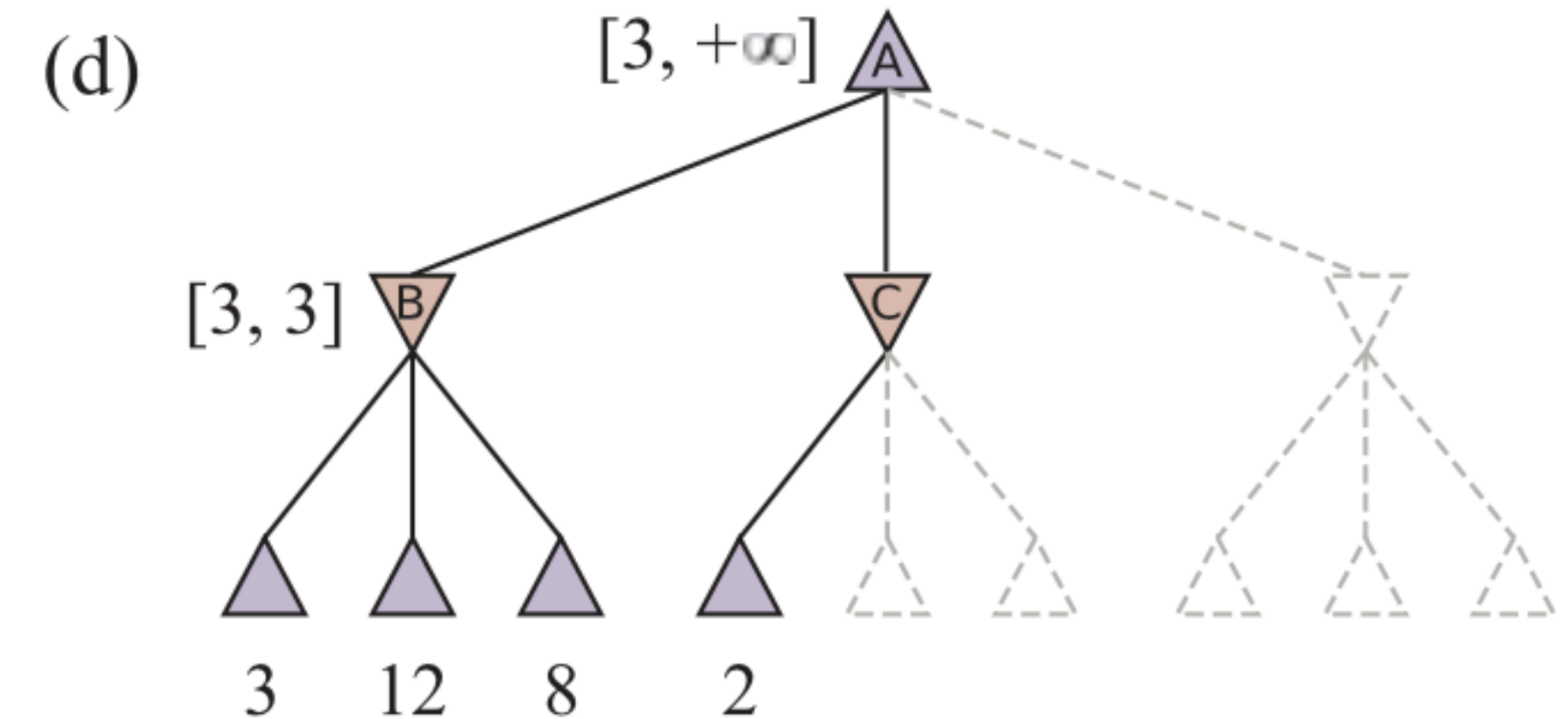
```
            v = min(v, MiniMax(child, d+1, true,  $\alpha$ ,  $\beta$ ))
```

```
            if v  $\leq$   $\alpha$  return v
```

```
             $\beta$  = min( $\beta$ , v)
```

```
        return v
```

```
Call MiniMax(initialState, 0, true)
```

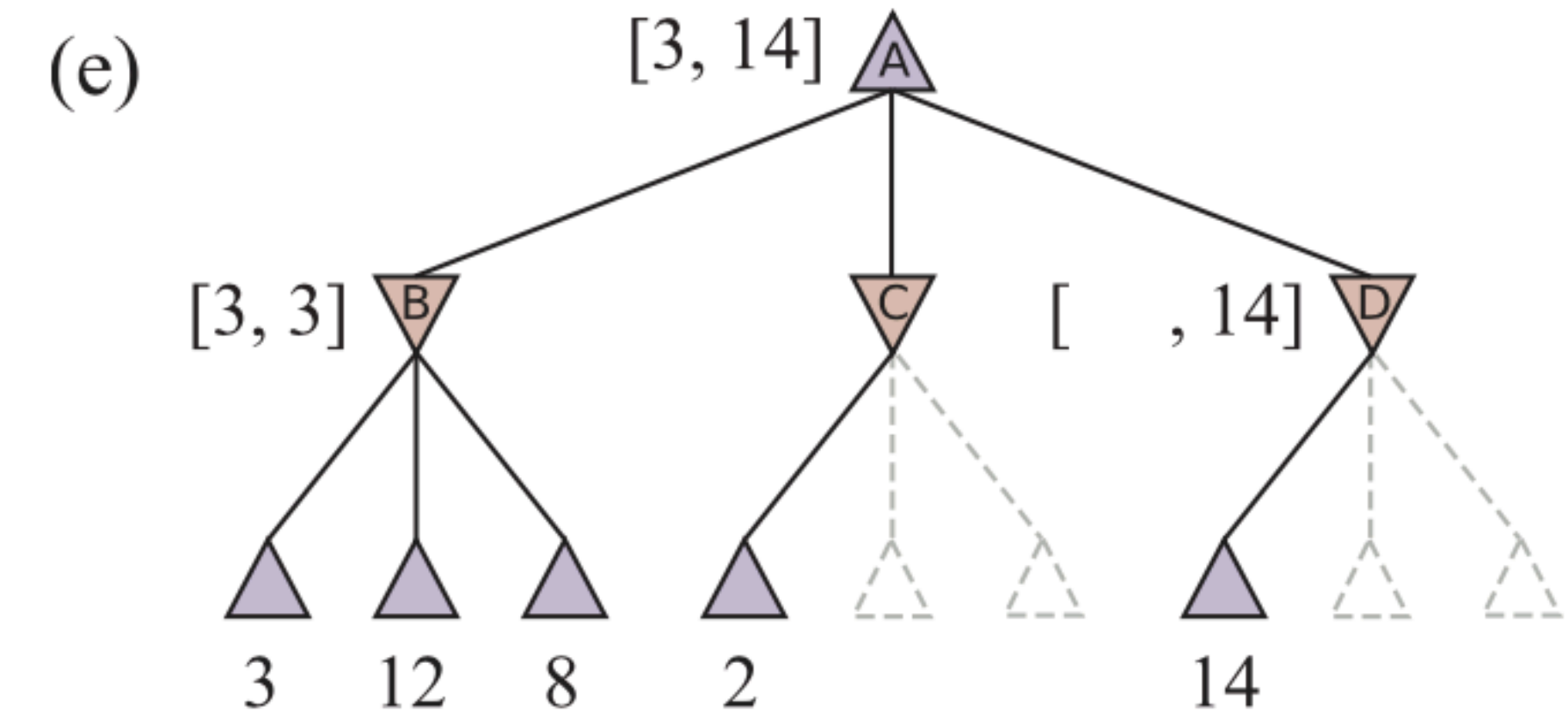


MiniMax Algorithm + Alpha/Beta Pruning

Fig 5.5, pg 153

```
function MiniMax(node, d, max,  $\alpha$ ,  $\beta$ ):  
    if terminal(node) or d > maxD  
        return eval(node)  
    if max  
        v =  $-\infty$   
        for each child of node  
            v = max(v, MiniMax(child, d+1, false,  $\alpha$ ,  $\beta$ )  
            if v >  $\beta$  return v  
             $\alpha$  = max( $\alpha$ , v)  
        return v  
    else  
        v =  $+\infty$   
        for each child of node  
            v = min(v, MiniMax(child, d+1, true,  $\alpha$ ,  $\beta$ )  
            if v  $\leq$   $\alpha$  return v  
             $\beta$  = min( $\beta$ , v)  
        return v
```

Call MiniMax(initialState, 0, true)

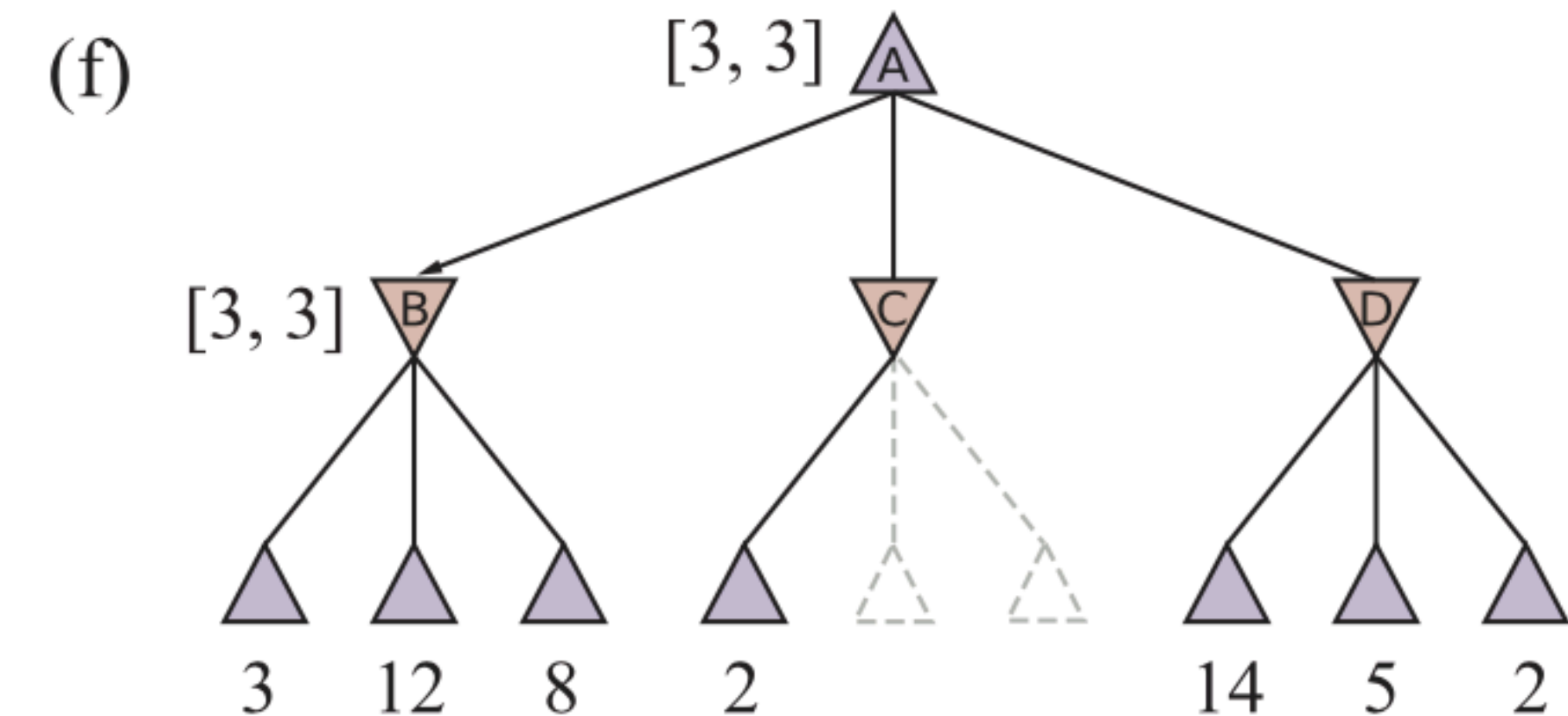


MiniMax Algorithm + Alpha/Beta Pruning

Fig 5.5, pg 153

```
function MiniMax(node, d, max,  $\alpha$ ,  $\beta$ ):  
    if terminal(node) or d > maxD  
        return eval(node)  
    if max  
        v =  $-\infty$   
        for each child of node  
            v = max(v, MiniMax(child, d+1, false,  $\alpha$ ,  $\beta$ )  
            if v >  $\beta$  return v  
             $\alpha$  = max( $\alpha$ , v)  
        return v  
    else  
        v =  $+\infty$   
        for each child of node  
            v = min(v, MiniMax(child, d+1, true,  $\alpha$ ,  $\beta$ )  
            if v  $\leq$   $\alpha$  return v  
             $\beta$  = min( $\beta$ , v)  
        return v
```

Call MiniMax(initialState, 0, true)



Monte Carlo Tree Search

Monte Carlo Tree Search

Introduction

- **Monte Carlo tree search (MCTS)** is a heuristic search algorithm
- Used in in game play.
 - MCTS was introduced in 2006 for computer Go
 - Used in board games like chess and shogi
 - Used in games with incomplete information such as bridge and poker
 - Used in real-time video games such as Total War: Rome II's implementation of the high level campaign AI.



Monte Carlo Tree Search

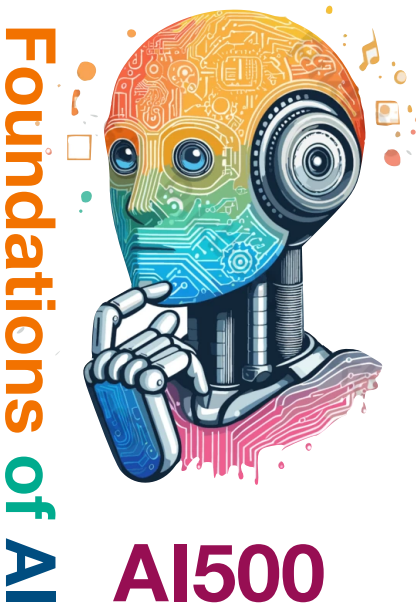
Principle of Operation



- The focus of MCTS is on the **analysis** of the **most promising** moves.
- It expands the search tree based on **random sampling** of the search space.
- The application of Monte Carlo tree search in games is based on many **playouts** also called **roll-outs**.
- In each playout, the game is played out to the very end by **selecting moves at random**.
- The final game result of each playout is then used to **weight the nodes** in the game tree so that better nodes are more likely to be chosen in future playouts.

Monte Carlo Tree Search

Principle of Operation

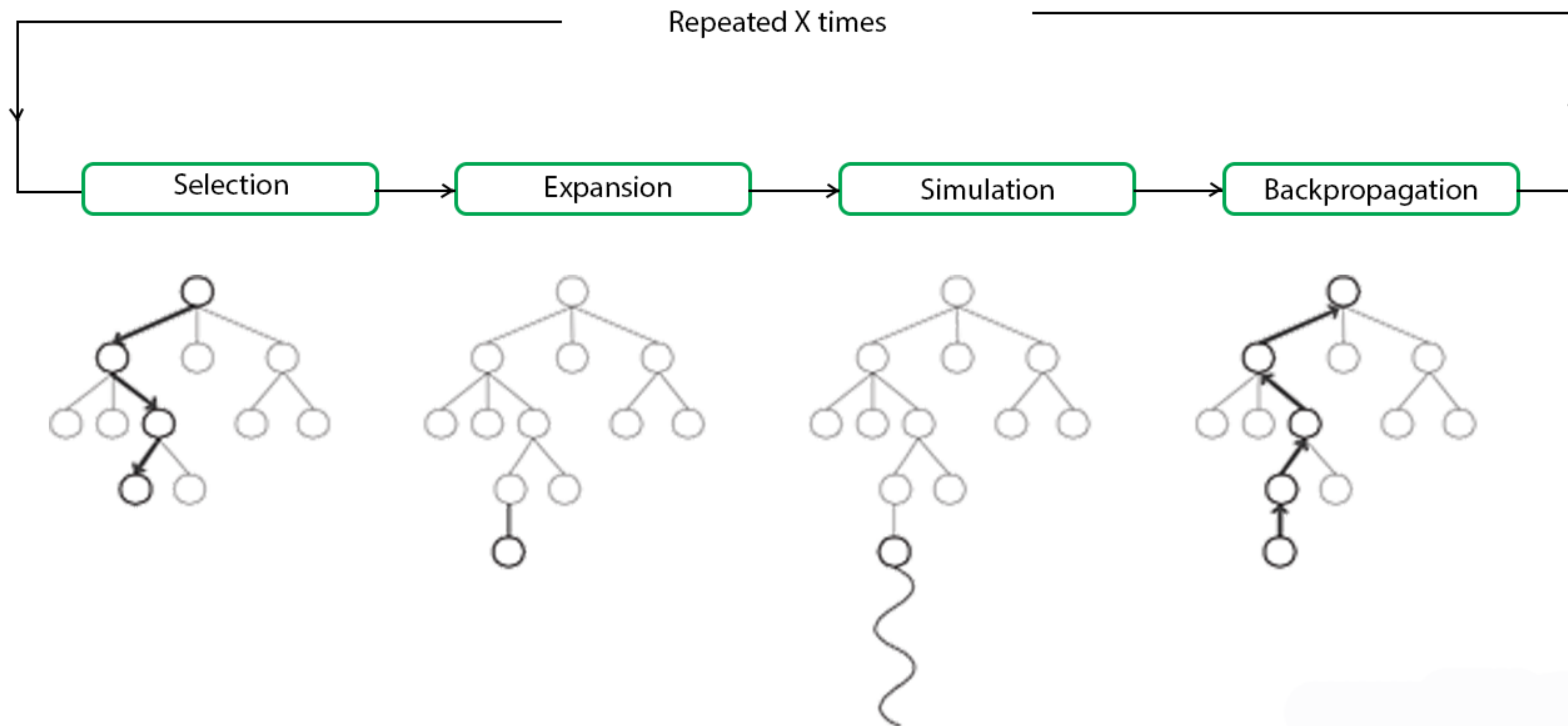


- The most basic way to use playouts is to apply the same number of playouts after each legal move of the current player, then choose the move which led to the most victories.
- The efficiency of this method—called Pure Monte Carlo Game Search—often increases with time as more playouts are assigned to the moves that have frequently resulted in the current player's victory according to previous playouts.

Monte Carlo Tree Search

Principle of Operation

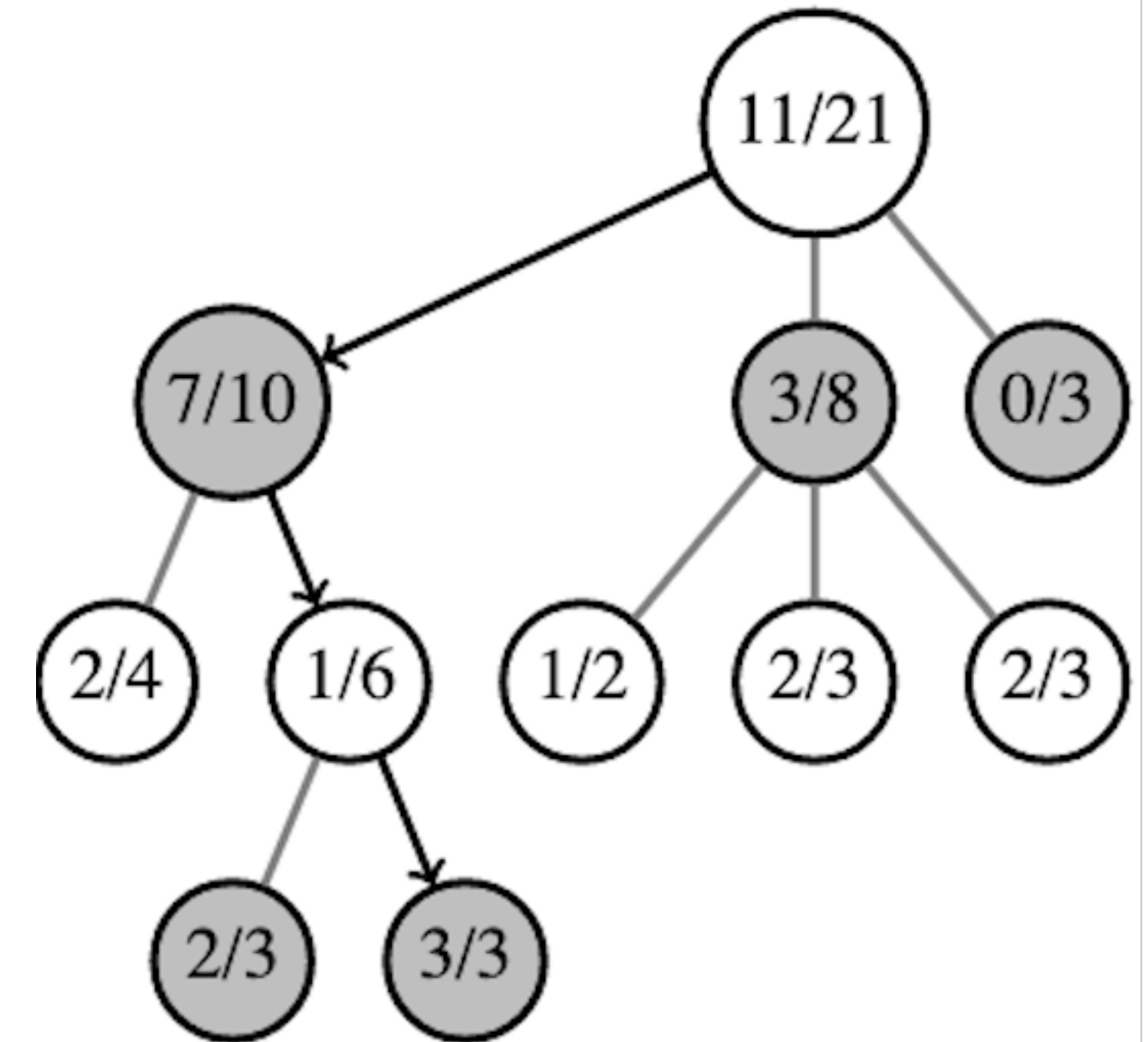
- Each round of Monte Carlo tree search consists of four steps:



Monte Carlo Tree Search

Selection

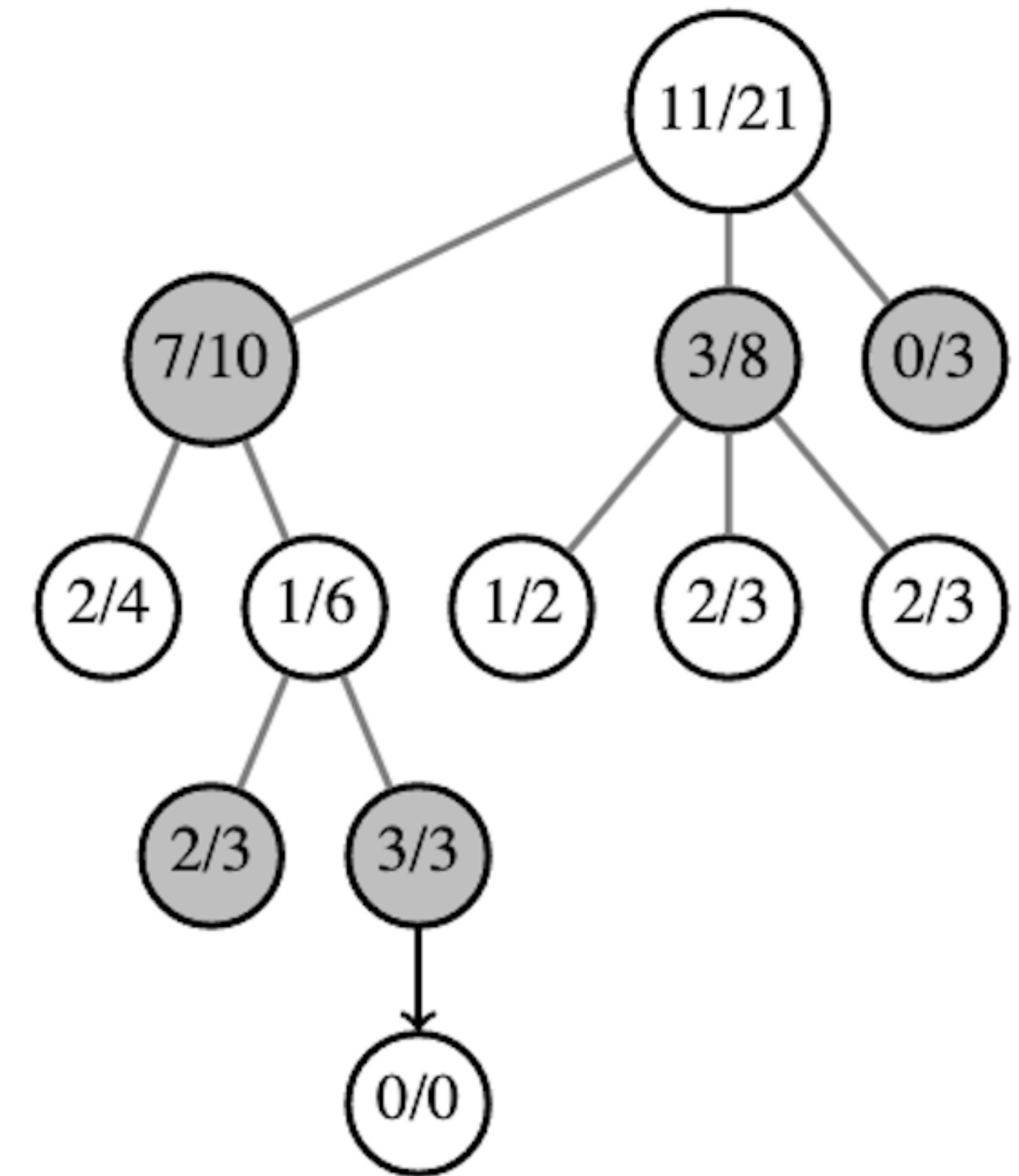
- A **selection** function is applied recursively until a leaf node is reached.
- The function attempts to balance **exploitation** and **exploration**.
- **Exploitation**: Select moves that lead to the best results so far
- **Exploration**: Select less promising moves to remove uncertainty of them.



Monte Carlo Tree Search

Expansion

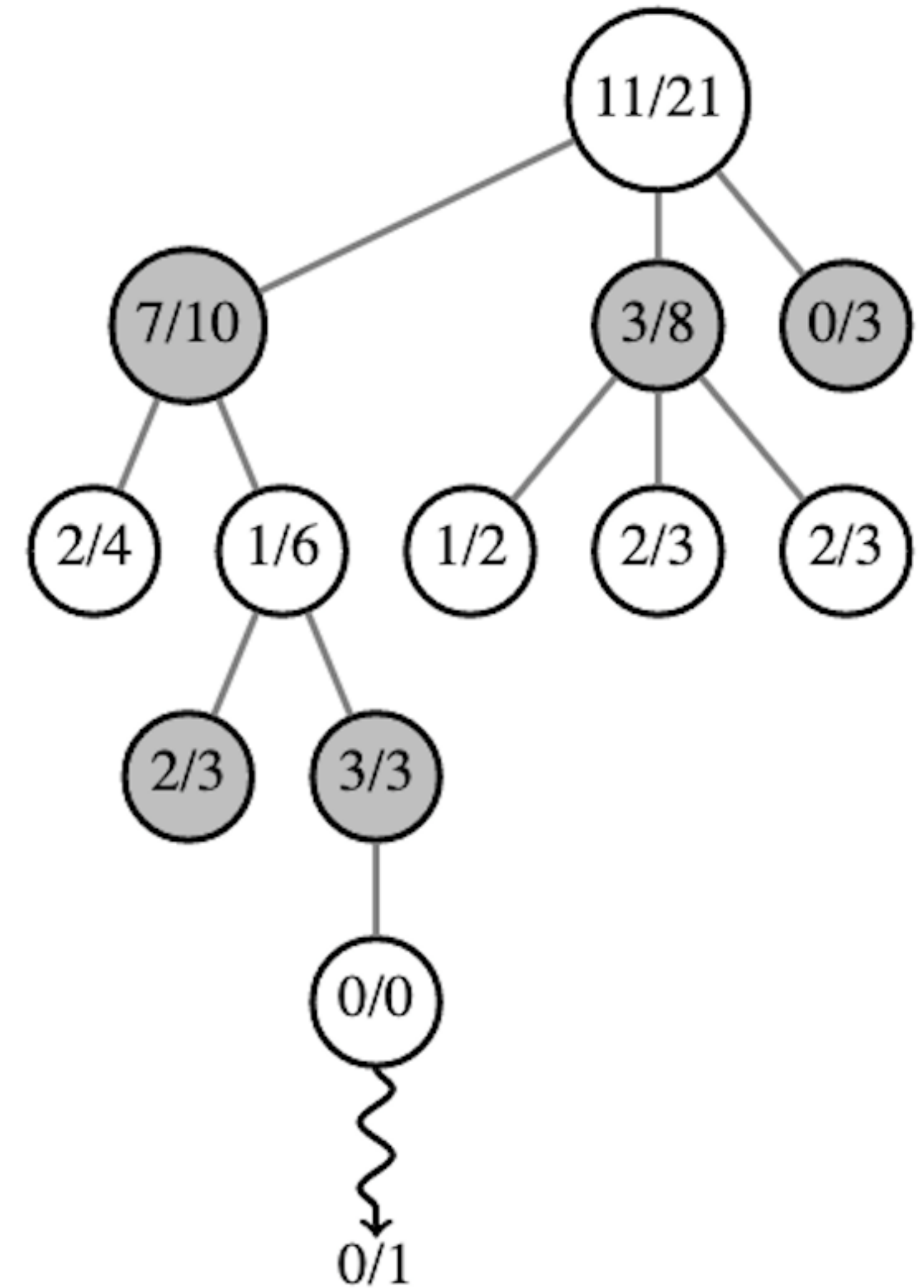
- One or more moves/states are created.



Monte Carlo Tree Search

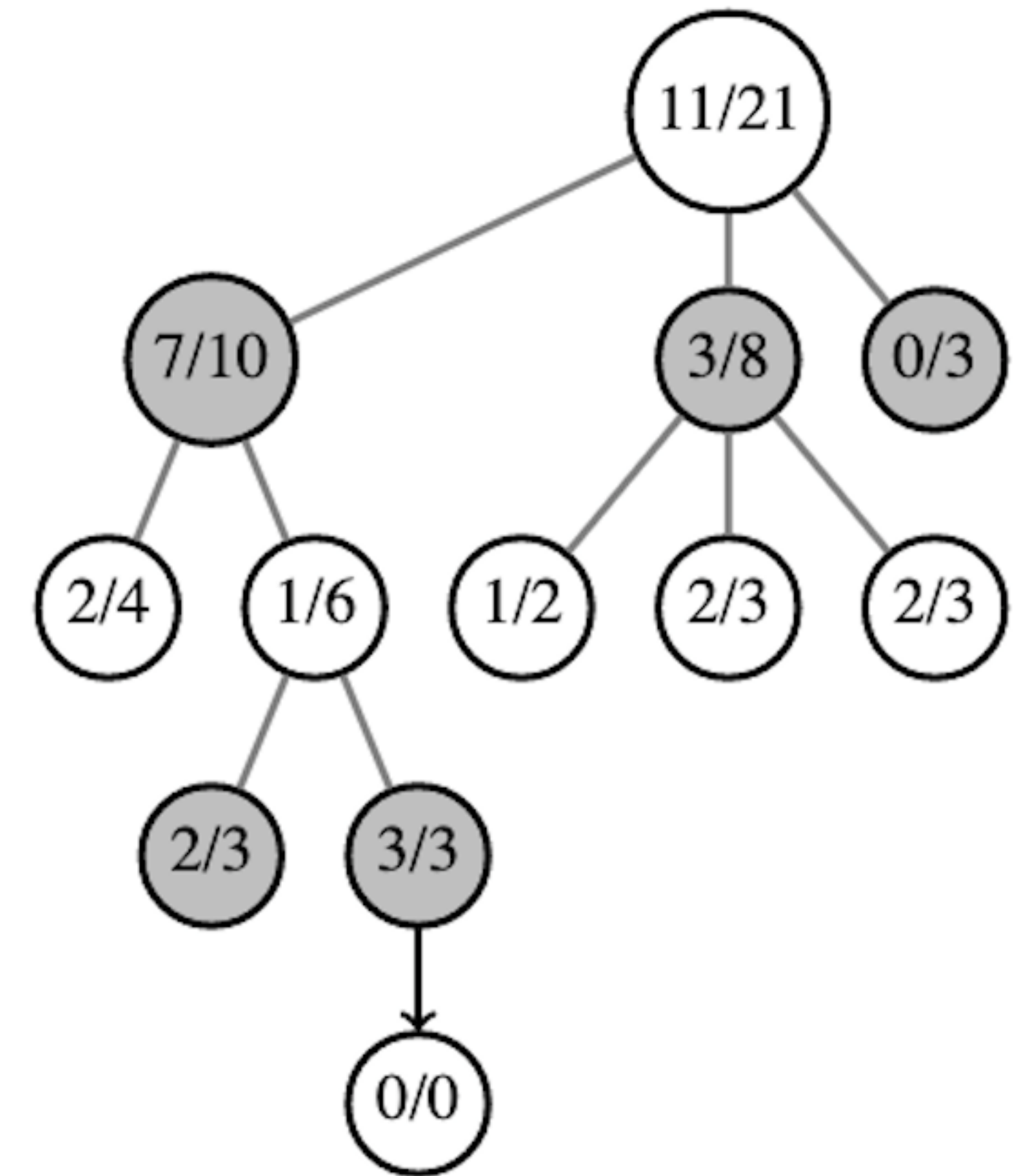
Simulation

- One simulation game is played.
- Actions are selected at **random** until game ends.
- Use heuristic knowledge to bias selection to actions that look more promising.



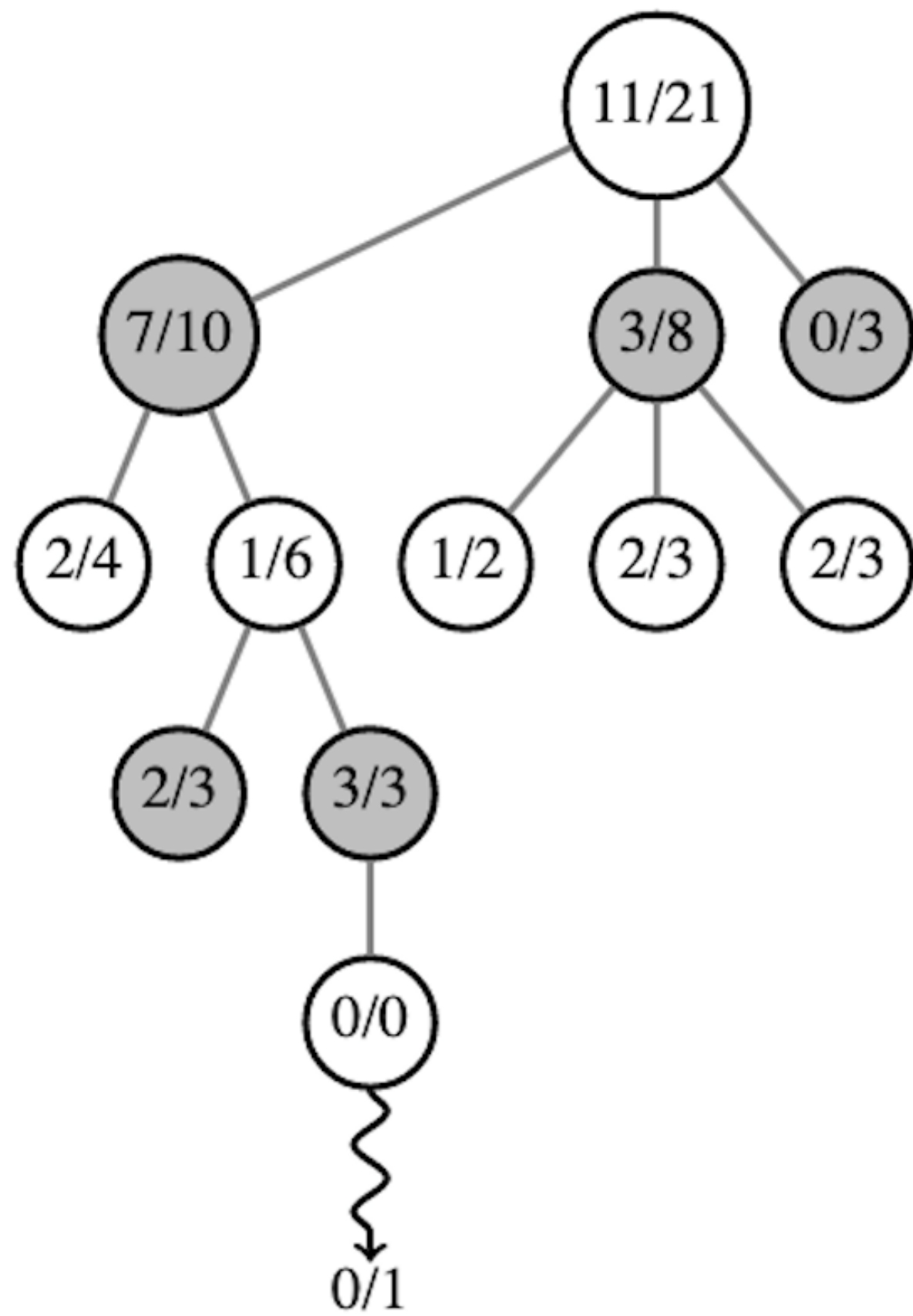
Backpropagation

- After reaching the end of the simulated game, update each tree node that was traversed during that game.
- The visit counts are increased and the win/loss ratio is modified according to the outcome.

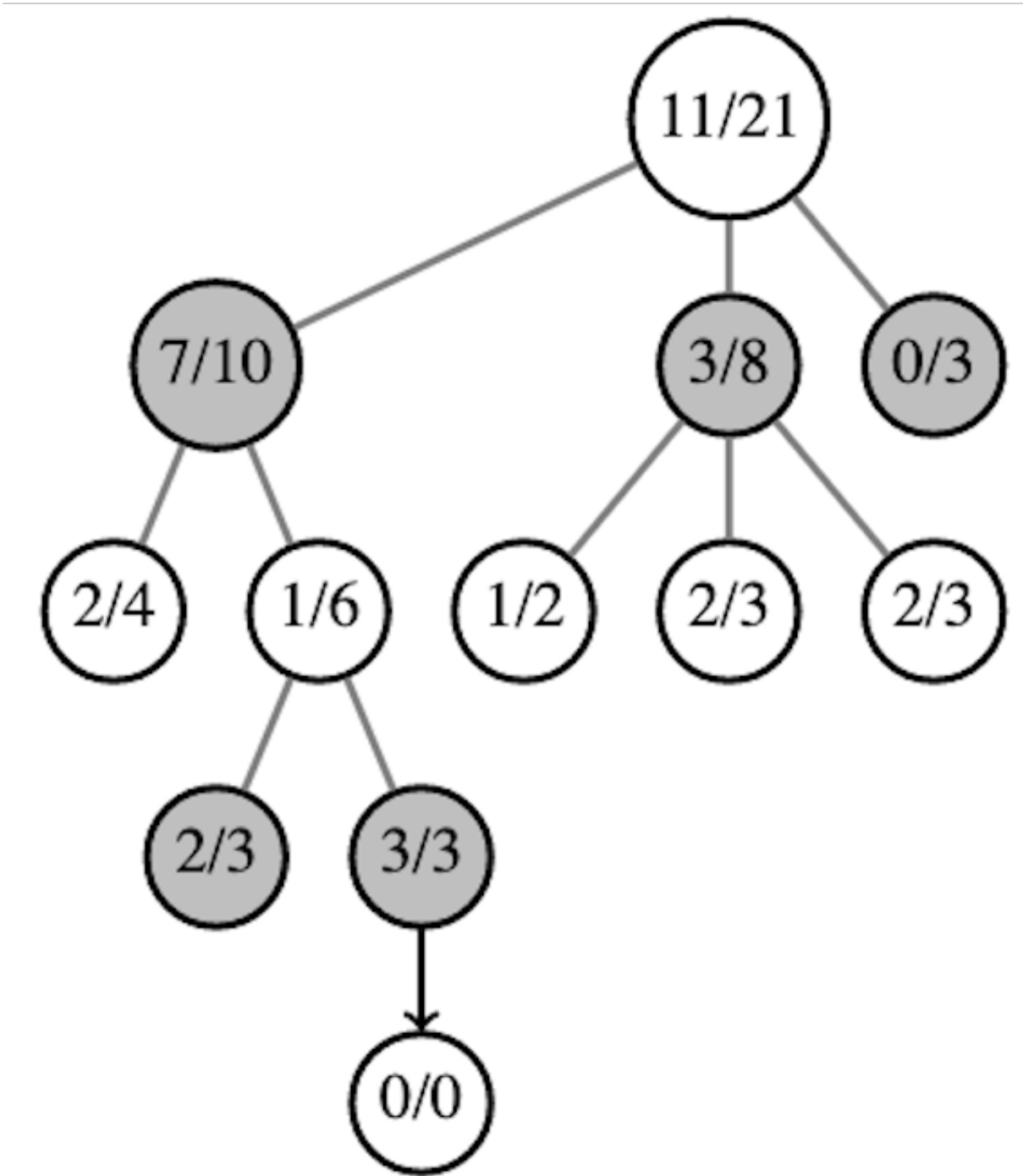


Monte Carlo Tree Search

Backpropagation

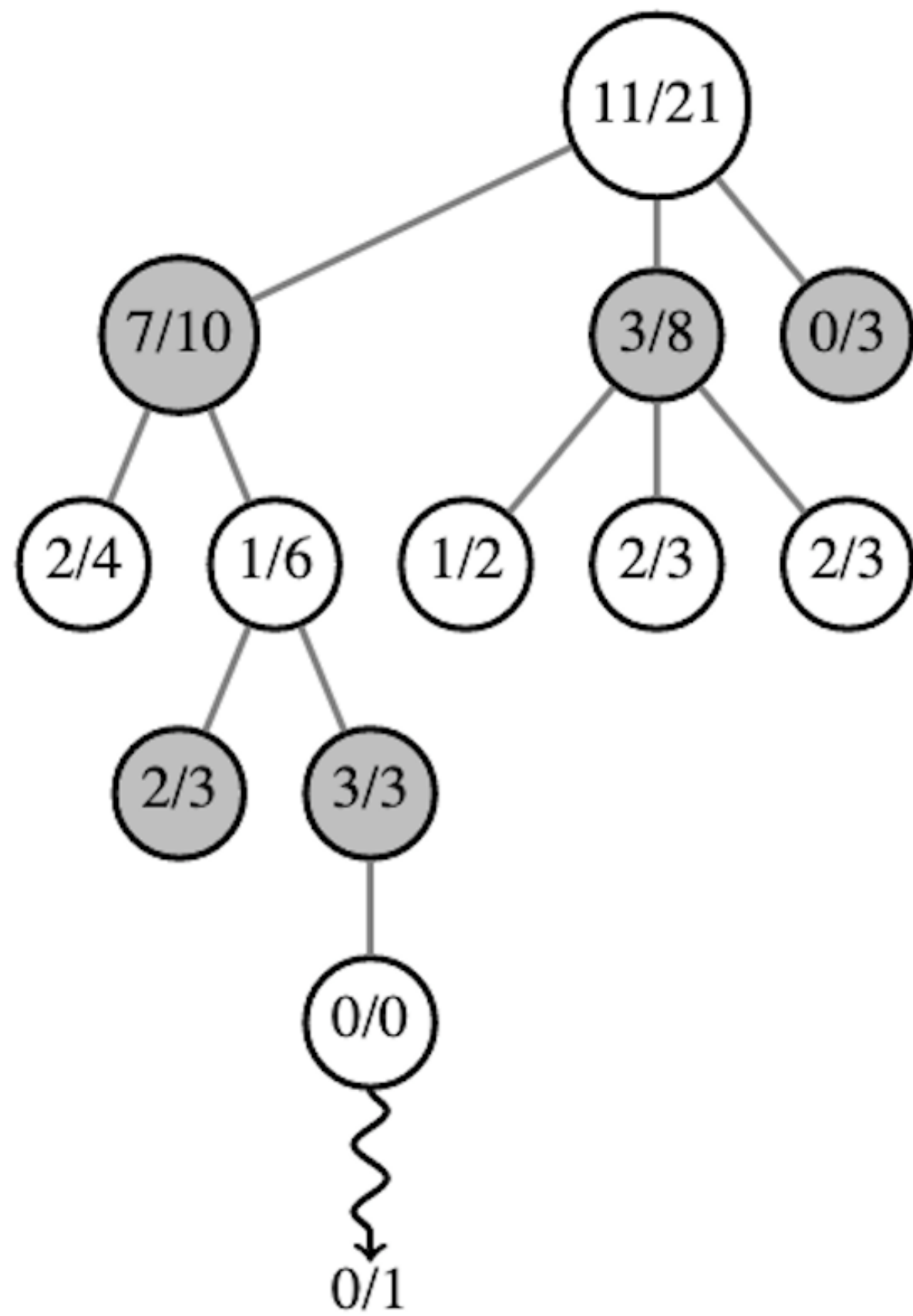


- Each node shows the **ratio** of wins to total playouts from that point in the game tree for the player that node represents.
- Black is about to move.
- The root node shows there are 11 wins out of 21 playouts for white from this position so far.
- It complements the total of 10/21 black wins shown along the three black nodes under it, each of which represents a possible black move.



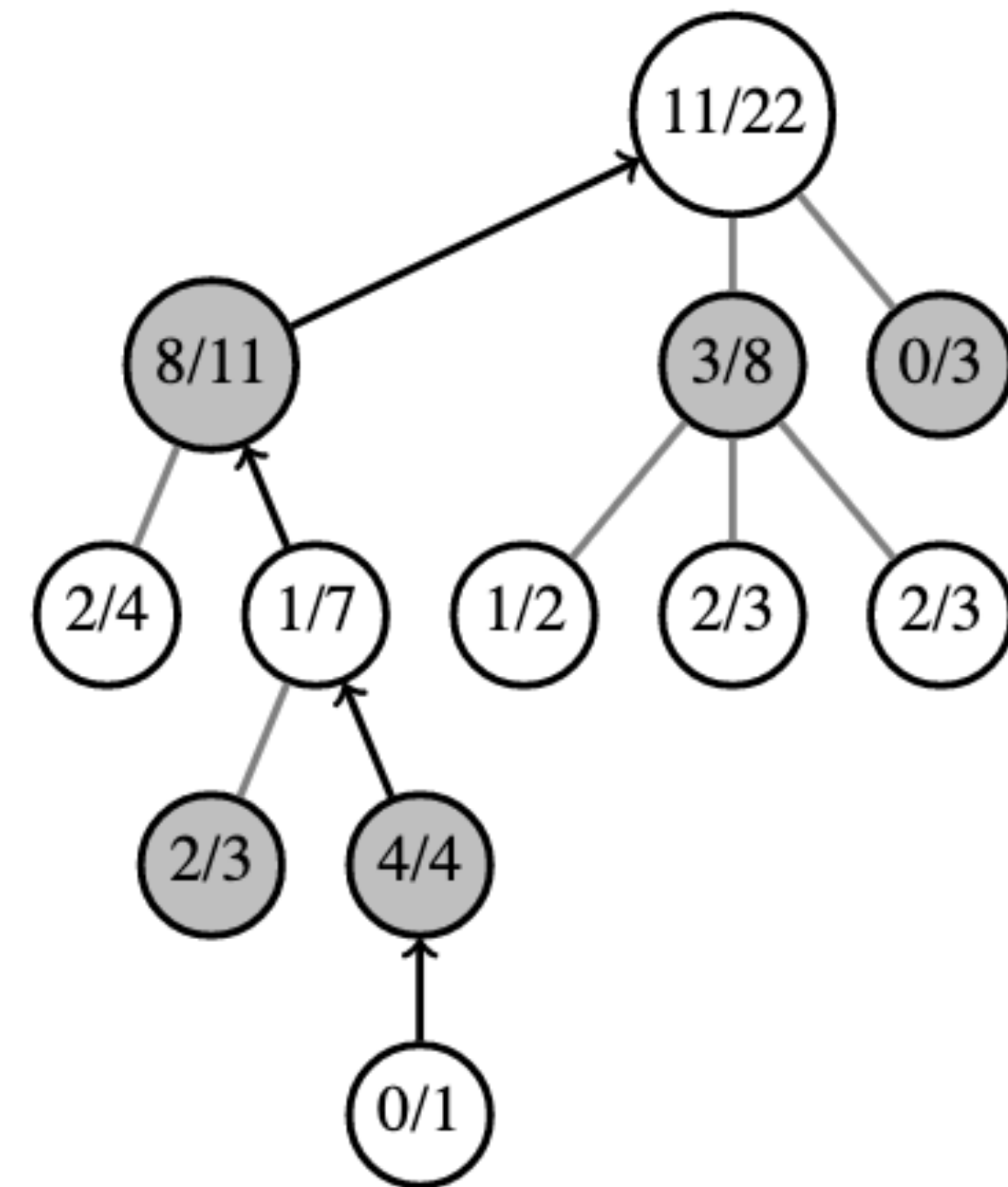
Monte Carlo Tree Search

Backpropagation



- All nodes along selection path increase their simulation count (denominator)
- If black wins, increment the win counts of all black nodes along the selection path (numerator)
- If white wins, increment the win counts of all white nodes
- Rounds of search are repeated as long as the time allotted to a move remains.
- The move with the most simulations made (i.e. the highest denominator) is chosen as the final answer.

Backpropagation

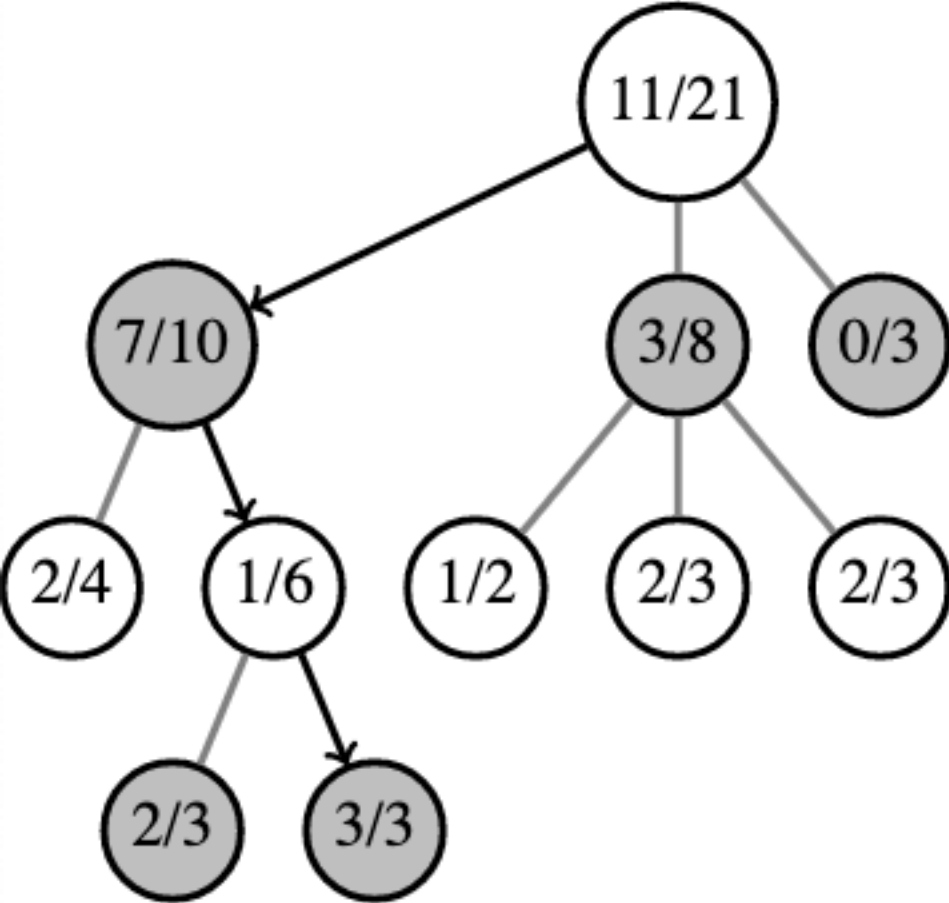


Monte Carlo Tree Search

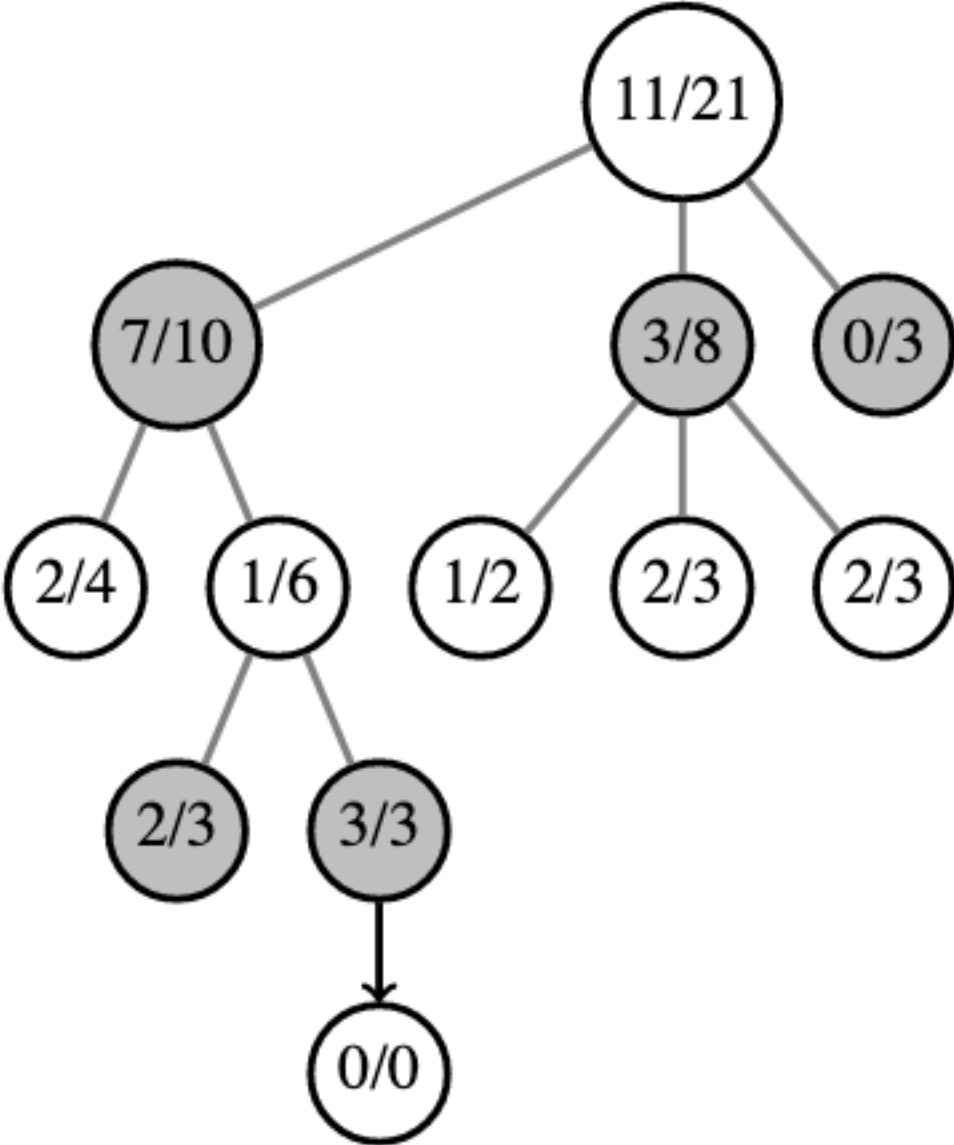
Putting it all together



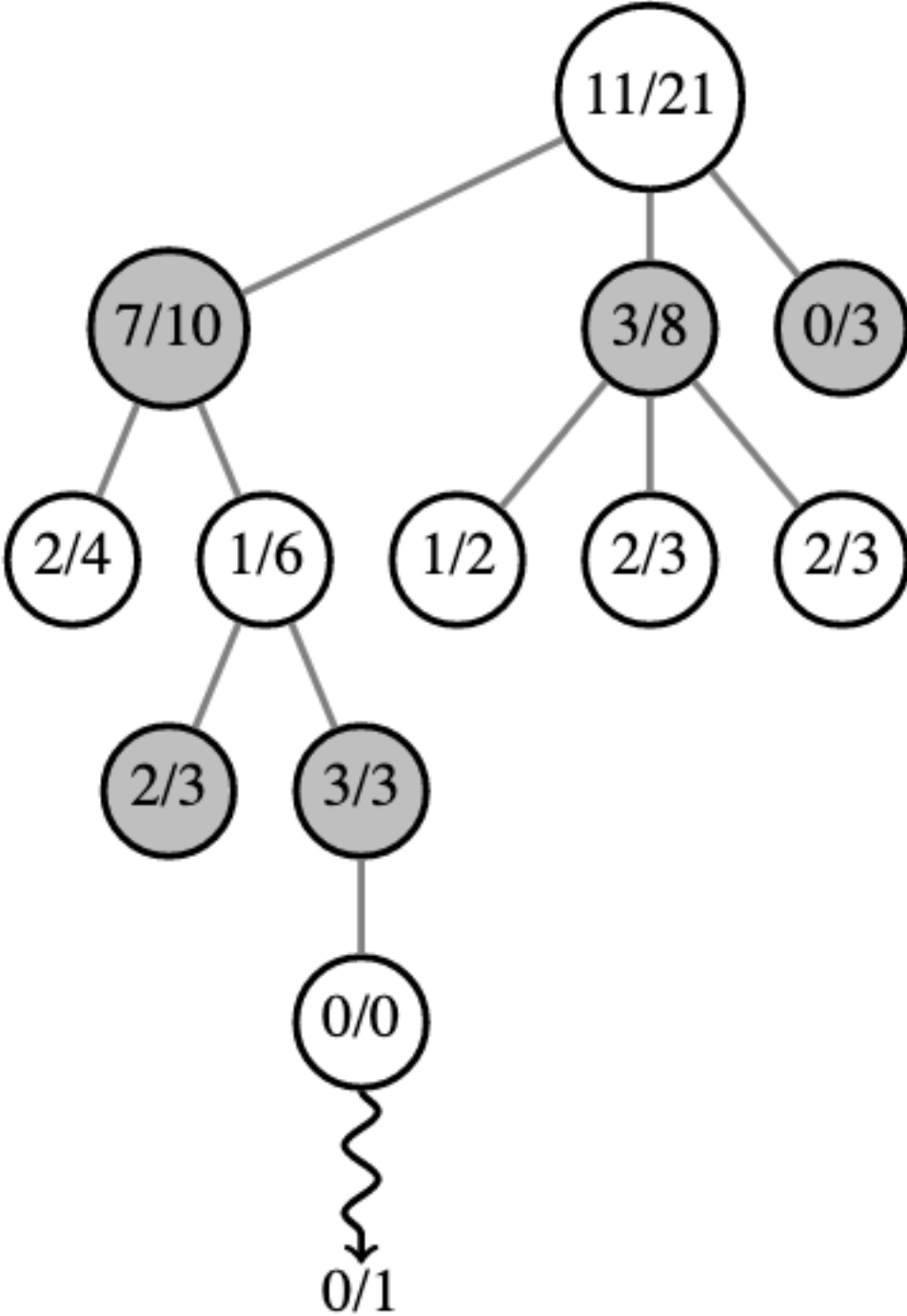
Selection



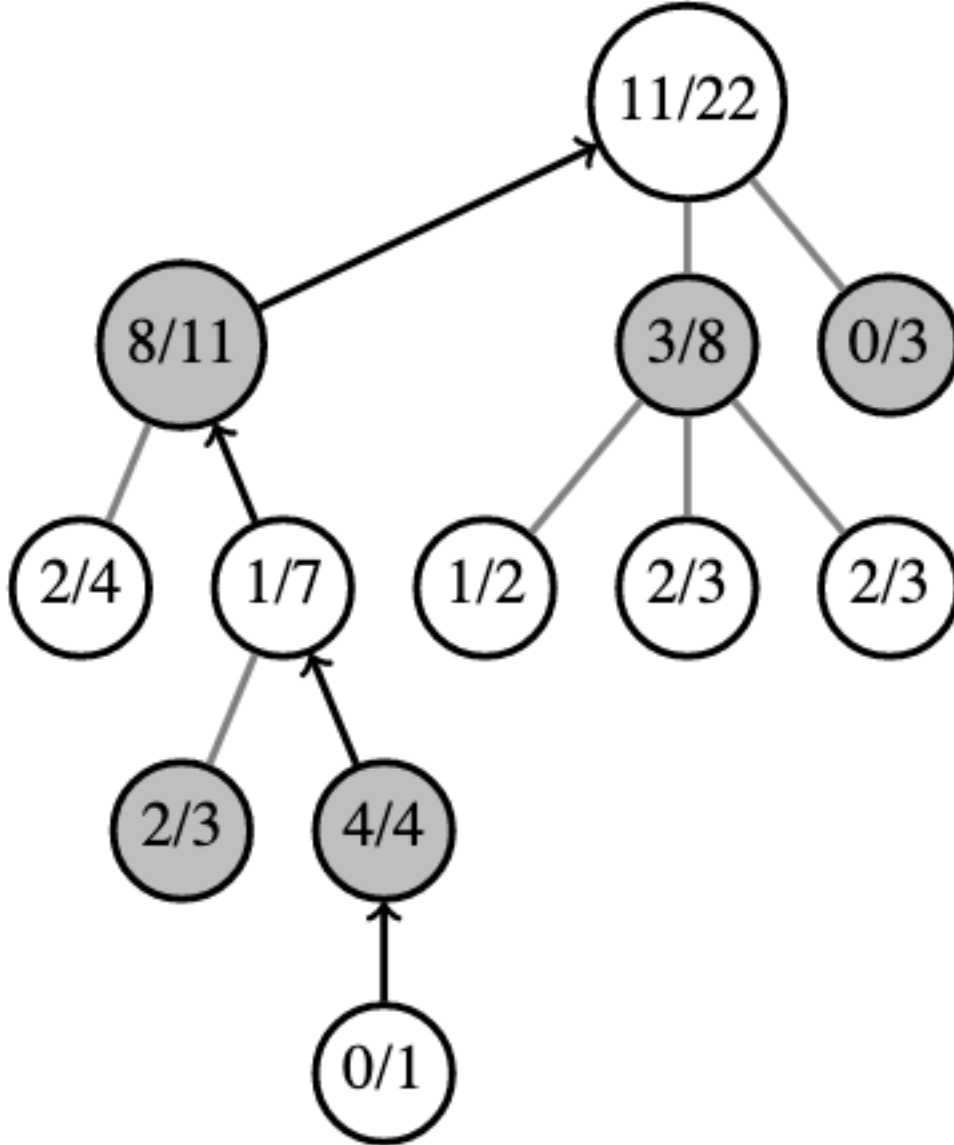
Expansion



Simulation



Backpropagation



Monte Carlo Tree Search

Upper Confidence bounds applied to Trees (UCT)

- For a child i of node s , define

$$\text{UCB}(s, i) = \hat{Q}(s, i) + c \sqrt{\frac{\ln N(s)}{n(s, i)}}$$

- $N(s)$: number of times node s is visited
- $N(s, i) = n_i$: number of times child i is visited
- $W(s, i) = \omega_i$: cumulative reward
- $\hat{Q}(s, i) = \frac{W(s, i)}{N(s, i)}$

- Tree traversal
- Node Expansion
- Rollout (random simulation)
- Backpropagation

Next ...

- Quiz 1
- Unsupervised Learning and Decision Trees

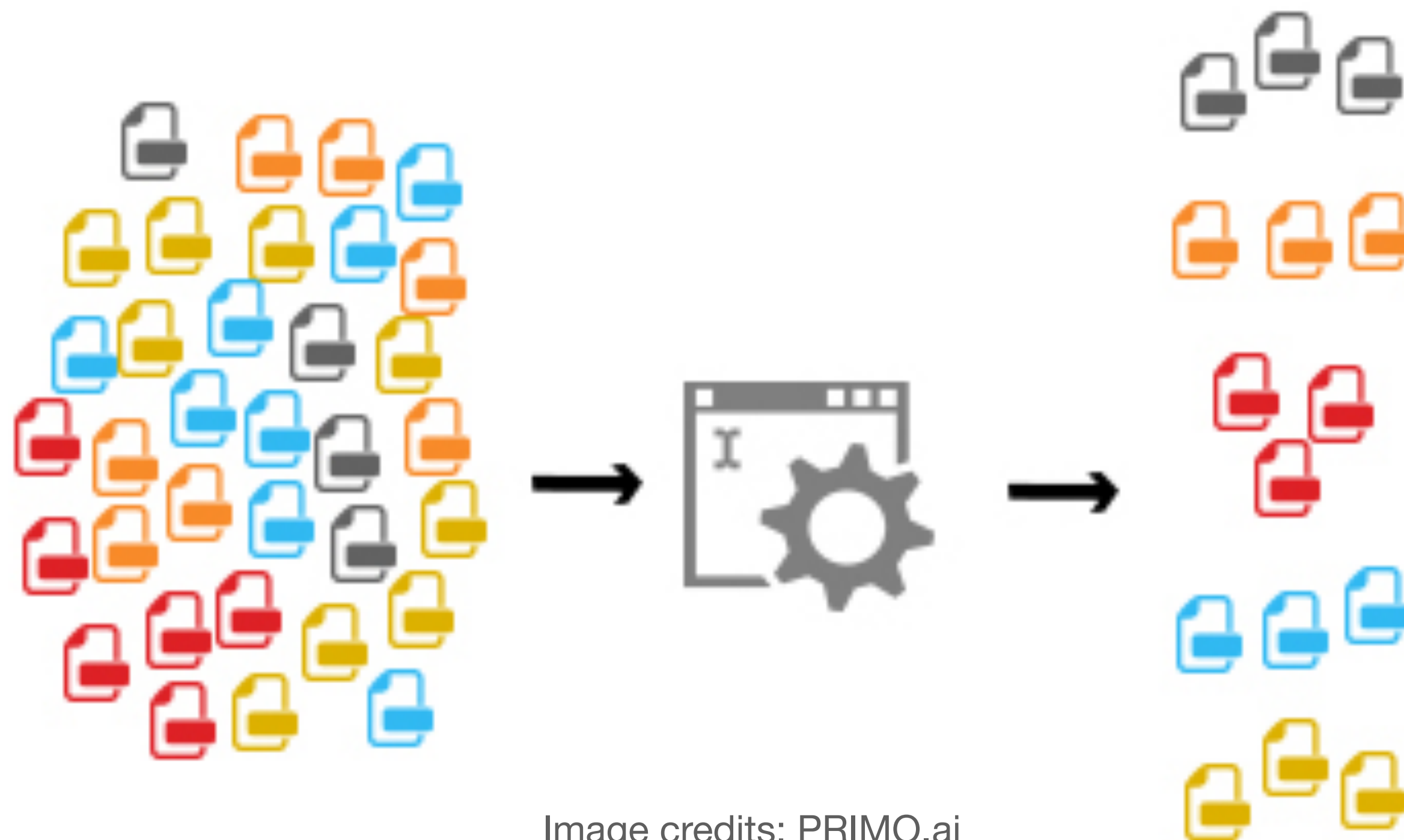


Image credits: PRIMO.ai